Gabriel Reimberg, Henrique Marques e Murilo Oliveira

Sistema de Votação Eletrônica em Blockchain Privada com Esquema Commit–Reveal: Prova de Conceito em Assembleias Condominiais

São Paulo - Brasil

2025

Gabriel Reimberg, Henrique Marques e Murilo Oliveira

Sistema de Votação Eletrônica em Blockchain Privada com Esquema Commit–Reveal: Prova de Conceito em Assembleias Condominiais

Monografia apresentada na disciplina Trabalho de Conclusão de Curso, como parte dos requisitos para obtenção do título de Bacharel em Ciência da Computação.

Centro Universitário Senac - Santo Amaro
Bacharelado em Ciência da Computação

Orientador: Thyago Quintas

São Paulo - Brasil
2025

Dedicamos este trabalho aos nossos pais e familiares, pelo amor, apoio incondicional e pela confiança que sempre depositaram em nós ao longo desta caminhada. Àqueles que partilharam conosco noites em claro, dúvidas, incentivos e palavras de motivação, ajudando-nos a seguir em frente mesmo nos momentos mais desafiadores. Estendemos esta dedicatória também aos amigos, colegas de classe e a todos que, de alguma forma, contribuíram para a nossa formação pessoal e acadêmica e para a concretização deste trabalho.

"Per aspera ad astra"
(Lúcio Aneu Sêneca)

Resumo

Este trabalho desenvolve e avalia uma prova de conceito de sistema de votação eletrônica baseada em blockchain privada, aplicada ao contexto de assembleias condominiais. O objetivo central é demonstrar a viabilidade de sigilo do voto com apuração pública e auditável, reduzindo elos fracos de confiança e preservando a rastreabilidade do processo decisório por meio de simulações controladas. A arquitetura separa responsabilidades entre componentes *off-chain* (emissor de credenciais e orquestração) e contratos *on-chain* com dois contratos. O primeiro implementa o fluxo *commit-reveal* com janelas temporais e eventos de domínio; o segundo trata da criação e transferências de token, quando necessário.

O compromisso do voto é calculado localmente por `keccak256` sobre a credencial derivada, a opção escolhida, um segredo e o endereço do votante, sendo verificado *on-chain*; a autenticidade das credenciais é assegurada por ECDSA. Esse desenho mantém o conteúdo do voto sigiloso até a revelação e permite recontagem independente apenas com dados da cadeia. O trabalho também discute o uso de *Proof of Stake* em ambiente privado, delimitando o escopo a segurança, transparência e auditabilidade em cenário controlado.

A implementação foi construída com Hardhat, preparada para *testnets*, com interface web e código de implantação. As simulações coletam métricas operacionais, como gás por votação, custo por voto, votos válidos por *ETH* e votos por segundo, e produzem artefatos de auditoria (eventos e leituras) capazes de reconstituir a linha do tempo do processo. Os resultados indicam o atendimento dos objetivos funcionais: o fluxo *commit-reveal* opera de forma íntegra, as janelas são respeitadas e a elegibilidade por NFT pode ser acoplada sem sacrificar auditabilidade e recontagem.

Palavras-chaves: blockchain; votação eletrônica; *commit-reveal*; privacidade; auditoria; *Proof of Stake*; transparência.

Abstract

This work develops and evaluates a proof of concept for an electronic voting system built on a private blockchain, applied to the context of condominium assemblies. The central objective is to demonstrate the feasibility of ballot secrecy with a public, auditable tally, reducing weak trust links and preserving the traceability of the decision-making process through controlled simulations. The architecture separates responsibilities between *off-chain* components (credential issuer and orchestration) and *on-chain* contracts comprising two parts. The first implements the *commit-reveal* flow with timed windows and domain events; the second handles token issuance and transfers when required.

The vote commitment is computed locally via `keccak256` over the derived credential, the chosen option, a secret, and the voter's address, and it is verified *on-chain*; credential authenticity is ensured by ECDSA. This design keeps the ballot content secret until revelation and enables an independent recount using only on-chain data. The work also discusses the use of *Proof of Stake* in a private setting, delimiting the scope to security, transparency, and auditability under controlled conditions.

The implementation was built with Hardhat, prepared for *testnets*, with a web interface and deployment code. Simulations collect operational metrics such as gas per vote, cost per vote, valid votes per *ETH*, and votes per second and produce audit artifacts (event logs and state reads) capable of reconstructing the process timeline. The results indicate that the functional objectives are met: the *commit-reveal* flow operates correctly, time windows are enforced, and NFT-based eligibility can be attached without sacrificing auditability and recountability.

Keywords: blockchain; electronic voting; commit-reveal; privacy; auditing; Proof of Stake; transparency.

Lista de ilustrações

Figura 1 – Diagrama de Ballot Marking Devices 16
Figura 2 – Diagrama de uma criptografia assimétrica 22
Figura 3 – Funcionamento do ECC em prática 24
Figura 4 – Validação do esquema de compromisso e revelação 26
Figura 5 – Exemplo do "Efeito avalanche" em diferentes algoritmos de *hash* 27
Figura 6 – Uma sequência de blocos usados para criar uma blockchain 31
Figura 7 – Uma visão geral da arquitetura da blockchain 32
Figura 8 – Blockchain P2P Network 35
Figura 9 – Funcionamento do Smart Contract 36
Figura 10 – Comparativo de votações: tradicionais vs blockchain 37
Figura 11 – Diagrama de funcionamento 46
Figura 12 – Sequência do protocolo: credencial, `commitVote`, `revealVote` e apuração 48
Figura 13 – Máquina de estados da votação: fases e janelas temporais 49
Figura 14 – Geração automatizada de cenários com LLM e validação 50
Figura 15 – Modelo de contratos e principais operações 51
Figura 16 – Visão geral do sistema de votação em blockchain 59
Figura 17 – Métricas operacionais da simulação de votação 60
Figura 18 – Processo criptográfico do protocolo commit–reveal 61
Figura 19 – Linha do tempo das fases de commit e reveal 62
Figura 20 – Registro das transações on-chain da votação 63
Figura 21 – Distribuição da taxa de votos por segundo nas simulações 64
Figura 22 – Votos válidos por unidade de ether consumida em gás 65
Figura 23 – Custo unitário em ether por voto válido 65
Figura 24 – Consumo total de gás por instância de votação 66

Lista de tabelas

Tabela 1 – Comparação entre SHA-2 e SHA-3 28
Tabela 2 – Divisão de camadas da blockchain 33
Tabela 3 – Comparação resumida entre PoS e PoA 44
Tabela 4 – Métricas agregadas 64

Lista de abreviaturas e siglas

API	Application Programming Interface
BTC	Bitcoin
ECC	Criptografia de Curva Elíptica
ECDSA	Algoritmo de Assinatura Digital em Curva Elíptica
ERC	Ethereum Request for Comments
ETH	Ether (criptomoeda nativa da rede Ethereum)
EVM	Máquina Virtual Ethereum
JSON	JavaScript Object Notation
LLM	Modelos de linguagem de grande porte
NFT	Token Não Fungível
P2P	Peer-to-Peer
PBFT	Practical-Byzantine-Fault-Tolerance
PoA	Proof-of-Authority
PoC	Proof-of-Concept
PoS	Proof-of-Stake
PoW	Proof-of-Work
RSA	Rivest–Shamir–Adleman
SHA	Secure Hash Algorithm
SHS	Secure Hash Standard
TBNB	Testnet BNB

Sumário

1 INTRODUÇÃO . . . 12
1.1 Justificativa . . . 13
1.2 Objetivos . . . 14
1.2.1 Objetivo Principal . . . 14
1.2.2 Objetivos Específicos . . . 15

2 FUNDAMENTAÇÃO TEÓRICA . . . 16
2.1 Sistemas de Votação . . . 16
2.1.1 Meios de Votação . . . 16
2.1.1.1 Votação Digital . . . 17
2.1.1.2 Segurança em Votações Online . . . 18
2.1.2 Votação Condominial . . . 18
2.2 Dados sintéticos e LLMs . . . 19
2.2.1 Eixos de qualidade para dados sintéticos . . . 19
2.2.2 Métodos de geração e papel dos LLMs . . . 19
2.2.3 Modelos de linguagem de grande porte (LLMs) . . . 20
2.2.3.1 *Gemma*: características e aplicabilidade . . . 20
2.3 Criptografia . . . 21
2.3.1 Algoritmos Assimétricos . . . 22
2.3.1.1 Criptografia de Curva Elíptica (ECC) . . . 23
2.3.1.2 Algoritmo de Assinatura Digital de Curva Elíptica (ECDSA) . . . 23
2.3.2 Esquema de Compromisso e Revelação . . . 25
2.3.2.1 Esquema de Compromisso e Revelação . . . 26
2.3.3 Funções de *Hash* . . . 26
2.3.3.1 *Secure Hash Algorithm* (SHA) . . . 27
2.3.4 Keccak-256 . . . 28
2.4 Tecnologia Blockchain . . . 29
2.4.1 A Blockchain . . . 30
2.4.2 Tipos de Blockchain . . . 31
2.4.3 Estrutura Arquitetural de uma Blockchain . . . 31
2.4.3.1 Divisão de camadas . . . 32
2.4.3.2 Camada de Consenso e Rede da Blockchain . . . 32
2.4.3.3 Rede da Blockchain . . . 34
2.4.3.4 Camada de Dados da Blockchain . . . 35
2.4.3.5 Camada de Computação e Contratos Inteligentes (*Smart Contracts*) . . . 35
2.4.3.6 Sistemas de votação eletrônica usando blockchain . . . 37
2.4.4 Frameworks . . . 37
2.4.4.1 Ethereum . . . 38
2.4.4.2 Solidity . . . 39
2.4.4.3 Hardhat . . . 39
2.5 Trabalhos Relacionados . . . 39
2.5.1 Proposta de Modelo para Votação Eletrônica Utilizando Blockchain e Contratos Inteligentes . . . 39

2.5.2 *Blockchain-Based E-Voting System* . . . 40
2.5.3 *System Design & Implementation of Hyperledger Fabric E-Voting System: A Case Study of the Lebanese American University* . . . 40
2.5.4 Transforming the voting process integrating blockchain into e-voting . . . 41

3 DESENVOLVIMENTO . . . 42
3.1 Visão Geral . . . 42
3.2 Requisitos Funcionais e Não Funcionais . . . 42
3.2.1 Requisitos Funcionais (RF) . . . 42
3.2.2 Requisitos Não Funcionais (RNF) . . . 43
3.3 Decisões de design . . . 44
3.3.1 Modelo de consenso: *Proof of Stake* frente ao *Proof of Authority* . . . 44
3.3.2 Linguagem de contratos escolhida . . . 44
3.3.3 Testes locais e estratégia de ensaio . . . 45
3.3.4 Autorização: análise comparativa entre credencial e token . . . 45
3.3.5 Observabilidade e simplicidade operacional . . . 45
3.4 Arquitetura de Solução . . . 45
3.4.1 Visão em Camadas e Diagrama Geral . . . 45
3.4.2 Componentes e Responsabilidades . . . 47
3.4.3 Fronteiras e Interfaces . . . 47
3.4.4 Ambiente de Execução e Consenso . . . 47
3.4.5 Implementação do *commit-reveal* . . . 48
3.4.5.1 Fluxo de geração, *commit* e *reveal* . . . 49
3.5 LLM e Gemma . . . 50
3.6 Contratos e Interface . . . 50
3.7 Segurança aplicada . . . 51
3.7.1 Assinaturas digitais (ECDSA) . . . 52
3.7.2 Sigilo do voto e aleatoriedade . . . 52
3.7.3 Elegibilidade via token não fungível . . . 52
3.7.4 Revogação e eventos de auditoria . . . 52
3.8 Implementação com Hardhat e Testnet . . . 53
3.8.1 Hardhat Network: rede local simulada . . . 53
3.8.2 Testnet pública . . . 53
3.9 Transparência e auditoria . . . 55
3.9.1 Mecanismos on-chain, reconstituição e auditoria . . . 55
3.10 Testes e Validação . . . 55
3.10.1 Casos de Teste . . . 55
3.10.2 Escopo e organização dos testes . . . 56
3.10.2.1 Testes de unidade (on-chain) . . . 56
3.10.2.2 Testes de integração com NFT . . . 56
3.10.2.3 Propriedades globais . . . 57

4 RESULTADOS . . . 58
4.1 Cobertura funcional dos cenários . . . 58
4.2 Conformidade do fluxo *commit–reveal* e regras de negócio . . . 58
4.3 Transparência e trilha de auditoria . . . 59
4.3.1 Visualização de dados . . . 59
4.4 Métricas empíricas e custo operacional . . . 63
4.5 Interpretação frente aos objetivos do projeto . . . 66

4.5.1 Evidências, artefatos e reprodutibilidade 66

5 CONCLUSÃO 68
5.1 Trabalhos Futuros 69

REFERÊNCIAS 70

APÊNDICES 77

APÊNDICE A – REPOSITÓRIO PRINCIPAL 78

APÊNDICE B – LINK DO REPOSITÓRIO COM O CÓDIGO PARA CÁLCULO DE MÉTRICAS 79

APÊNDICE C – PROMPT PARA GERAÇÃO DOS CENÁRIOS EM JSON 80

1 Introdução

No geral, os ambientes de votação representam um espaço onde decisões financeiras, de manutenção e normas são tomadas. Problemas habituais desses processos afetam a visão de legitimidade e a efetividade dessas reuniões, como os baixos índices de participação, a falta de registros auditáveis e questionamentos sobre fraudes ou manipulações.

Com a pandemia de COVID-19, esses problemas se agravaram, o que gerou uma necessidade urgente de digitalização dos processos (NITAHARA, 2021), resultando em uma migração para plataformas genéricas, sem garantia de autenticidade ou sigilo da votação em pauta. A falta de ferramentas digitais confiáveis para substituir a impossibilidade de assembleias presenciais fez com que ficasse evidente a necessidade de uma solução segura e acessível para votação remota (NETO; VASCONCELOS, 2025).

Nos últimos anos, a necessidade de modernização tem transformado grande parte dos processos tradicionais da administração de maneira a acelerá-los, tanto no contexto da eficiência quanto na redução da burocracia. A modernização abrange processos como a coleta de votos, prestação de contas, a emissão de documentos e tramitação de decisões administrativas. Essa transformação é evidenciada por iniciativas de inovação na administração, que buscam promover uma gestão mais eficaz e transparente (SOUSA, 2024; ALMEIDA, 2024).

Os meios de organização social, sua estrutura e os métodos de tomada de decisão coletiva trazem consigo uma preocupação ainda maior com a segurança e a transparência. Conforme destaca Zou (2022, p. 68) em sua análise sobre o município de Xangai, "[...] a implementação do e-governo proporcionou mais transparência na condução dos processos.", demonstrando o poder da tecnologia para fortalecer a confiança das operações.

A evolução na gestão, com foco na digitalização e na simplificação de processos, evidenciou a necessidade de soluções que não se limitem à automação, mas que garantam níveis superiores de segurança, integridade e auditabilidade dos dados (MOURA; BRAUNER; JANISSEK-MUNIZ, 2020). Nesse contexto, as propriedades dos sistemas de registro distribuído, como descentralização, criptografia e imutabilidade dos registros, se encaixam aos objetivos de redução de burocracia e aumento da transparência, especialmente em processos sensíveis (PEREIRA; ANGONESE, 2023).

Quanto as exigências de integridade, auditabilidade e governança apontadas na digitalização de processos sensíveis, este projeto adota uma rede blockchain privada e avalia o uso de Proof of Stake (PoS) como mecanismo de consenso em ambiente privado, posicionando-o frente a alternativas típicas de redes privadas, quanto a reduzir da burocracia e aumento da transparência em assembleias (MOURA; BRAUNER; JANISSEK-MUNIZ, 2020; PEREIRA; ANGONESE, 2023).

A compatibilidade entre verificabilidade e privacidade do voto é tratada pela separação *on-chain/off-chain* com provas e metadados imutáveis na cadeia, conteúdos sensíveis e identificadores diretos fora dela, por perfis de leitura controlados e por mecanismos criptográficos adequados ao domínio, preservando o sigilo sem abrir mão da auditabilidade (PAAR; PELZL, 2009).

Neste trabalho, é adotada a votação condominial como prova de conceito, reconhe-

cendo que o escopo está limitado à avaliação de mecanismos de segurança, transparência e auditabilidade em um ambiente controlado, sem abordar aspectos como experiência do usuário em larga escala, integração com sistemas legados ou análise de performance em redes de alta concorrência. Adicionalmente, uma abordagem tecnológica pode prover segurança no armazenamento e gerenciamento de dados, o que permite fortalecer a justificativa da sua adoção (MOURA; BRAUNER; JANISSEK-MUNIZ, 2020).

Segundo o Sumário Executivo Levantamento da Tecnologia Blockchain do União (2021b), a blockchain se enquadra como uma tecnologia de propósito geral, capaz de impactar drasticamente as relações econômicas e sociais e prover melhorias significativas em diversos setores governamentais.

Nesse contexto, a tecnologia de razão distribuída (DLT — *Distributed Ledger Technology*) presente na blockchain se destaca, pois tem a integridade e auditabilidade como seu traço fundamental, trazendo com ela aspectos ainda mais relevantes, como a descentralização e a resistência à violação.

Além disso, a base criptográfica da blockchain proporciona a segurança, através dos algoritmos de hash e dos algoritmos assimétricos, que dão a garantia de integridade e confidencialidade (PAAR; PELZL, 2009). Tais mecanismos são essenciais para assegurar que apenas participantes com autorização prévia possam participar e consultar informações internas do sistema.

A rastreabilidade das informações e a garantia de sequencialidade são possíveis por conta de que, em sua parte estrutural, a blockchain funciona como uma lista ligada de blocos, que se encadeiam através de um identificador, o *hash*, que cada bloco possui para o anterior, se assemelhando a uma lista ligada (NAKAMOTO, 2008, p. 2).

Frente a essas circunstâncias, a blockchain tem sido utilizada em vários setores da sociedade e administrativos, no âmbito condominial, tendo usos em registros imutáveis de atas de assembleias, automação da cobrança de taxas condominiais e controle de acesso a áreas comuns, entre outras aplicações. Neste trabalho, a votação condominial foi adotada como prova de conceito, mas o mesmo modelo pode ser estendido a outras modalidades de votação, assim como assembleias de associações de moradores, conselhos escolares, plebiscitos municipais, entre outros, demonstrando a versatilidade da blockchain como base. Reuniões de condomínio, que eram tradicionalmente realizadas de forma presencial, agora estão se deparando com outros tipos de desafios que exigem a procura de soluções mais inovadoras (PEREIRA; ANGONESE, 2023).

Nesse cenário, a adoção de tecnologias como a blockchain tem sido estudada como alternativa viável para enfrentar esses desafios, oferecendo segurança, transparência e ampliação da participação ao possibilitar votações eletrônicas auditáveis e acessíveis remotamente (PEREIRA; ANGONESE, 2023).

1.1 Justificativa

Os rumos sociais e tecnológicos evidenciam a necessidade de soluções eficientes a fim de amenizar a baixa participação e a contestação de resultados e suprir a ausência de mecanismos auditáveis. A implementação de um sistema baseado em blockchain se mostra como uma solução viável para garantir transparência, legitimidade e segurança em processos de decisão coletivos (MOURA; BRAUNER; JANISSEK-MUNIZ, 2020;

PEREIRA; ANGONESE, 2023).

A opção por uma rede privada alinha governança, controle de participantes validadores e isolamento operacional com exigências de auditoria e conformidade do domínio condominial, preservando a rastreabilidade e a verificabilidade dos registros sem exposição desnecessária do ambiente (MOURA; BRAUNER; JANISSEK-MUNIZ, 2020).

A implementação deste sistema justifica-se pela transparência inerente à tecnologia blockchain, que elimina dúvidas sobre a legitimidade dos resultados, reduzindo conflitos e contestações (CIPA, 2023). Além disso, os registros imutáveis garantem uma auditabilidade completa de todo o processo, assegurando que qualquer condômino possa verificar a integridade das votações realizadas.

A adoção da blockchain enfrenta diretamente os problemas identificados em assembleias condominiais, como a falta de registros verificáveis, baixa adesão e questionamentos quanto à legitimidade dos resultados. Por meio da imutabilidade e da descentralização dos registros, a tecnologia contribui para elevar o nível de confiança em processos desse tipo, mesmo em ambientes virtuais (PEREIRA; ANGONESE, 2023).

Conforme apontado pelo União (2021a), a blockchain proporciona uma nova camada de segurança, permitindo o acompanhamento descentralizado seguro do processo de cada um dos envolvidos, sem a necessidade de um intermediário. Tal sistema estaria alinhado com a incorporação de soluções digitais e estratégias de modernização especialmente relevantes no contexto pós-pandemia, onde soluções digitais tornaram-se mais aceitas e necessárias nas rotinas de administração de condomínios, como integrações automatizadas e plataformas de gestão digital (MOURA; BRAUNER; JANISSEK-MUNIZ, 2020).

Para avaliar a solução sob condições realistas e diversas, serão integrados modelos de linguagem (LLMs - *Large Language Model*) para gerar cenários de simulação reprodutíveis, cobrindo variações de quórum, padrões de participação, comportamentos adversariais e falhas de rede com base em configurações iniciais determinadas, formatos padronizados de solicitações ao modelo e condições que limitam o conteúdo a cenários plausíveis, sem violar a privacidade (BOMMASANI et al., 2021; FIGUEIRA; VAZ, 2022).

Esses cenários sintetizados, que complementam casos do domínio anonimizados, permitem avaliar métricas coerentes com os objetivos por exemplo, cobertura de caminhos, taxa de inconsistências e latência, conectando diretamente a geração de dados de teste às perguntas de pesquisa e às exigências de auditoria (FIGUEIRA; VAZ, 2022).

1.2 Objetivos

1.2.1 Objetivo Principal

Desenvolver um sistema de votação baseado em tecnologia blockchain privada, utilizando o contexto de assembleias condominiais como prova de conceito, com o propósito de demonstrar a viabilidade de mitigar a concentração de controle, promover a transparência dos registros, garantir a rastreabilidade dos processos de tomada de decisão coletiva e preservar a privacidade do voto, com validação por meio de simulações de cenários em assembleias. (UNIãO, 2021b; MOURA; BRAUNER; JANISSEK-MUNIZ, 2020)

1.2.2 Objetivos Específicos

1. Especificar e documentar a arquitetura do sistema, gerando um diagrama de componentes e um conjunto de especificações técnicas que atendam aos requisitos de auditabilidade, autenticação e integridade;

2. Implementar uma prova de conceito em blockchain privada, com interface web funcional e backend descentralizado, capaz de registrar e validar votos de forma segura;

3. Elaborar um relatório de cenários de uso em assembleias condominiais, apresentando casos de teste e fluxos de votação simulados que demonstrem a aplicabilidade da solução em contextos reais;

4. Conduzir experimentos de votação na prova de conceito, coletando e analisando métricas de desempenho, segurança e usabilidade, com o objetivo de avaliar a adequação dos processos de votação em ambiente digital controlado.

2 Fundamentação Teórica

2.1 Sistemas de Votação

Os sistemas de votação têm como objetivo a organização e a conversão de votos em resultados para uma tomada de decisão coletiva, visando um sistema eficiente, seguro e imparcial que tende, democraticamente, à vontade popular (TSE, 2021).

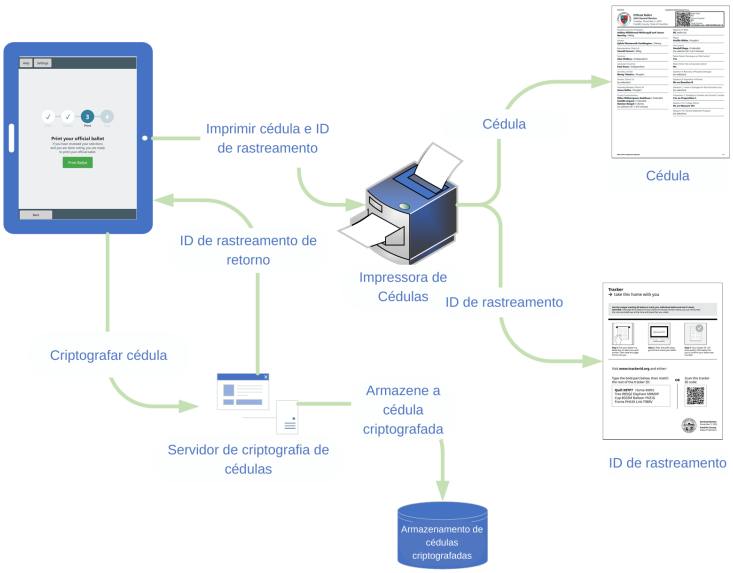
A votação é o momento em que ocorre a tomada de decisão, resultando em uma representação do desejo da maioria. Existem dois tipos distintos, com base no sigilo adotado, sendo eles o secreto e o aberto (ostensivo).

A votação secreta se caracteriza por ser realizada de forma anônima, de uma maneira em que não se é possível identificar qual foi o voto de cada votante. Já a votação aberta é feita de forma ostensiva, ou seja, é realizada de forma aberta em que se é possível identificar qual foi o voto de cada um (BRASIL, s.d.).

2.1.1 Meios de Votação

Assim como há diferenças entre tipos de votação, também existem diferenças entre as formas de praticar o voto, dividindo-se entre modelos que utilizam ou não a tecnologia como base. O modelo que prioriza a abordagem tecnológica utiliza alguma inovação, no formato de sistema, que possibilite algum tipo de automação no processo de votação, como, por exemplo, o uso de *Ballot Marking Devices* (BMDs) como exemplificado na figura 1, utilizados nos Estados Unidos da América, e a Urna Eletrônica, utilizada no Brasil (Capitol Technology University, 2023).

Figura 1 – Diagrama de Ballot Marking Devices



Fonte: ElectionGuard (s.d.)

O modelo tradicional é uma forma que se utiliza de processos manuais, em sua maioria as urnas, que recebem papéis assinalados com a escolha do votantes, que são contados manualmente por um grupo de pessoas responsáveis pela apuração (Capitol Technology University, 2023).

A tecnologia aplicada a votações evoluiu significativamente ao longo dos anos, sendo incorporada em diferentes contextos com o objetivo de aprimorar tanto a segurança quanto a eficiência dos processos. Com a informatização, alcançamos maior precisão na contagem e agregação de votos, minimizando falhas humanas que costumavam comprometer a confiabilidade dos resultados (Capitol Technology University, 2023).

No aspecto da segurança de resultados, a tecnologia representa um avanço crucial ao combater fraudes comuns em sistemas baseados em cédulas de papel, que são suscetíveis à adulteração ou destruição durante uma triagem completamente manual. A digitalização dos votos contribui para garantir registros mais consistentes e auditáveis, com menor exposição à interferência humana. Contudo, especialmente em contextos remotos e online, a presença da tecnologia por si só não é suficiente para eliminar todos os riscos, sendo necessário um desenho cuidadoso dos sistemas para mitigar novas ameaças cibernéticas (Capitol Technology University, 2023).

Segundo a pesquisa de Mugica (2015), o tipo de votação mais vulnerável é aquele que não adota nenhuma forma de tecnologia em nenhuma etapa do processo. Sistemas eletrônicos bem projetados e dedicados à finalidade de votação são capazes de reduzir drasticamente a probabilidade de manipulação de resultados, com ganhos de segurança que variam de 10 a 1000 vezes, dependendo do grau de automação.

Além da segurança, a adoção de sistemas eletrônicos também proporciona agilidade, pois votos geridos digitalmente permitem a apuração dos resultados em menor tempo após o fim da votação, diferentemente dos processos não tecnológicos e manuais, que tendem a ser mais lentos (Capitol Technology University, 2023).

Por fim, um dos principais propósitos da tecnologia em votações é reforçar a privacidade do votante. Nesse contexto, a tecnologia blockchain se destaca como base promissora para sistemas de votação modernos, ao oferecer garantias de integridade, segurança, transparência e auditabilidade em todas as etapas do processo.

2.1.1.1 Votação Digital

Conforme explicado no tópico 2.1.1, o uso da tecnologia proporciona inúmeras vantagens em diferentes pontos do processo da votação, sendo uma das aplicações exploradas o voto digital, em que votantes realizam seu voto por meio da internet. Diante desta nova forma de votação, surge a avaliação do novo modelo quanto às suas vantagens e desvantagens quando comparado aos demais. O meio digital permite a votação a distância, com uma rápida auditabilidade e contabilização. Por outro lado, pesquisas sugerem que o voto digital não traz um aumento significativo na participação da população e ainda carece de sistemas eficientes de garantia da segurança e de políticas anti-fraude, pois, uma vez conectado à internet, fica suscetível a ataques cibernéticos, tornando a votação menos credibilizada (GOODMAN, 2020).

Mesmo assim, países como Canadá, Estônia e Austrália já implementaram e seguem utilizando modelos de votação pela internet, com a proposta da redução de custos atrelados e facilidade de acesso da população. Entretanto, mesmo após quinze anos do início dos

testes, ainda não há evidências robustas de que a adoção do voto online tenha promovido um aumento significativo na participação de votantes (GOODMAN, 2020).

2.1.1.2 Segurança em Votações Online

Embora a digitalização traga avanços importantes quanto à agilidade e redução de erros humanos, a segurança nas votações online continua sendo um dos maiores desafios da modernização deste tipo de processo. Isso se deve ao fato de que, ao contrário de sistemas eletrônicos presenciais (como a urna eletrônica brasileira), os sistemas remotos que utilizam a internet enfrentam riscos devido à sua exposição (AAAS, 2020).

Até o presente momento, não há tecnologia amplamente aceita que garanta alguns dos pilares para votações remotas seguras, sendo eles: privacidade, verificabilidade e integridade. Segundo especialistas, ameaças como *malwares* nos dispositivos, ataques maliciosos como negação de serviço (DDoS e DoS) e falhas nos sistemas de autenticação dos usuários comprometem a confiabilidade de todo o sistema. Além disso, em 2015, a *US-Vote-Foundation* alertou que nenhum sistema de votação online de larga escala é seguro, destacando que a falta de garantias de privacidade e auditabilidade torna a modalidade de voto remoto vulnerável a fraudes e manipulações sistêmicas (AAAS, 2020).

Por conta disso, a construção deste tipo de sistema deve ser muito bem elaborada e pensada, e é justamente nesse contexto que a tecnologia blockchain se destaca como uma possível solução para trazer segurança e confiabilidade aos votantes.

2.1.2 Votação Condominial

A votação condominial, usualmente realizada por assembleias, constitui um dos principais instrumentos de deliberação coletiva em ambientes residenciais. Trata-se de um mecanismo democrático que permite e assegura aos condôminos o exercício de seus direitos e deveres, inclusive o direito ao voto nas decisões que impactam o coletivo (BRASIL, 1964). Previsto na Lei n° 4.591/1964 (Lei do Condomínio) e no Código Civil Brasileiro, nos artigos 1.334 a 1.355 (BRASIL, 2002).

As assembleias são instâncias formais nas quais os moradores se reúnem para discutir, deliberar e votar sobre temas de interesse comum. Elas são de extrema importância, pois é neste espaço em que o síndico presta contas sobre sua gestão, são aprovadas reformas e manutenções, onde se fixam tarifas e taxas e também se definem alterações no regimento interno do condomínio, além de demais decisões relevantes que impactam a coletividade dos demais moradores (Cysne Administradora, 2023).

Existem dois tipos principais de assembleias condominiais: ordinárias e extraordinárias. As assembleias ordinárias são realizadas, em regra, de forma anual e têm como finalidade deliberar sobre matérias recorrentes da administração do condomínio, como a prestação de contas, a aprovação do orçamento, a definição de despesas e a eleição do síndico, conforme previsto no art. 1.350 do Código Civil. Já as assembleias extraordinárias podem ser convocadas a qualquer momento, pelo síndico ou por uma fração dos condôminos, destinando-se à deliberação de assuntos específicos ou urgentes que não se enquadram na pauta regular das ordinárias, como obras emergenciais, contratações pontuais ou alterações no regimento interno (uCondo, 2023).

2.2 Dados sintéticos e LLMs

A geração de dados sintéticos tem como objetivo construir conjuntos artificiais de observações que permitam exercitar um sistema em regimes de operação relevantes, sem expor indivíduos reais nem depender de coletas demoradas ou juridicamente sensíveis. Em ambientes de registro distribuído privados, como o contexto desta monografia, isso viabiliza a avaliação de propriedades funcionais (correção de regras, integridade de estados) e não funcionais (desempenho, resiliência) sob cenários controláveis e reprodutíveis, preservando a privacidade dos participantes reais (FIGUEIRA; VAZ, 2022).

Do ponto de vista conceitual, entende-se por conjunto sintético $\tilde{D}$ um conjunto amostrado de uma distribuição indutora P_{sint} que aproxima, com erro controlado, aspectos da distribuição real P_{real} julgados relevantes para o estudo. A questão central passa a ser se eles são adequados para as tarefas para as quais foram construídos, sem reproduzir registros observados no mundo físico (FIGUEIRA; VAZ, 2022; STADLER; OPRISANU; TRONCOSO, 2022).

2.2.1 Eixos de qualidade para dados sintéticos

A avaliação de dados sintéticos se dá em três eixos principais: utilidade, fidelidade e privacidade. A utilidade diz respeito à capacidade do conjunto sintético de sustentar as tarefas para as quais foi criado. Em termos práticos, isso significa disponibilizar os atributos necessários, com tipos e granularidade adequados, assegurando as condições de integridade do domínio, coerência temporal e integridade básica entre entidades. Um conjunto é considerado útil quando, ao substituir dados reais por sintéticos em testes de carga, experimentos funcionais ou análises de métricas, o sistema reage de forma consistente com as expectativas de engenharia traçadas para o ambiente (FIGUEIRA; VAZ, 2022).

A fidelidade mede o quanto os dados sintéticos preservam estruturas e regularidades relevantes da distribuição de referência, sem reproduzir registros reais. Inclui tanto a coerência estrutural (esquema, chaves, integridade referencial) quanto a proximidade estatística de distribuições e dependências entre variáveis. (FIGUEIRA; VAZ, 2022; STADLER; OPRISANU; TRONCOSO, 2022).

Por fim, a privacidade avalia o risco de que dados sintéticos revelem informações sobre indivíduos reais, mesmo quando a intenção é apenas uso em testes. Os riscos típicos incluem memorização de exemplos, possibilidade de inferir pertencimento de pessoas a bases de referência ou de reidentificar indivíduos por meio de combinações raras de atributos (STADLER; OPRISANU; TRONCOSO, 2022). Em ambientes de teste, a proteção é tratada principalmente na origem, onde se tenta evitar o uso direto de registros identificáveis, reduzindo instruções excessivamente específicas e introduz-se variabilidade suficiente para afastar coincidências diretas com dados reais, mantendo o foco em padrões agregados.

2.2.2 Métodos de geração e papel dos LLMs

Os métodos de geração de dados sintéticos podem ser organizados, de forma resumida, em quatro grupos que se complementam. O primeiro grupo é o de regras e simulação, em que os dados são gerados a partir de especificações programáticas, catálogos de valores e restrições de integridade, frequentemente com simuladores de eventos ou agentes que reproduzem fluxos de negócio. O segundo abrange técnicas de perturbação (*augmentation*),

nas quais se parte de instâncias reais e se aplicam transformações controladas (substituições, inserções, remoções) para ampliar um conjunto de exemplos, preservando rótulos e estrutura essencial. O terceiro grupo corresponde aos modelos generativos treináveis (como GAN, VAE e modelos de difusão), que aprendem uma distribuição a partir de dados e geram novas amostras, em geral com foco em alta fidelidade estatística. Por fim, o quarto grupo compreende os modelos de linguagem de grande porte (LLMs) como geradores condicionais, capazes de produzir textos e dados semiestruturados a partir de instruções e exemplos em linguagem natural, sem a necessidade de treinar um modelo específico por tarefa (BROWN et al., 2020).

Há uma relação boa entre modelos de simulação e LLMs, as regras e estruturas de domínio, por exemplo são usadas como "esqueleto" dos cenários. Sobre esse esqueleto, um modelo de linguagem é empregado como gerador condicional para construir descrições e parâmetros de simulação que respeitam as restrições definidas, mas introduzem diversidade suficiente para explorar diferentes situações de uso do sistema (BROWN et al., 2020).

A utilização de LLMs nesse papel permite especificar em linguagem natural as variações desejadas, delegando ao modelo a tarefa de preencher detalhes de forma consistente com o contexto (BROWN et al., 2020). A geração automática é seguida por etapas de validação estrutural e numérica, garantindo que os cenários resultantes sejam compatíveis com o modelo de dados e com as regras do protocolo de votação descritos nos capítulos seguintes.

2.2.3 Modelos de linguagem de grande porte (LLMs)

Modelos de linguagem de grande porte são redes neurais treinadas para processar e gerar texto, baseadas, em sua maioria, na arquitetura Transformer, introduzida em 2017 (VASWANI et al., 2017). Essa arquitetura se apoia em mecanismos de atenção que permitem ao modelo considerar, em paralelo, diferentes partes de uma sequência ao estimar a próxima palavra, o que viabiliza o treinamento em grandes corpora de texto e o manejo de dependências de longo alcance.

O desenvolvimento contemporâneo de LLMs segue, em geral, um processo de duas etapas: um pré-treinamento em grandes volumes de dados textuais, no qual o modelo aprende padrões gerais de linguagem e conhecimento de mundo, seguido de um refinamento para seguir instruções e produzir respostas mais alinhadas a expectativas humanas (BROWN et al., 2020). Nessa segunda etapa, exemplos curados e *feedback* humano são usados para ajustar o comportamento do modelo em tarefas como resposta a perguntas, síntese de texto e execução de instruções complexas.

2.2.3.1 *Gemma*: características e aplicabilidade

A família de modelos *Gemma*, desenvolvida pelo *Google* como versão aberta baseada na tecnologia *Gemini*, oferece uma solução prática para aplicações que requerem capacidades avançadas de linguagem sem os custos e complexidades de modelos proprietários (Gemma Team, 2024). Disponível em diferentes tamanhos (2, 7, 9 e 27 bilhões de parâmetros), cada variante é oferecida em duas versões: modelos base que aprenderam padrões gerais de linguagem e modelos instrucionais refinados para seguir comandos específicos com precisão e alinhamento às expectativas humanas (Gemma Team, 2024). Essas características técnicas incluem capacidade de processar textos longos (até 8.000

palavras simultaneamente) e otimizações que permitem performance competitiva com modelos proprietários maiores, mantendo requisitos computacionais acessíveis para pesquisa acadêmica e aplicações empresariais com orçamento limitado (Gemma Team, 2024).

Os modelos *Gemma* apresentam vantagens significativas em termos de flexibilidade de implantação, funcionando eficientemente desde computadores pessoais até infraestruturas empresariais, além de licenciamento aberto que permite modificações e redistribuição para fins comerciais e acadêmicos (Gemma Team, 2024). As versões instrucionais demonstram capacidade competitiva em seguimento de instruções detalhadas, adaptação a diferentes estilos de escrita e geração de conteúdo controlado, características importantes para aplicações como geração de dados sintéticos e avaliação automática (Gemma Team, 2024). No entanto, como modelos de menor escala comparados a alternativas proprietárias, podem apresentar limitações em tarefas de raciocínio muito complexas ou conhecimento especializado altamente específico, além de requererem ajustes cuidadosos de *prompts* para otimização.

2.3 Criptografia

A criptografia, segundo Paar e Pelzl (2009, p. 3), se baseia na ciência envolvida em ocultar informações por meio da escrita. Desde os egípcios, a criptografia é utilizada de uma ou mais formas em muitas das culturas que desenvolveram uma linguagem escrita.

Após sua ampliação, não foi surpresa que versões mais simples de criptografia foram sendo descobertas e, com isso, novas formas mais sofisticadas foram surgindo com o tempo, tendo sua aplicabilidade saindo de cartas oficiais entre governos até planos de guerra. Sua aplicabilidade pôde ser bem aprofundada no desenvolvimento de comunicações de computadores, especificamente na parte de dados e telecomunicações, onde a criptografia é necessária quando ao comunicar dentro de um meio sem total confiança, o que inclui a maioria dos tipos de rede, particularmente na internet (KESSLER, 2025).

A criptografia se separa em dois ramos principais, a primeira é a estrutura de cifra mais antiga, criada junto à linguagem escrita, é a qual se utiliza em algoritmos simétricos, em que duas partes têm um método de encriptação e descriptografia pelo qual elas compartilham uma mesma chave, usada especialmente em encriptação de dados e checagem de integridade (PAAR; PELZL, 2009).

Em 1976, um tipo diferente de encriptação foi criado por Whitfield Diffie e Martin Hellman. No algoritmo assimétrico, um usuário possui uma chave privada, assim como na criptografia simétrica, mas também uma chave pública. A encriptação utilizando-se da chave pública só poderá ser descriptografada pela chave privada, a operação inversa também é possível, mantendo a segurança (PAAR; PELZL, 2009, p. 3). Assim, com a segurança de que a chave privada esteja em posse unicamente de quem a criou, podemos ter certeza sobre quem é o remetente da mensagem ou garantir que apenas o destinatário descubra o conteúdo da mensagem (KESSLER, 2025).

Por fim, a aplicação desses dois é o ramo de protocolos de criptografia. Nele, algoritmos simétricos e assimétricos são vistos como bases para que aplicações de comunicação segura na Internet possam ser realizadas (PAAR; PELZL, 2009, p. 3). Um exemplo de uso é a segurança da camada de transporte (TLS - *Transport Layer Security*) que usa um protocolo de segurança que criptografa os dados enviados por uma rede como a Internet, geralmente entre um dispositivo cliente e um servidor (KESSLER, 2025).

2.3.1 Algoritmos Assimétricos

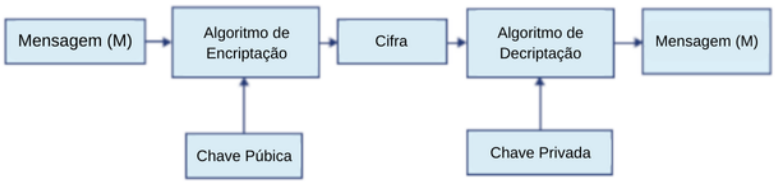
A criptografia assimétrica, também conhecida como criptografia de chave pública, representa um dos desenvolvimentos mais significativos na história moderna em criptografia. Em seu artigo, Whitfield Diffie e Martin Hellman, descreveram um sistema criptográfico de duas chaves em que se torna possível estabelecer uma comunicação segura mesmo sobre um canal inseguro, sem compartilhar a mesma chave secreta (DIFFIE; HELLMAN, 1976).

Entre os algoritmos mais utilizados atualmente estão o RSA (Rivest-Shamir-Adleman) sendo o mais antigo e amplamente utilizado, principalmente por sua robustez; o DSA (*Digital Signature Algorithm*) que tem como característica a eficiência para processos de assinatura e verificação; e o ECC (*Elliptic Curve Cryptography*) com uma maior força criptográfica e chaves mais curtas. Cada um deles tem um foco particular como desempenho computacional, segurança e tamanho da chave, tendo aplicações diferentes dependendo do contexto (HANKERSON; MENEZES; VANSTONE, 2011).

Esse tipo de criptografia se baseia em um par de chaves que se complementam matematicamente. Seu funcionamento depende da existência de funções chamadas de "funções de mão única", nas quais o processamento é simples, porém revertê-lo é inviável computacionalmente sem o conhecimento da chave específica. A Figura 2 exemplifica esse processo, mostrando a relação entre a chave pública usada na encriptação e a chave privada utilizada na decriptação. Desse modo, uma mensagem criptografada com uma chave pública só pode ser decifrada com sua chave privada correspondente, e vice-versa (KESSLER, 2025).

A segurança destes algoritmos se dá sobre a confiança em problemas da teoria dos números, como a fatoração de inteiros e o cálculo de logaritmos discretos em corpos finitos ou em grupos de pontos sobre curvas elípticas. Porém, as vulnerabilidades não se conectam diretamente com a fraqueza sobre esses problemas, mas, na verdade, sobre os problemas em suas implementações, como ataques de canal lateral e problemas de implementação (BOS et al., 2013).

Figura 2 – Diagrama de uma criptografia assimétrica



Fonte: Hasan et al. (2021, p .7)

Em um cenário prático, um remetente pode encriptar uma informação usando a chave pública do destinatário, sendo assim, apenas ele poderá descriptografar o conteúdo encriptado utilizando-se de sua chave privada. Esse método também pode ser utilizado para provar quem é o remetente da mensagem, uma vez que ele pode criptografar com sua chave privada para que o receptor decodifique com a chave pública, inibindo que a mensagem seja alterada no processo (KESSLER, 2025).

2.3.1.1 Criptografia de Curva Elíptica (ECC)

A criptografia de curva elíptica (ECC — *Elliptic Curve Cryptography*) é uma aplicação de algoritmo assimétrico baseada nas propriedades relacionadas a curvas elípticas da matemática. A segurança dada por esses processos é encontrada através da força computacional necessária para o problema do logaritmo discreto em curvas elípticas (HANKERSON; MENEZES; VANSTONE, 2011).

Estas curvas são descritas, no caso mais comum, pela forma simplificada de Weierstrass:

$$E : y^2 = x^3 + ax + b \tag{2.1}$$

Mais do que detalhar a álgebra envolvida, o ponto central aqui é o propósito dessa modelagem: ao escolher parâmetros a e b sobre um corpo finito primo F_p, obtém-se um conjunto finito de pontos com uma aritmética interna estável. Isso permite definir operações eficientes de "soma de pontos" e sua repetição, a multiplicação escalar usada para gerar chaves, e garantir que o caminho inverso permaneça computacionalmente difícil (o ECDLP), sustentando a segurança do esquema. Em síntese, a forma de Weierstrass e o uso de F_p são adotados porque tornam as operações previsíveis e rápidas, ao mesmo tempo em que preservam a dificuldade do problema subjacente, justificando a aplicação criptográfica de curvas elípticas (HANKERSON; MENEZES; VANSTONE, 2011).

A partir dessa estrutura, define-se em $E(F_p)$ uma operação interna de adição de pontos que permite "percorrer" a curva de forma eficiente e previsível. Na prática, escolhe-se um ponto base G (padronizado) com um grande ciclo de repetições, e usa-se essa operação para construir as chaves: a chave privada é um inteiro aleatório d e a chave pública é o ponto obtido ao aplicar essa repetição d vezes sobre G:

$$Q = d \cdot G \tag{2.2}$$

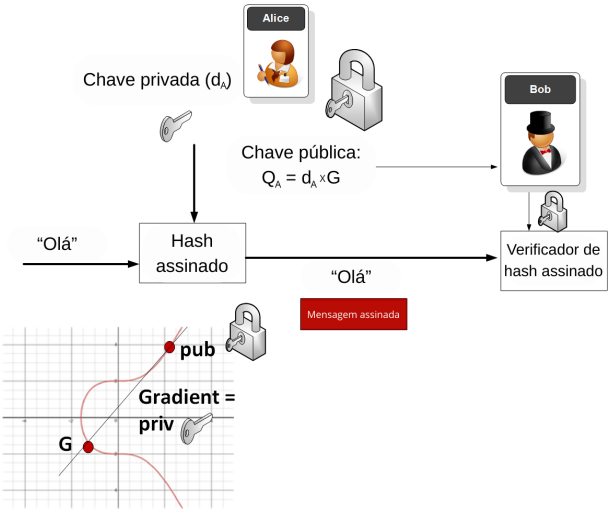
O objetivo dessa construção é explorar uma assimetria computacional: calcular Q a partir de d e G é rápido e fácil de verificar, mas recuperar d apenas de G e Q é computacionalmente inviável. Essa "mão única" viabiliza, de forma prática, protocolos como acordo de chaves e assinaturas digitais com chaves curtas e alto desempenho, mantendo a segurança mesmo em ambientes com recursos limitados. O funcionamento desse processo pode ser visualizado na Figura 3.

A segurança do esquema baseia-se no fato de que, dados apenas Q e G, é intratável recuperar d, o problema do logaritmo discreto em curvas elípticas (ECDLP - *Elliptic Curve Discrete Logarithm Problem*). Em termos conceituais, curvas com grande ordem n e escolhidas sobre corpos F_p com p suficientemente grandes ampliam o espaço de busca e desestimulam ataques práticos. Isso permite alcançar níveis de segurança equivalentes aos de esquemas como RSA ou DSA com tamanhos de chave muito menores, o que torna a ECC especialmente atraente para ambientes com recursos restritos (PAAR; PELZL, 2009).

2.3.1.2 Algoritmo de Assinatura Digital de Curva Elíptica (ECDSA)

Assim como foi discutido através do tópico 2.3.1.1, curvas elípticas apresentam diversas vantagens sobre o RSA e DSA, especialmente quanto à eficiência. Por conta de sua saída ser de um tamanho menor que os outros e ter um tempo de processamento

Figura 3 – Funcionamento do ECC em prática



Fonte: Buchanan (2018)

menor, o algoritmo de assinatura digital de curva elíptica (ECDSA) foi padronizado pelo *American National Standards Institute* (ANSI) em 1998, consolidando-se até os dias atuais como um padrão seguro e eficiente quanto as assinaturas digitais (PAAR; PELZL, 2009).

Os passos para a geração do ECDSA são conceitualmente próximos ao DSA, porém o problema do logaritmo discreto é construído sobre uma curva elíptica, fazendo com que a aritmética ao computar um ECDSA seja completamente diferente do DSA. Dessa forma, a geração das chaves para o ECDSA acontece nos seguintes passos (PAAR; PELZL, 2009):

1. Define-se uma curva elíptica E com:
   - módulo p;
   - coeficientes a e b;
   - ponto A, que gera um subgrupo cíclico de ordem prima q.
2. Escolhe-se um número inteiro aleatório d, tal que $0 < d < q$.
3. Calcula-se $B = dA$.

As chaves geradas são:

$$k_{\text{pub}} = (p, a, b, q, A, B) \quad \text{e} \quad k_{\text{pr}} = (d) \tag{2.3}$$

Sendo assim, k_{pub} representa a chave pública, enquanto k_{pr} corresponde à chave privada.

Depois de geradas as chaves, se torna possível a realização dos processos de geração e verificação de assinaturas digitais através do ECDSA. Esse processo garante a autenticidade da mensagem e a integridade dos dados assinados, que são primordiais para as aplicações de segurança.

Usando-se da chave privada, a assinatura da mensagem x é computada através dos seguintes passos (PAAR; PELZL, 2009):

1. Escolhe-se um número inteiro aleatório como chave temporária k_T, tal que $0 < k_T < q$.
2. Calcula-se $R = k_T A$.
3. Define-se $r = x_R$, ou seja, a coordenada x do ponto R.
4. Calcula-se $s \equiv (h(x) + d \cdot r) \cdot k_T^{-1} \mod q$.

O valor r é definido como a coordenada x do ponto R, resultado da multiplicação entre a chave temporária k_T e o ponto gerador A da curva elíptica. A variável x_R refere-se à abscissa do ponto $R = k_T A$.

A função $h(x)$ representa o valor obtido após a aplicação de uma função *hash* sobre a mensagem x, onde a saída deve possuir tamanho pelo menos tão grande quanto a ordem q, utilizando d como a chave privada do assinante. E por fim, se obtém s representando junto a r a resposta do assinante à mensagem original. Essa função tem como objetivo condensar a mensagem em uma representação compacta, chamada de impressão digital (*fingerprint*), que é utilizada no cálculo da assinatura (PAAR; PELZL, 2009).

Seguindo a ideia de comprimir a mensagem x e gerando uma impressão digital pelo processo da geração, agora precisa ser possível a verificação da assinatura. E esse processo é feito a partir das etapas descritas (PAAR; PELZL, 2009):

1. Calcula-se o valor auxiliar $w \equiv s^{-1} \mod q$.
2. Calcula-se o valor auxiliar $u_1 \equiv w \cdot h(x) \mod q$.
3. Calcula-se o valor auxiliar $u_2 \equiv w \cdot r \mod q$.
4. Calcula-se o ponto $P = u_1 A + u_2 B$.
5. A verificação da assinatura $ver_{k_{\text{pub}}}(x, (r, s))$ decorre da seguinte regra:

$$x_P \begin{cases} \equiv r \mod q & \Rightarrow \text{assinatura válida} \\ \not\equiv r \mod q & \Rightarrow \text{assinatura inválida} \end{cases} \tag{2.4}$$

O ponto P é obtido a partir de uma combinação linear dos pontos A (ponto gerador da curva) e B (chave pública do signatário), ponderados por u_1 e u_2, respectivamente. O símbolo x_P refere-se à coordenada x do ponto P. A assinatura é considerada válida se essa coordenada, reduzida módulo q, for compatível com o valor r na assinatura recebida. Caso contrário, a assinatura é rejeitada, pois não corresponde ao par de chaves utilizado na geração (PAAR; PELZL, 2009).

2.3.2 Esquema de Compromisso e Revelação

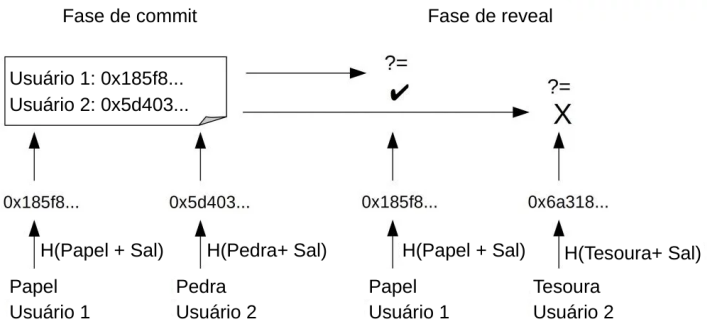
Na dissertação de Santos (2022), os contratos inteligentes são analisados a partir de suas potencialidades e limitações, demonstrando a necessidade de garantir, de forma a conceitos de conexão de blocos, tanto a privacidade de dados sensíveis quanto a ordem correta da execução (SANTOS, 2022).

2.3.2.1 Esquema de Compromisso e Revelação

O esquema de compromisso e revelação é um protocolo que é aplicado em duas fases distintas, usado habitualmente para prevenir fraudes e manipulações em contratos inteligentes. Na primeira fase (compromisso), cada participante gera um valor secreto, criando assim um compromisso e envia apenas este ao contrato inteligente. Esse compromisso não revela nada sobre o valor em si, mas vincula o usuário ao valor, pois qualquer alteração traria um resultado diferente ao final, garantindo que nenhum participante alterou seu valor após conhecer o dos demais (SANTOS, 2022). A Figura 4 exemplifica esse processo de validação, mostrando como os compromissos são enviados e posteriormente revelados para verificação.

Só após todos os participantes tiverem enviado seus compromissos, começa a fase de revelação, onde cada usuário revela o valor original e o aleatório usado na criação do compromisso. Desse modo, o contrato verifica se o valor revelado corresponde ao dado registrado (SANTOS, 2022).

Figura 4 – Validação do esquema de compromisso e revelação



Fonte: Oettler (2021)

Através do estudo de Luu et al. (2016) é possível afirmar que a combinação de compromisso e revelação com limites de tempo e penalidades para quem não revela a entrada diminui o risco de boicote interno. Por meio desse cuidado, o protocolo se mostra robusto o suficiente para a adoção em contratos inteligentes.

2.3.3 Funções de *Hash*

As funções de *hash* são algoritmos matemáticos unidirecionais amplamente utilizados em protocolos de segurança digital, autenticação e integridade. Geram um valor *hash* de tamanho fixo, a partir de uma entrada qualquer como uma mensagem ou arquivo. O valor *hash* é uma representação compacta e única do conteúdo original, normalmente exibida como uma sequência de bits ou dígitos hexadecimais, funcionando como uma "impressão digital" do dado de entrada, passando a ser impossível recuperar o conteúdo do texto original (KESSLER, 2025).

Esse tipo de algoritmo é tipicamente utilizado para fornecer uma assinatura digital de arquivos, assegurando que não tenha sido alterado, seja por um erro, ataques ou até mesmo programas maliciosos. As funções *hash* apresentam três propriedades fundamentais: determinismo, em que o valor *hash* permanece idêntico para uma mesma entrada; o "efeito avalanche", no qual qualquer mudança, por menor que ela seja, produzirá uma mudança

significativa no valor de saída, como demonstrado na figura 5 (KESSLER, 2025; PAAR; PELZL, 2009); e a resistência a colisões, que assegura que duas entradas diferentes não gerem a mesma saída, sendo fundamental para garantir a integridade em sistemas que os exploram, como contratos inteligentes ou blocos de uma blockchain (PAAR; PELZL, 2009).

Figura 5 – Exemplo do "Efeito avalanche" em diferentes algoritmos de *hash*

```
Hash string 1: The quick brown fox jumps over the lazy dog
Hash string 2: The quick brown fox jumps over the lazy dog.

MD5 [hash string 1] = 37c4b87edffc5d198ff5a185cee7ee09
MD5 [hash string 2] = 0d7006cd055e94cf614587e1d2ae0c8e

SHA1 [hash string 1] = be417768b5c3c5c1d9bcb2e7c119196dd76b5570
SHA1 [hash string 2] = 9c04cd6372077e9b11f70ca111c9807dc7137e4b

RIPEMD160 [hash string 1] = ee061f0400729d0095695da9e2c95168326610ff
RIPEMD160 [hash string 2] = 99b90925a0116c302984211dbe25b5343be9059e
```

Fonte: Kessler (2025)

2.3.3.1 *Secure Hash Algorithm* (SHA)

O algoritmo do padrão de *hash* seguro (SHS - *Secure Hash Standard*) foi desenvolvido pelo *National Institute of Standards and Technology* (NIST) como parte de uma família de funções de *hash* voltadas à segurança digital, conhecidas como SHA (*Secure Hash Algorithm*) (NIST, 2015).

O SHA-1 produz um valor *hash* de 160 bits (20 bytes) a partir de uma entrada de tamanho arbitrário, processada em blocos de 512 bits (64 bytes) (KESSLER, 2025). Desde 2011, instituições como o NIST e a IETF (*Internet Engineering Task Force*) passaram a recomendar o desuso desse algoritmo após a divulgação de colisões teóricas por WANG, YIN e YU (2005) que foram posteriormente comprovadas de forma prática por pesquisadores do CWI (*Centrum Wiskunde & Informatica*) e do Google (STEVENS et al., 2017).

Por sua vez, o SHA-2 é constituído por quatro algoritmos dentro do SHS: o SHA-224, SHA-256, SHA-384, e o SHA-512 que produzem valores com suas respectivas quantidades, indicados no nome, como tamanhos de bit de processamento. Mensagens de até 2^{64} bits são recomendados os algoritmos SHA-224 e SHA-256, que implicam em um bloco de 512 bits (64 bytes). Já os algoritmos SHA-384 e SHA-512 são recomendados para mensagens de até 2^{128} bits, que implicam em um bloco de 1024 bits (128 bytes) (NIST, 2015).

A base de construção do SHA-2 é dada pelo algoritmo Merkle-Damgård, no qual divide a mensagem em blocos fixos, então aplica funções de compressão e, por fim, encadeia os resultados, diferenças estruturais, de segurança e de desempenho sintetizadas na Tabela 1. Essa abordagem pode ser vulnerável a ataques como a extensão de comprimento, se não implementada com precauções adicionais. Já o SHA-3 utiliza uma função esponja (*sponge function*) com duas fases: absorção e compressão. Baseado no algoritmo Keccak, esse modelo evita vulnerabilidades da construção Merkle-Damgård, com forte resistência a colisões e pré-imagem (PAAR; PELZL, 2009).

Por fim, o SHA-3 é o atual SHS, apesar de ainda não terem surgido ataques efetivos para o padrão do SHA-2, o NIST decidiu que deveria haver uma alternativa ao SHA-2 com um algoritmo diferente em meio a se antever a uma possível descoberta. Em 2012, após uma competição lançada cinco anos antes para a criação de um novo tipo de algoritmo de

hash, foi anunciada a nova versão baseada em funções esponja, recomendada para novas aplicações que exigem alta resistência a colisões e pré-imagens. Essa nova versão suporta tanto saídas de 256 quanto 512 bits (NIST, 2012).

Tabela 1 – Comparação entre SHA-2 e SHA-3

Aspecto	**SHA-2**	**SHA-3**
Construção	Merkle–Damgård	Função esponja
Núcleo interno	Compressão iterativa	Permutação de estado ("esponja")
Segurança	Vulnerável a ataques de extensão de comprimento e colisões parciais	Resistente a ataques conhecidos ao SHA-2 e a extensão de comprimento
Performance	Mais rápido em hardware tradicional	Mais eficiente em software, mas mais lento em hardware genérico

Fonte: Adaptado de Paar e Pelzl (2009)

A relevância real da família SHA é observada em sistemas distribuídos e aplicações que exigem integridade e autenticidade de dados. Embora o SHA-2 ainda seja amplamente utilizado, o SHA-3 representa uma evolução significativa em termos de arquitetura e segurança, tendo sido projetado para resistir a ataques estruturais que poderiam comprometer construções iterativas anteriores (PAAR; PELZL, 2009).

No contexto de tecnologias descentralizadas e registros imutáveis, o SHA-3 passou a ser adotado em diversos protocolos de blockchain e sistemas de assinatura digital, especialmente aqueles que buscam reduzir a dependência de funções herdadas do SHA-2. Sua estrutura baseada em permutação permite reforçar a confiança criptográfica na verificação de blocos, transações e identidades digitais (BAMBARA; ALLEN, 2018).

Além disso, o uso do SHA-3 em autenticação e geração de identificadores garante unicidade e rastreabilidade sem comprometer a privacidade, característica essencial em aplicações sensíveis (NIST, 2012). Dessa forma, o SHA-3 consolida-se como o novo SHS, combinando robustez criptográfica com flexibilidade, elevando o nível de integridade e confiabilidade.

2.3.4 Keccak-256

Keccak-256 é uma função hash criptográfica pertencente à família Keccak, adotada como base do padrão SHA-3. Trata-se de uma instância parametrizada que produz, para qualquer entrada, um resumo (*digest*) de 256 bits. A saída possui comprimento fixo e independe do tamanho ou da complexidade dos dados de entrada (Cyfrin, 2025).

Do ponto de vista estrutural, o Keccak é construído a partir do paradigma de função esponja (*sponge function*). Nessa construção, mantém-se um estado interno de tamanho fixo, particionado em duas regiões: a parte que interage diretamente com os blocos de mensagem, e a parte que é mantida protegida e está relacionada ao nível de segurança. A mensagem de entrada é primeiro dividida em blocos e, na fase de absorção (*absorbing*), cada bloco é combinado com a região exposta do estado, seguida pela aplicação de uma permutação não linear sobre todo o estado. Ao final da absorção, a fase de extração (*squeezing*) produz

o hash de saída lendo bits da região exposta e, se necessário, aplicando novas permutações até se obter o número desejado de bits de digest. O Keccak-256 corresponde ao conjunto específico de parâmetros dessa esponja que resulta em saídas de 256 bits (Cyfrin, 2025).

As propriedades de segurança esperadas de Keccak-256 incluem resistência a colisões, pré-imagem e segunda pré-imagem, no sentido de que é computacionalmente inviável encontrar duas entradas distintas que produzam o mesmo hash, ou reconstruir a entrada original a partir apenas do digest. Pequenas alterações na entrada geram saídas completamente diferentes (efeito avalanche), o que reforça a utilidade do algoritmo como mecanismo de verificação de integridade: caso os dados sejam modificados, o hash resultante deixa de coincidir com o originalmente calculado (Cyfrin, 2025).

Esse algoritmo é amplamente utilizado em sistemas de registro distribuído, em especial no ecossistema Ethereum, onde é exposto pela EVM sob o nome `keccak256`. Em plataformas desse tipo, transações, blocos e estruturas auxiliares recebem hashes Keccak-256 que permitem rastreabilidade e denunciam adulterações. Nos contratos inteligentes, o hash é empregado na derivação de seletores de funções, na indexação de eventos e na composição de identificadores e compromissos criptográficos em processos *on-chain*. Dessa forma, desenvolvedores e auditores podem armazenar e validar resumos de dados em vez de seu conteúdo completo, o que simplifica a verificação de integridade e a construção de provas de não alteração ao longo do tempo (Cyfrin, 2025).

2.4 Tecnologia Blockchain

A tecnologia blockchain vem sendo discutida amplamente como ferramenta inovadora para garantir confiança, transparência e auditabilidade em processos digitais. Sua estrutura distribuída possibilita registros rastreáveis e imutáveis, com grande potencial para a aplicação em votações em meios eletrônicos, gestão de identidade e registros financeiros (BAMBARA; ALLEN, 2018).

O conceito da blockchain começou por volta de 1991, com Stuart Haber e W. Scott Stornetta, ao proporem um sistema em que os registros de data e hora fossem “imutáveis”. Mas foi só em 2008, com Satoshi Nakamoto, que trouxe uma grande notoriedade para a tecnologia com um projeto, o BTC ou *Bitcoin* que foi publicado com o relatório técnico "*Bitcoin: A Peer-to-Peer Electronic Cash System*".

A blockchain é composta por uma cadeia de blocos interligados, em que cada bloco contém um conjunto de dados, um registro de tempo (carimbo temporal) e um *hash* criptografado que representa o bloco anterior. Essa estrutura baseada em funções *hash*, por conta de sua forma sequencial e encadeamento de blocos, assegura a integridade dos dados armazenados.(TABATABAEI; VITENBERG; VEERARAGHAVAN, 2022)

A principal característica apresentada pelas inúmeras criptomoedas atualmente ativas, como o BTC, é a ausência de um órgão central/instituição financeira que detém o papel de intermediador de qualquer transação realizada, um controle descentralizado (NAKAMOTO, 2008). A confiabilidade e a auditabilidade são garantidas pelo sistema de nós em uma grande rede encadeada que tem a responsabilidade de autenticação das transações (BAMBARA; ALLEN, 2018).

Antes de abordar os conceitos relacionados a blockchain e sua arquitetura, podemos usar como base a evolução dos sistemas de processamento de transações e o papel da

blockchain. Dessa forma, é possível dar um pequeno passo para compreender o porquê ela garante confiabilidade e auditabilidade.

Durante os anos de 1960, organizações bancárias precisaram encontrar uma maneira de guardar um grande volume de dados, dentre eles: registros bancários e transações. Como medida, a IBM lançou para a indústria um computador que tinha como objetivo solucionar este problema, nomeado *mainframes*. Esses *mainframes* tinham arquitetura centralizada e permitiam acesso e manipulação por um único terminal direto, mas eram muito limitados (ALVES et al., 2020, p. 167), foi necessário o desenvolvimento de um novo modelo.

Em 1980, com a ascensão dos computadores pessoais, surgiu o modelo cliente/servidor, abrindo as possibilidades para deixar a limitação de um mainframe como uma unidade única de trabalho, centralizado, para um modelo dividido. Desta forma, as transações podiam ser realizadas de forma remota, tornando-o independente de acessos pelo terminal direto (ALVES et al., 2020, p. 167). Por um lado, facilitou o acesso às informações, mas, com essa mudança, uma preocupação surgiu com a segurança do sistema, a exposição era maior em rede (NIST, 2008).

A principal desvantagem são as brechas criadas para acessos indevidos ao servidor, já que o servidor é o responsável por prover o acesso e realizar as operações de armazenar e tratar/trabalhar com os dados. Servidores abertos sem um sistema de segurança são suscetíveis a ataques e invasões (NIST, 2008).

Neste cenário, a blockchain aparece para corrigir os problemas, provendo sistemas de segurança sobre a rede para esses dados e um controle descentralizado que garante segurança nas transações (BAMBARA; ALLEN, 2018; NAKAMOTO, 2008). De forma simplificada, a blockchain pode ser definida, segundo Alves et al. (2020) como uma tecnologia que faz uso de uma arquitetura distribuída e descentralizada que tem o objetivo de registrar transações de forma que um registro não possa ser alterado.

2.4.1 A Blockchain

A blockchain é uma tecnologia de registro distribuído que mudou o mercado de transações, como no BTC, e como o armazenamento de informações acontece (NAKAMOTO, 2008). Segundo Bambara e Allen (2018, p. 1), definem blockchain como um banco de dados, uma cadeia de blocos de tamanho fixo que podem conter de uma a mais transações, onde para cada transação adicionada a um bloco precisa ser validada antes de ser inserida.

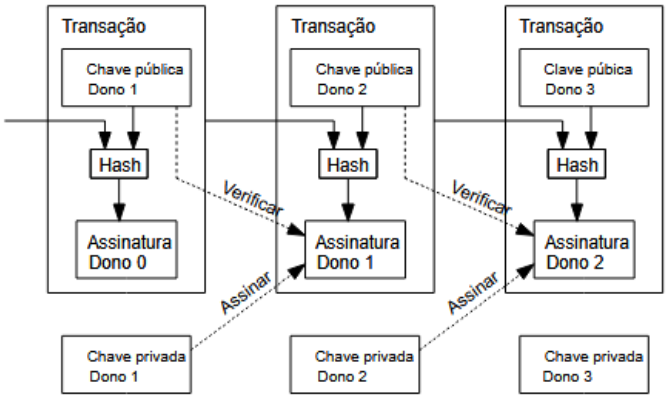
Logo após as validações, o bloco é inserido ao final da cadeia, ou seja, funciona como um registro estruturado de dados, pois possui a funcionalidade de armazenar informações. E suas vertentes definem como essas informações são armazenadas, definidas como blockchains públicas, privadas e permissionadas ou híbridas (ALVES et al., 2020, p. 174).

Essas transações são armazenadas em blocos de tamanho fixo e quando completadas, os blocos são adicionados ao final de uma cadeia de forma sequencial através de nós. E funcionam de forma que cada bloco possui o código *hash* criptografado do bloco anterior, semelhante a uma assinatura do bloco, uma impressão digital (ALVES et al., 2020, p. 169).

Assim sendo, cada bloco armazena o *hash* do anterior, sendo definido um grau de parentesco entre os blocos, definidos como bloco pai e bloco filho. o *hash* do bloco pai é aplicado no cálculo do filho. A Figura 6 ilustra essa sequência de blocos encadeados, evidenciando como cada transação é validada e assinada digitalmente antes de ser vinculada ao próximo bloco. Ou seja, caso ocorra qualquer alteração nas informações presentes no

bloco, subsequentemente o *hash* também será alterado, então o *hash* dos blocos anteriores também precisará ser recalculado, originando um efeito dominó (ALVES et al., 2020, p. 179).

Figura 6 – Uma sequência de blocos usados para criar uma blockchain



Fonte: Nakamoto (2008)

2.4.2 Tipos de Blockchain

Comumente uma blockchain pode ser categorizada de três formas, mencionadas como: pública, privada e permissionadas. A pública é dita como uma rede que tem as informações abertas ao público e permite que qualquer usuário veja e interaja com a rede. A privada pode ser acessada somente por um grupo, criado na definição da governança da rede, difundindo da ideia original de descentralização da rede pública, ela se limita a um pequeno grupo de nós, geralmente utilizada em casos de armazenamento de informações sensíveis, informações que não devem ser expostas para qualquer um (ALVES et al., 2020; BAMBARA; ALLEN, 2018).

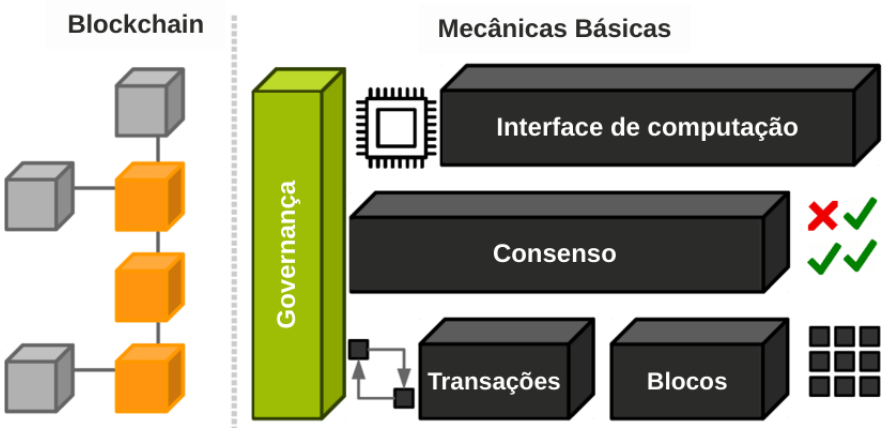
Por sua vez, a permissionada, ou híbrida, é desenvolvida de forma que o grupo detentor da governança tem a responsabilidade de disponibilizar a rede para outros grupos. Essa categoria é trabalhada de forma que os detentores da governança terão o poder de disponibilizar, ou não, a possibilidade de leitura e escrita ao público (ALVES et al., 2020; BAMBARA; ALLEN, 2018).

2.4.3 Estrutura Arquitetural de uma Blockchain

A estrutura de uma blockchain é classificada como modular e descentralizada, dividida em camadas que trabalham de forma conjunta. Essa arquitetura é desenvolvida a fim de garantir a integridade, segurança e transparência dos dados ali armazenados.

A imagem 7 proporciona um pequeno vislumbre de sua estrutura. Uma pequena ilustração, à esquerda, de uma pequena cadeia de blocos ao lado de uma exemplificação dos mecanismos que compõem a blockchain. Comumente, essas camadas são definidas em: Dados, Rede, Consenso, Contratos e Aplicação (TABATABAEI; VITENBERG; VEERARAGHAVAN, 2022). Essas camadas são uma abstração que tem a finalidade de facilitar o entendimento modular do sistema, separando os componentes, não sendo uma regra fixa.

Figura 7 – Uma visão geral da arquitetura da blockchain



Fonte: Casino, Dasaklis e Patsakis (2019)

2.4.3.1 Divisão de camadas

Do ponto de vista conceitual, uma blockchain pode ser entendida como um sistema em camadas, em que cada nível agrupa responsabilidades e abstrai detalhes do nível inferior, de forma semelhante ao que ocorre em arquiteturas clássicas de redes e sistemas distribuídos (ALVES et al., 2020). As camadas mais próximas do usuário reúnem a lógica de negócio e a interação com aplicações (por exemplo, contratos inteligentes e interfaces web), enquanto as camadas inferiores lidam com o consenso entre nós, a propagação de blocos em uma rede *peer-to-peer*, a organização criptográfica dos dados e a infraestrutura de hardware (ANDROULAKI et al., 2018).

Na camada de aplicação e apresentação situam-se os contratos inteligentes, *chaincode* e interfaces de usuário, que traduzem regras de negócio em transações executáveis na cadeia. A camada de consenso agrupa mecanismos como PoW, PoS, DPoS, PoET e PBFT, responsáveis por definir como os nós concordam sobre a validade e a ordem dos blocos, influenciando propriedades como segurança, desempenho e consumo de recursos. A camada de rede corresponde à malha *peer-to-peer* que dissemina blocos e transações, enquanto a camada de dados organiza assinaturas digitais, funções de *hash*, árvores de Merkle e o próprio modelo de transação que garante integridade e encadeamento dos blocos (ANDROULAKI et al., 2018).

Por fim, a camada de hardware e infraestrutura reúne máquinas virtuais, contêineres, serviços de suporte e mecanismos de mensageria que permitem implantar esses nós em ambientes reais, inclusive em redes permissionadas modernas (ANDROULAKI et al., 2018). A Tabela 2, adaptada de Cointelegraph (2025), sintetiza essa organização ao associar cada camada conceitual a exemplos concretos de componentes utilizados em diferentes plataformas.

2.4.3.2 Camada de Consenso e Rede da Blockchain

A camada de consenso tem a função de permitir que os participantes da blockchain entrem em acordo quanto ao estado atual da rede. É nessa camada onde são definidas as

Tabela 2 – Divisão de camadas da blockchain

Aplicação e apresentação	Smart Contracts, Chaincode, DApps, UI
Consenso	PoW, PoS, DPoS, PoET, PBFT
Rede	Peer-to-Peer (P2P)
Dados	Assinatura digital, Hash, Árvore de Merkle, Transação
Hardware / Infraestrutura	Maquina virtual, Containers, Serviços, Mensagens

Fonte: Adaptado de (Cointelegraph, 2025).

regras e mecanismos de validação pelos quais novos blocos são validados e enviados ao histórico compartilhado (CASINO; DASAKLIS; PATSAKIS, 2019).

O conceito de consenso é descrito na computação distribuída, usado na blockchain, como um acordo entre os usuários da rede para definir qual vai ser o estado final-atual do bloco que será distribuído para todos os nós da rede. Quando ou qual bloco será inserido na rede é feito de acordo com os mecanismos de consenso implementados. Existem diversos mecanismos de consenso e é válido o estudo para definir qual melhor se encaixa dependendo do contexto da aplicação. Popularmente, em criptomoedas, blockchains públicas, os mais utilizados são o *Proof-of-Work* (PoW) e o *Proof-of-Stake* (PoS), e em blockchains permissionadas ou privadas, são usados modelos como *Proof-of-Authority* (PoA) e *Practical-Byzantine-Fault-Tolerance* (PBFT) (ALVES et al., 2020).

Proof-of-Work (PoW). Baseado em poder computacional, os nós competem para resolver um problema matemático complexo, e o primeiro a apresentar a prova válida transmite um bloco com a solução para a rede. O alto custo energético e de hardware gera um alto custo, por outro lado, isso faz com que ataques sejam muito caros. A confirmação dos blocos é gradual quanto mais blocos são adicionados depois, mais difícil fica desfazer aquele registro, o que garante maior segurança com o tempo (NAKAMOTO, 2008).

Proof-of-Authority (PoA). Nesse modelo, comum em redes permissionadas, apenas um grupo de validadores previamente autorizado pode criar e confirmar novos blocos. Esses validadores são identificados por suas chaves e seguem regras de governança definidas pelo operador ou grupo responsável pela rede. A admissão, remoção e supervisão da reputação desses validadores também ficam sob responsabilidade dessa entidade gestora. O resultado é alta velocidade e simplicidade operacional, em troca de um nível de confiança institucional nas autoridades envolvidas (CASINO; DASAKLIS; PATSAKIS, 2019).

Tolerância prática a falhas bizantinas (PBFT). Os protocolos tolerantes a falhas bizantinas permitem que a rede continue operando mesmo quando alguns validadores agem de forma maliciosa. No PBFT, um líder propõe o bloco e os demais validadores passam pelas etapas de *pre-prepare*, *prepare* e *commit*, até que se forme um quórum que confirma a decisão. Esse processo garante finalidade determinística, ou seja, uma vez aprovado, o bloco não pode ser revertido. O protocolo suporta até f validadores defeituosos em um total de $3f + 1$, em troca de maior volume de comunicação entre os nós. Por isso, é especialmente eficiente em redes com número moderado de validadores, ao mesmo tempo em que fornece confirmações rápidas (CASTRO; LISKOV, 1999).

Proof-of-Stake (PoS). Ao contrário do PoW, em que a segurança depende do poder computacional, o PoS baseia-se no comprometimento econômico dos validadores. Para participar do consenso, o agente precisa bloquear uma quantia de tokens como garantia (*stake*). A ideia central é que validar blocos se torna um direito proporcional ao risco assumido; logo, atacar a rede deixaria de ser apenas uma questão de força computacional e passaria a ter custo patrimonial direto, o que desestimula comportamentos mal-intencionados (KING; NADAL, 2012; BUTERIN, 2020).

Na prática, o protocolo seleciona periodicamente um nó para propor o próximo bloco, seja por rotação, sorteio ou outro critério, enquanto um conjunto de validadores atua como comitê. Como esse *stake* pode ser punido em caso de fraude, a própria estrutura econômica do sistema fornece incentivo para que os validadores ajam corretamente. Em muitas implementações de Prova de Participação, onde se define um período de vinculação de stake (*bonding*) e um período de desvinculação de stake (*unbonding*). Esses intervalos funcionam como uma janela de segurança: permitem que comportamentos maliciosos sejam detectados e penalizados (*slashing*) antes que o agente consiga liberar integralmente seu capital (BUTERIN, 2020).

Esse desenho torna o PoS especialmente adequado para contextos não financeiros, nos quais a segurança institucional depende mais de responsabilidade e governança verificável do que de poder computacional. Onde os validadores são identificáveis, mas precisam de incentivos para não se colidirem, o stake atua como um "penhor"de boa conduta. Para cenários de votação, isso é relevante: quem certifica o resultado é economicamente responsabilizado por sua atuação, ao mesmo tempo em que os votantes não precisam de infraestrutura técnica especializada (BUTERIN, 2020).

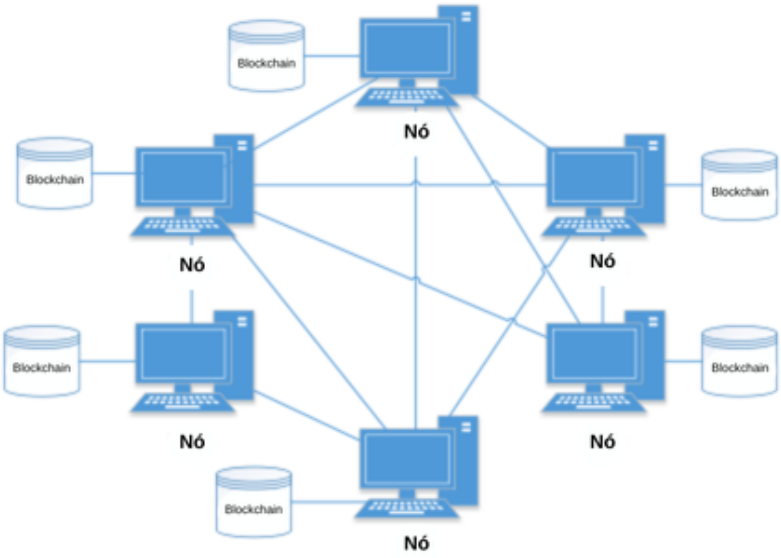
Em votações eletrônicas de ambientes privados, como consórcios ou sistemas condominiais, o PoS combina eficiência energética com transparência verificável, mantendo o permissionamento sem se limitar a poucas autoridades fixas, como ocorre no PoA. Isso permite a pluralidade de validadores com responsabilização econômica proporcional, preservando a auditabilidade, a integridade da apuração e a defesa contra tentativas de captura, sem comprometer a confidencialidade do voto (HARDJONO; LIPTON; PENTLAND, 2018).

2.4.3.3 Rede da Blockchain

A rede de uma blockchain, em sua maioria dos casos, é descentralizada, casos como blockchains privadas e permissionadas onde há uma camada de autoridade central que rege as permissões dos usuários, quem pode ou não visualizar ou interagir com a rede, e quem valida as transações. Os nós da rede recebem a função de validação de transações e a rede desenvolvida precisa encaminhar essa mensagem através dos nós, e o mais utilizado é o protocolo *Peer-to-Peer* (P2P). A Figura 8 exemplifica essa arquitetura, mostrando como os nós estão interconectados em uma topologia plana, na qual cada participante pode validar e retransmitir transações (ZHAI et al., 2019, p.2).

Todos os nós da blockchain são interconectados sobre uma topologia plana de ponto-a-ponto, não havendo um nó central. Os nós têm liberdade para entrar e sair da rede, podendo haver transações entre eles livremente.

Figura 8 – Blockchain P2P Network



Fonte: Mollah, Zhao e Niyato (2017)

2.4.3.4 Camada de Dados da Blockchain

A camada de dados constitui a base estrutural, responsável pela organização e armazenamento das informações registradas na rede. É nela que transações, blocos e outros registros são codificados de forma imutável, garantindo rastreabilidade e integridade dos dados utilizando algoritmos criptográficos (NARAYANAN et al., 2016).

Os blocos são a unidade básica de dados da blockchain, uma estrutura denotada por ser responsável por armazenar informações sobre um conjunto de transações. Uma transação em um bloco é formada por uma data, proprietário e mais uma informação adicional, no BTC, seria o valor transferido, em um sistema de votação, o próprio voto, por exemplo. Além disso, o bloco possui o seu identificador ou impressão digital. Garantindo unicidade e permitindo a identificação do bloco e a permanência de um registro. O registro histórico é mantido pela cadeia gerada entre os blocos, o laço criado entre eles com o uso do *hash*. O registro temporal é uma instância de tempo com informações sobre a data e hora, essa característica incrementa na segurança, pois, além do identificador único, um bloco possui essa propriedade que varia de um a um (HABER; STORNETTA, 1991).

2.4.3.5 Camada de Computação e Contratos Inteligentes (*Smart Contracts*)

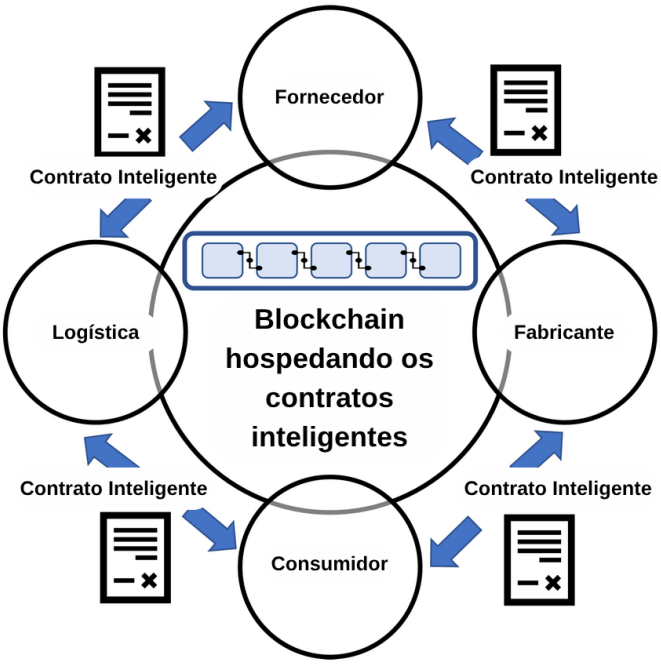
A camada de computação é uma abstração da união das camadas de aplicação e de contratos. Tendo como a sua principal função a possibilidade de aplicações e execução de lógicas de programação dentro da estrutura da blockchain, operações autônomas realizadas sobre os dados registrados. Essa lógica programável é possível através da aplicação de contratos inteligentes, trechos de código armazenados e executados na blockchain que, quando acionados através de certas condições, são atendidos, aumentando a confiabilidade pela eliminação de intermediários. Além dessas programações, é nessa camada onde ocorrem as integrações com interfaces que interagem com os dados, aplicações web, *Application Programming Interface* (APIs) e painéis (TABATABAEI; VITENBERG; VEERARAGHAVAN, 2022).

Os contratos inteligentes são como contratos digitais armazenados em uma rede blockchain e executados automaticamente quando determinadas condições previamente programadas são atendidas. Funcionam como agentes autônomos dentro da rede, podendo ser acionados por usuários, sistemas externos ou até mesmo por outros contratos. Para isso, recebem um endereço único, semelhante ao de uma transação, que permite sua identificação e chamada (ALVES et al., 2020).

Eles apresentam características fundamentais da blockchain: imutabilidade, transparência e descentralização. Isso significa que, uma vez publicados na rede, não podem ser alterados. Por esse motivo, devem ser previamente analisados e revisados, visto que modificações posteriores não são possíveis. A única forma de intervenção prevista é a utilização de funções específicas que encerram sua execução, removendo o contrato da rede (ALVES et al., 2020).

A Figura 9 representa a blockchain como infraestrutura comum que hospeda contratos inteligentes entre fornecedor, fabricante, logística e consumidor. Esses contratos automatizam regras de negócio e registram transações de forma rastreável, permitindo coordenação entre os atores sem depender de intermediários centrais.

Figura 9 – Funcionamento do Smart Contract



Fonte: Alqarni et al. (2023)

Outro aspecto relevante é que a execução destes exige um custo computacional que varia de acordo com a complexidade das instruções implementadas. Esse custo é pago pelo usuário que o invoca, garantindo que os recursos da rede sejam utilizados de forma eficiente (ALVES et al., 2020).

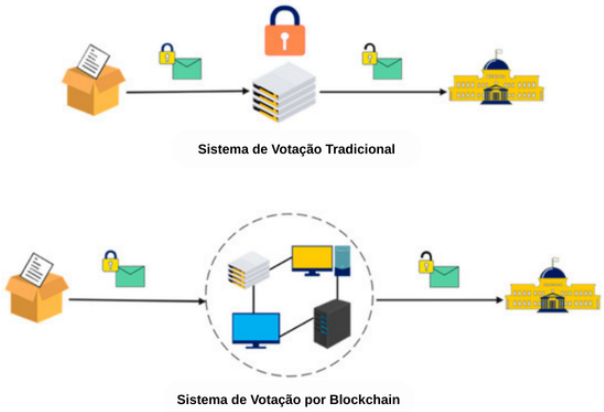
Diversas aplicações podem ser observadas, indo além da simples automatização de acordos entre partes. Em sistemas de votações eletrônicas, por exemplo, destacam-se usos como: registro e armazenamento seguro de votos, verificação da elegibilidade, contabilização automática de resultados e disponibilização de auditoria para os participantes autorizados

da rede. Assim, a imutabilidade e a transparência dos contratos inteligentes tornam o processo confiável, auditável e resistente a fraudes (ALVES et al., 2020).

2.4.3.6 Sistemas de votação eletrônica usando blockchain

A utilização de blockchain em sistemas eletrônicos de votação é muito significativa para os processos atuais, pois se tornou uma solução viável para superar os problemas atuais encontrados nas votações eletrônicas convencionais. A Figura 10 evidencia esse contraste, mostrando a diferença entre o modelo tradicional, baseado em um servidor central, e o modelo blockchain, descentralizado e distribuído. Se a aplicação do modelo for feita corretamente, a tecnologia cumpre o seu papel de se tornar um registro geral digital, atuando de forma descentralizada, criptografada e transparente, gerando resistência a fraudes e, principalmente, manipulação. Entretanto, o voto eletrônico via blockchain necessita de uma infraestrutura completamente distribuída (JAFAR; AZIZ; SHUKUR, 2021).

Figura 10 – Comparativo de votações: tradicionais vs blockchain



Fonte: Jafar, Aziz e Shukur (2021)

Por conta dos inúmeros riscos relacionados ao voto eletrônico convencional, este por si só não se torna uma opção viável de implementação, pois uma vez hackeado, as consequências serão de longo alcance. Porém, como uma rede blockchain é inteira, descentralizada, aberta e orientada para o consenso, o design deste tipo de produto garante que uma fraude não possa ocorrer, desde que a implementação esteja completamente correta (JAFAR; AZIZ; SHUKUR, 2021).

A votação utilizando a tecnologia seria basicamente um pedaço criptografado de dados, em que tudo seria aberto e armazenado publicamente em uma rede distribuída, ao invés da utilização de um único servidor. Por conta do processo de consenso, cada voto seria criptografado, validado e registrado em diversas cópias distribuídas no livro-razão da rede blockchain (JAFAR; AZIZ; SHUKUR, 2021).

2.4.4 Frameworks

O ecossistema de desenvolvimento em blockchain amadureceu nos últimos anos, com forte consolidação em torno de ambientes e ferramentas abertas que aceleram o ciclo de vida de contratos inteligentes (XU et al., 2017). Diferentes camadas de software cobrem

desde clientes de nó (execução e consenso) até *toolchains* de compilação, testes, migrações e integração com aplicações web.

No contexto de blockchain, um *framework* não é apenas um conjunto de ferramentas isoladas, mas uma organização em camadas que padroniza como as aplicações são desenvolvidas e executadas. Em vez de resolver tudo por conta própria, o desenvolvedor passa a se apoiar em convenções estáveis (APIs, ABI, JSON-RPC) e em padrões de interoperabilidade, que conferem previsibilidade ao fluxo de desenvolvimento. A partir dessa perspectiva, a camada de execução e consenso fornece o ambiente onde o código realmente roda; a camada de programação traduz a lógica de negócio em contratos; e a camada de engenharia cuida do ciclo de vida e da automação (XU et al., 2017).

Na camada de execução e consenso, o framework corresponde ao runtime padronizado da rede: é ali que transações são propagadas, ordenadas e finalizadas, e onde a máquina virtual atualiza o estado global. Essa camada define regras que todas as aplicações assumem como estáveis como limites de custo de uma transação (ou gás[1]), formato de blocos, logs e recibos e também as interfaces que conectam carteiras, exploradores e provedores de RPC. A interoperabilidade ocorre porque redes compatíveis com a mesma máquina virtual e ABI podem executar o mesmo *bytecode* (WOOD, 2021).

Na camada de programação, o *framework* aparece como um modelo de componentes usado para transformar requisitos em contratos executáveis. A linguagem e o compilador definem como funções são selecionadas, como eventos são emitidos e como o contrato se comporta ao ser implantado. Esses artefatos de compilação, como bytecode e ABI, seguem padrões estáveis que permitem integração entre módulos distintos (SOLIDITY, 2024). Exemplos incluem Solidity, Vyper, Yul/Huff e bibliotecas padronizadas como OpenZeppelin.

Por fim, a camada de engenharia organiza o ciclo de vida do projeto e converte o desenvolvimento em processo repetível. As toolchains impõem convenções de estrutura, fornecem automação para tarefas recorrentes e fazem a ponte entre os contratos e a rede. Também permitem instrumentar métricas e gerenciar migrações por ambiente, oferecendo um pipeline previsível que consiste em construir, testar, implantar, verificar e monitorar (Nomic Foundation, 2024). Exemplos incluem Hardhat, Foundry, Truffle e bibliotecas cliente como `ethers.js` e `viem`.

2.4.4.1 Ethereum

Ethereum é uma rede *blockchain* descentralizada e uma plataforma de desenvolvimento de software que opera com a criptomoeda *ether* (ETH). Lançada em 2015 por Vitalik Buterin e colaboradores, oferece um ecossistema programável baseado em contratos inteligentes, apto a hospedar transações de criptoativos e aplicações diversas, como finanças descentralizadas (DeFi), tokens não fungíveis (NFTs) e redes sociais descentralizadas (Ethereum Foundation, 2025a).

No ecossistema Ethereum, *tokens* são representações digitais emitidas por contratos inteligentes que seguem interfaces padronizadas. Em termos gerais, classificam-se em fungíveis, cujas unidades são indistinguíveis, por exemplo o ERC-20, e não fungíveis (NFTs), cujas unidades possuem identidade própria e não são intercambiáveis uma a uma.

[1] *gás* é a unidade de medida do custo computacional de uma transação ou chamada de contrato inteligente. Cada operação consome uma certa quantidade de gás, que é convertida em valor monetário multiplicando-se o total de gás usado pelo *gas price* definido na rede, resultando no custo pago.

NFTs constituem um mecanismo prático para expressar titularidade de itens únicos como ativos digitais diretamente na cadeia (Ethereum Foundation, 2025b).

As *Ethereum Request for Comments* (ERC) correspondem a propostas de padronização de interfaces de contratos, promovendo interoperabilidade entre carteiras, aplicativos que operam em rede blockchain e ferramentas. Esses padrões não dizem respeito ao consenso da rede, mas especificam a "linguagem comum" de funções e eventos. Para tokens, destacam-se o ERC-20 (fungível), o ERC-721 (não fungível) e o ERC-1155 (multiativo). O ERC-721, em particular, define uma interface mínima que inclui, entre outras, operações para consultar o proprietário de um token específico, apurar o número de tokens associados a um endereço, realizar transferências entre endereços, com ou sem verificações adicionais de segurança, e registrar essas movimentações. A adesão a essa interface permite que carteiras e bibliotecas compatíveis reconheçam, listem e transfiram o ativo de forma padronizada, sem necessidade de adaptações específicas (Ethereum Foundation, 2025b; ENTRIKEN et al., 2018).

2.4.4.2 Solidity

Solidity é a linguagem de programação construída especificamente para a criação de contratos inteligentes na rede Ethereum. Com ela, é possível definir regras e comportamentos para as aplicações descentralizadas que rodam e funcionam em Ethereum (SOLIDITY, 2024).

2.4.4.3 Hardhat

Hardhat é um ambiente completo de desenvolvimento para softwares Ethereum. Foi construída com o intuito de facilitar, de forma flexível, o desenvolvimento dos contratos inteligentes. Além disso, ajuda na implementação de nós Ethereum locais, que inclui contas pré carregadas para testes (Nomic Foundation, 2025).

2.5 Trabalhos Relacionados

Com o intuito de contextualizar este trabalho, esta seção apresenta estudos e iniciativas que exploram o uso da blockchain aplicados a sistemas de votação. São abordadas propostas que visam aumentar a segurança, a auditabilidade e a transparência de processos de votação em diferentes contextos, incluindo votações governamentais, institucionais, etc.

2.5.1 Proposta de Modelo para Votação Eletrônica Utilizando Blockchain e Contratos Inteligentes

O artigo de Neto e Vasconcelos (2025) apresenta um modelo de votação eletrônica em ambiente compatível com a Máquina Virtual Ethereum (EVM), baseado em contratos inteligentes que estruturam o ciclo de vida do pleito em fases (configuração, registro, votação e apuração), com garantias de voto único, janelas temporais e trilha de auditoria por eventos e transações na cadeia. A proposta enfatiza a integridade e a verificabilidade dos registros, discutindo viabilidade operacional e econômica e apontando a necessidade de um arranjo permissionado para assegurar governança e previsibilidade de operação.

No contexto deste trabalho, a referência é adotada como base de modelagem de entidades, políticas e fases, preservando a instrumentação por eventos on-chain e a rastreabilidade dos estados, porém com adaptação da camada de consenso para Proof of Stake (PoS) em rede privada. Essa adaptação mantém as propriedades de imutabilidade e auditabilidade do modelo original, ao mesmo tempo em que introduz governança por staking e uma finalidade operacional coerente com os requisitos de assembleias e de conformidade do domínio.

Em termos de implementação, a proposta está alinhada ao uso de *Solidity* e ao pipeline de desenvolvimento e testes com *Hardhat*, permitindo a compilação, implantação e validação automatizadas em ambiente local. Com isso, o desenho de contratos, a definição de eventos e os scripts reprodutíveis integram-se à arquitetura adotada, assegurando verificabilidade independente, rastreabilidade de votos como transações e coerência com os objetivos de segurança, transparência e auditabilidade estabelecidos para a rede privada PoS.

2.5.2 *Blockchain-Based E-Voting System*

Este artigo apresenta uma arquitetura de votação eletrônica sobre uma plataforma compatível com EVM, onde contratos inteligentes estruturam o processo em fases e expõem eventos e transações que permitem a auditoria e a verificabilidade independente do resultado. A solução adota uma rede permissionada para assegurar a governança e o controle de acesso, preservando as propriedades de integridade e imutabilidade necessárias ao contexto de votação. A modelagem é compatível com práticas de desenvolvimento consolidadas no ecossistema Ethereum, com ênfase na rastreabilidade de estados e separação clara de papéis e permissões.

No contexto deste estudo, a implementação por Hjálmarsson et al. (2018) é posicionada como estrutura base para ambiente privado, mantendo o desenho dos contratos, a instrumentação por eventos e a organização das fases do pleito. Preservando a verificabilidade e auditabilidade on-chain e introduz-se governança por *staking* e finalidade operacional adequadas ao cenário de assembleias, sem exigir alterações estruturais no modelo de dados do contrato. A migração do consenso para PoS atende aos requisitos de segurança e de previsibilidade do sistema, mantendo compatibilidade com as políticas de identidade e de acesso do ambiente permissionado.

Em termos de implementação, a proposta harmoniza-se com o uso de *Solidity* e com o pipeline de desenvolvimento e testes baseado em *Hardhat*, viabilizando compilação, implantação e validação reprodutíveis em rede privada. A instrumentação por eventos e os recibos transacionais suportam auditoria contínua, inspeção de logs e aferição de regras como voto único e janelas temporais. Dessa forma, o trabalho serve como referência prática para organização de contratos, scripts de automação e critérios de teste no contexto da arquitetura privada adotada.

2.5.3 *System Design & Implementation of Hyperledger Fabric E-Voting System: A Case Study of the Lebanese American University*

O trabalho de Shamout (2023) descreve o projeto e a implementação de um sistema de votação eletrônica sobre Hyperledger Fabric, com ênfase em identidade baseada em certificados, canais e políticas de endosso, e execução de *chaincode* para garantir integridade

transacional e auditabilidade em ambiente permissionado. A solução apresenta separação clara de papéis, governança explícita dos nós validadores e mecanismos de controle de acesso, promovendo previsibilidade operacional e conformidade com requisitos institucionais. Os artefatos de configuração, a definição de políticas e o fluxo de transações compõem um arcabouço metodológico sólido para sistemas de votação em redes com forte gestão de identidade.

Em termos de implementação e testes, a adaptação harmoniza os conceitos de governança e auditoria de Fabric com o pipeline de desenvolvimento e validação em *Solidity*/*Hardhat*, permitindo a compilação, implantação e testes reprodutíveis em rede privada. A instrumentação on-chain, por meio de eventos e logs, sustentando a inspeção independente e auditoria contínua, enquanto as políticas de identidade e acesso são refletidas por papéis e permissões definidos no contrato e na configuração da rede privada PoS.

2.5.4 Transforming the voting process integrating blockchain into e-voting

O artigo por Tedyyana et al. (2024) apresenta uma arquitetura de votação eletrônica construída sobre a blockchain do ecossistema Ethereum por meio de contratos inteligentes, com foco na transparência, rastreabilidade e auditabilidade de cada etapa do pleito, desde a criação da votação e o cadastro de participantes até a submissão e apuração dos votos em um ledger imutável. A implementação é realizada em *Solidity* com suporte de *Hardhat* para desenvolvimento, testes e implantação em ambiente local/privado, o que permite validar o ciclo de vida da votação e inspecionar transações e eventos para fins de auditoria técnica.

No desenho do contrato, são especificadas funções para cadastro de candidatos e votantes, definição de janelas de votação, garantia de voto único e apuração automática, emitindo eventos em pontos críticos (registro, voto e encerramento) a fim de facilitar a verificabilidade independente por meio de logs on-chain e exploradores locais. O uso do *Hardhat* padroniza scripts de compilação, migração e testes automatizados, viabilizando um fluxo reprodutível de validação funcional que condiz com o *toolchain* previsto neste trabalho (*Ethereum*, *Solidity* e *Hardhat*).

Em termos de avaliação, os autores demonstram a capacidade do sistema de registrar cada voto como uma transação verificável com referência ao contrato, preservando a privacidade por ausência de vínculo direto voto–identidade e assegurando integridade e imutabilidade do resultado por meio do registro distribuído. O artigo reforça a adequação de ambientes locais para testes e implantação controlada de e-voting, dada a necessidade de governança, previsibilidade de custos e requisitos de auditoria do domínio, alinhando-se ao contexto de rede privada adotado no presente trabalho.

Para este trabalho, a contribuição principal está no alinhamento metodológico e tecnológico: a combinação *Solidity*, *Hardhat* e rede Ethereum privada fornece um modelo prático de desenvolvimento e validação que pode ser diretamente replicado, incluindo a organização de contratos, a estruturação de eventos, scripts de teste e deploy, e instrumentação para auditoria on-chain. Além disso, a abordagem é compatível com a decisão de operar a camada de consenso por *Proof of Stake* (PoS) em cenário privado, preservando o stack de desenvolvimento e a verificabilidade dos registros exigidos no escopo de votações condominiais.

3 Desenvolvimento

3.1 Visão Geral

Este projeto apresenta uma proposta de votação apoiada em tecnologia blockchain, implementada em *Solidity* sobre o ecossistema Ethereum e operacionalizada em ambiente *Hardhat*. A solução foi concebida para materializar quatro pilares fundamentais do escopo acadêmico: anonimato do voto, transparência pública, auditabilidade e segurança. A implementação enfatiza uma arquitetura clara e um fluxo de votação completo, capaz de servir de referência para adaptações e evoluções.

O núcleo da aplicação reside em contratos inteligentes que definem regras imutáveis para criação de votações, o registro de opções, coleta de votos e apuração. O sigilo do conteúdo do voto é garantido por um protocolo de *commit-reveal*: durante a fase de *commit*, cada votante registra *on-chain* apenas um compromisso criptográfico (*hash*) calculado off-chain; na fase de *reveal*, divulga-se o par (*opção*, *salt*[1]) que reconstitui o compromisso, permitindo a contagem correta sem expor preferências antecipadamente. A integridade do processo decorre das validações de janela temporal, unicidade de credenciais e consistência dos compromissos, enquanto a transparência emerge da disponibilidade pública de eventos e estados suficientes para recontagem independente.

Para controle de participação, o projeto pode vincular o direito de voto à posse de um token de elegibilidade no padrão ERC-721, conforme discutido na Seção 2.4.4.1. Esse token é verificado no momento do *commit* e marcado para impedir reuso, sem comprometer o anonimato do conteúdo do voto, pois o contrato evita correlacionar identificadores *on-chain* diretamente ao compromisso revelado. O ambiente *Hardhat* proporciona a infraestrutura de desenvolvimento, implantação, simulação e testes automatizados, viabilizando a reprodutibilidade dos experimentos e a verificabilidade pública dos artefatos gerados.

3.2 Requisitos Funcionais e Não Funcionais

Esta seção consolida os requisitos que orientam a implementação da solução. Os Requisitos Funcionais (RF) expressam comportamentos observáveis do sistema na cadeia (*on-chain*) e na interação entre cliente e contrato. Os Requisitos Não Funcionais (RNF) qualificam propriedades como anonimato, transparência, auditabilidade, segurança e interoperabilidade.

3.2.1 Requisitos Funcionais (RF)

RF-1 **Criação e parametrização de votação.** O sistema deve permitir a criação de uma votação com definição de janelas de *commit*/*reveal* e do conjunto de opções válidas.

[1] Em criptografia aplicada, *salt* é um valor aleatório acrescentado ao dado original antes do cálculo do *hash*, de modo que duas opções idênticas produzam compromissos diferentes. Isso dificulta ataques e impede a dedução do conteúdo do voto apenas pela comparação de *hashes*.

RF-2 **Publicação de metadados.** O sistema deve expor, via funções, os metadados da votação (registro temporal, opções, emissor autorizado), sem necessidade de transação.

RF-3 **Registro de *commit*.** O sistema deve aceitar o registro de um compromisso criptográfico (*commitment*) associado a uma credencial, dentro da janela de *commit*, rejeitando duplicidade.

RF-4 **Verificação de assinatura do emissor.** O sistema deve verificar, no *commit*, a autenticidade da credencial por meio de assinatura ECDSA do emissor autorizado.

RF-5 **Revelação de voto.** O sistema deve aceitar, dentro da janela de *reveal*, o par (opção, *salt*) para recomputar o comprometimento e contabilizar o voto, em caso de consistência.

RF-6 **Apuração e consulta.** O sistema deve disponibilizar contagem por opção, total de votos e indicação da opção líder por meio de funções determinísticas.

RF-7 **Elegibilidade por NFT.** O sistema deve condicionar o *commit* à prova de propriedade de um `tokenId` ERC-721 no instante da submissão, marcando o identificador como utilizado para impedir reuso.

RF-8 ***Owner-only* no voto via token.** Quando a elegibilidade por NFT estiver habilitada, o sistema deve aceitar *commit* via token apenas a partir do endereço que é proprietário do `tokenId`, rejeitando operadores aprovados.

RF-9 **Emissão de eventos de domínio.** O sistema deve emitir eventos que sinalizem *commit*, *reveal*/apuração e uso de token, preservando mínima divulgação de identificadores.

RF-10 **Tratamento de erros.** O sistema deve explicitar falhas por meio de *custom errors*, por exemplo, janelas inválidas, duplicidade de *commit*, opção fora do domínio, token já utilizado.

3.2.2 Requisitos Não Funcionais (RNF)

RNF-1 **Segurança operacional.** O sistema deve prevenir reuso de credenciais e de `tokenId`, rejeitar operações fora das janelas e mitigar *front-running*[2]

RNF-2 **Anonimato do conteúdo do voto.** O sistema deve garantir que endereços ou `tokenId` não sejam correlacionados ao conteúdo do voto antes do *reveal*, nem durante a auditoria posterior.

RNF-3 **Transparência pública.** O sistema deve expor estados e eventos suficientes para permitir recontagem independente por terceiros, sem acesso a dados off-chain.

RNF-4 **Auditabilidade.** O sistema deve permitir reconstrução da linha do tempo da votação (janelas, *commits*, *reveals* e apuração) exclusivamente a partir do estado e dos eventos.

RNF-5 **Interoperabilidade.** O token de elegibilidade deve seguir a interface ERC-721, garantindo compatibilidade com carteiras e indexadores.

[2] *Front-running* é um ataque em que alguém observa uma transação pendente e envia outra com maior taxa de gás para ser incluída antes, explorando a informação antecipadamente revelada.

RNF-6 **Eficiência de gás.** O armazenamento e as operações devem ser modelados para execução em tempo constante e uso comedido de escrita.

RNF-7 **Reprodutibilidade.** O ambiente de desenvolvimento deve permitir implantação e execução determinística via Hardhat, com versões de compilador fixadas e scripts de simulação e teste.

3.3 Decisões de design

3.3.1 Modelo de consenso: *Proof of Stake* frente ao *Proof of Authority*

Optou-se por operar em redes compatíveis com *Proof of Stake* (PoS) em vez de *Proof of Authority* (PoA). Em termos simples, PoS distribui a validação entre participantes que comprometem valor econômico, enquanto PoA concentra a validação em um conjunto limitado de validadores autorizados. Para um sistema de votação que pretende ser auditável por terceiros, a descentralização prática e a resistência a captura do conjunto validador foram consideradas mais relevantes do que a latência mínima.

Tabela 3 – Comparação resumida entre PoS e PoA

Aspecto	*Proof of Stake* (PoS)	*Proof of Authority* (PoA)
Governança e confiança	Validação por participantes com *stake* econômico.	Validação por nós autorizados e identificáveis.
Finalidade prática	Redes públicas com auditoria aberta.	Redes privadas/consorciadas com controle de acesso.
Censura e captura	Maior resistência à captura quando o *stake* é distribuído.	Maior risco de captura se a lista de validadores for mal governada.
Latência e custo	Latência e custo variáveis.	Baixa latência e custos mais previsíveis.
Transparência	Alta transparência em redes públicas.	Transparência dependente da confiança nos validadores.
Adequação ao protótipo	Favorece auditoria aberta e neutralidade prática do validador.	Adequado para pilotos institucionais em redes permissionadas.

Fonte: Adaptado de (FAHIM; RAHMAN; MAHMOOD, 2023).

Em síntese, PoS reforça a transparência e a auditabilidade sem exigir confiança prévia em uma entidade operadora. PoA permanece como alternativa válida para ambientes controlados, onde menor custo e prazos rígidos de confirmação superam a necessidade de neutralidade ampliada.

3.3.2 Linguagem de contratos escolhida

Adotou-se Solidity pela maturidade do ecossistema, disponibilidade de ferramentas de verificação e testes, ampla base de exemplos e bibliotecas e compatibilidade com redes amplamente utilizadas. Alternativas como *Vyper* priorizam legibilidade e simplicidade da linguagem, mas hoje oferecem menos integração com ferramentas e bibliotecas. No contexto, a prioridade foi reduzir atrito de desenvolvimento, acelerar iterações e facilitar

auditorias por terceiros. A desvantagem é a maior complexidade de certas construções onde foi mitigado através de padrões de projeto simples, funções de leitura claras e testes unitarios focados no funcionamento.

3.3.3 Testes locais e estratégia de ensaio

Os testes locais em rede simulada oferecem três benefícios principais: execução rápida, controle de tempo e reprodutibilidade. Isso permite exercitar cenários de janela temporal e de rejeição sem custos financeiros e sem depender da variabilidade da rede pública. Por outro lado, redes de teste públicas são utilizadas para observar latência, ordenação de transações e comportamento sob condições mais realistas. O compromisso adotado foi: experimentar e validar lógica localmente e, antes de consolidar resultados, repetir fluxos críticos em rede pública de teste. Esse encadeamento fez com que houvesse um baixo custo com velocidade e confiabilidade.

3.3.4 Autorização: análise comparativa entre credencial e token

O projeto contempla dois modelos de autorização. A credencial baseada em compromisso prioriza dissociação entre identidade civil e voto, com baixo acoplamento a ativos on-chain. A autorização por token simplifica a verificação de titularidade em ecossistemas já tokenizados, mas caso mal implementado poderia introduzir riscos de correlação. A diretriz adotada foi manter os eventos mínimos necessários e evitar expor identificadores que permitam associação direta entre identidade e conteúdo do voto. Em ambientes institucionais, o modelo por token pode ser preferível pela governança e rastreabilidade; em contextos públicos, a credencial tende a oferecer melhor proteção à privacidade.

3.3.5 Observabilidade e simplicidade operacional

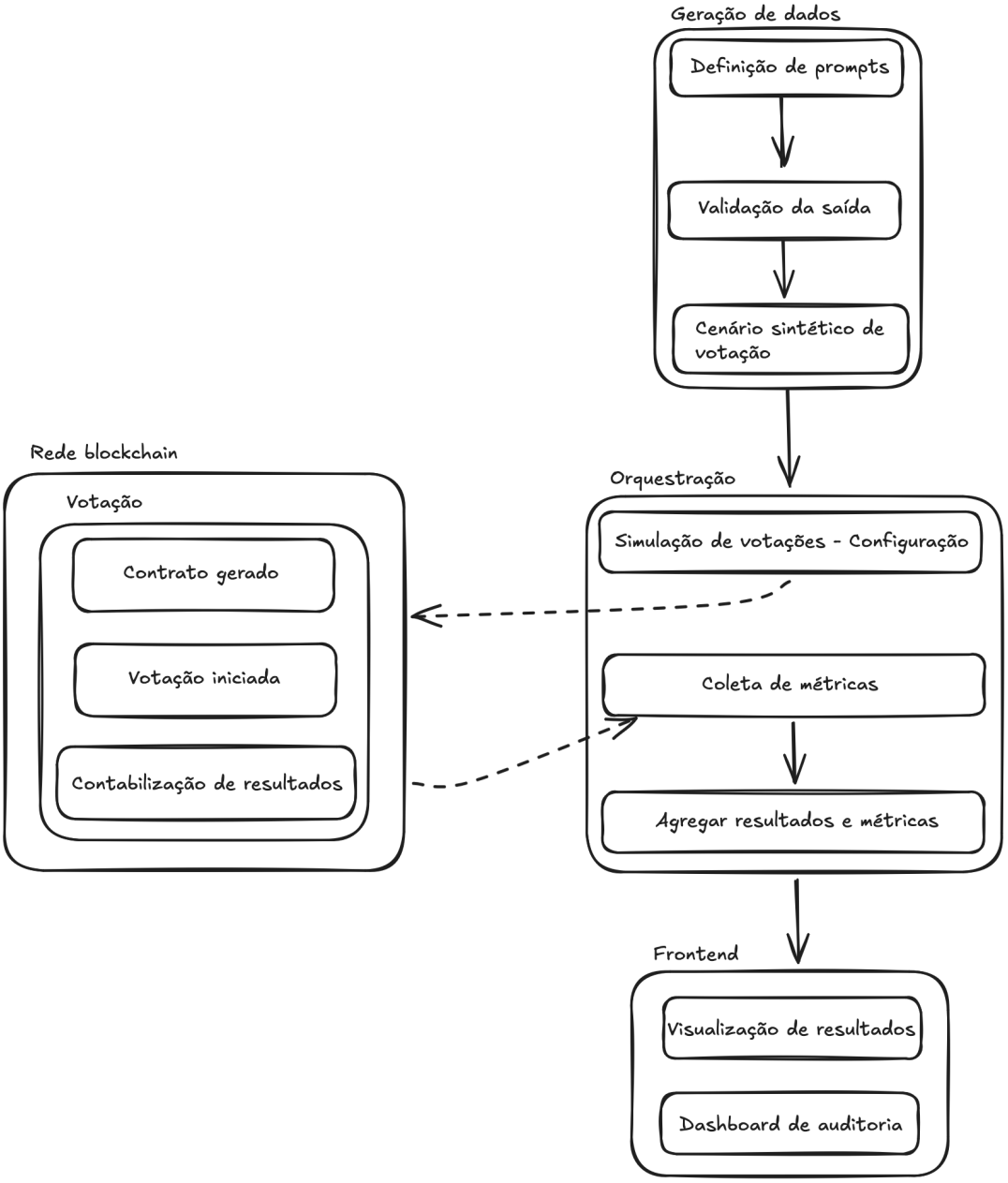
A estratégia foi expor dados suficiente para auditoria independente, mas evitar vazamento de metadados que enfraqueçam o anonimato. Eventos e funções de leitura foram desenhados para permitir recontagem e reconstrução de linha do tempo, sem exigir infraestrutura fora da rede. O custo operacional é mantido previsível por meio de janelas de votação claras, funções de leitura simples e ausência de componentes off-chain obrigatórios.

3.4 Arquitetura de Solução

3.4.1 Visão em Camadas e Diagrama Geral

A camada *on-chain* concentra as regras mínimas do domínio (janelas de commit/reveal, unicidade e apuração) e emite eventos para a auditoria. A camada off-chain engloba o *frontend*, serviços de orquestração (API/Workers), chamada de eventos e armazenamento de dados de auditoria. Essa separação de responsabilidades pode ser visualizada no diagrama 11 de arquitetura end-to-end (Frontend, Blockchain e Off-chain).

Figura 11 – Diagrama de funcionamento



Fonte: Elaboração dos autores

No diagrama, destacam-se:

(I) *Geração de dados*, onde os dados sintéticos utilizados nas simulações são gerados pela LLM/Gemma.

(II) *Orquestração*, onde ocorrem as configurações das simulações.

(III) *Rede Blockchain / Votação*, onde os contratos são gerados e as votações ocorrem on-chain.

(IV) *Frontend*, onde é realizada a visualização das métricas e resultados das votações, além dos dashboards de auditoria pública.

3.4.2 Componentes e Responsabilidades

O frontend atua como interface web predominantemente voltada à visualização dos dados, consumindo leituras da blockchain para apresentar as votações. Do ponto de vista do administrador, é possivel acompanhar as votações existentes, inspecionar metadados como janelas, opções e emissor, além de visualizar a apuração final. Do ponto de vista do votante, a interface viabiliza consultar o status da votação e da própria participação, bem como visualizar o comprovante e o resultado final disponibilizado.

Já o backend, o que inclui a API e os Workers, é onde se aplicam as regras de orquestração fora do consenso, publicam operações nos contratos e indexa eventos para gerar dados de auditoria, que por sua vez tem eventos e funções permitem reconstrução determinística da linha do tempo e recontagem independente, sem exigir infraestrutura proprietária externa.

Na parte On-chain temos os contratos que se dividem em: *SimpleVoting/TokenizedVoting* que preservam metadados imutáveis (janelas, opções, emissor), aceitam *commit* e *reveal* (opção, *salt*), contabilizam e emitem eventos de domínio (`CommitSubmitted`, `VoteRevealed`, `TallyFinalized`). Quando habilitado, *TokenizedVoting* agrega elegibilidade por NFT ERC-721.

3.4.3 Fronteiras e Interfaces

As fronteiras entre os componentes são definidas pelo conjunto de funções externas e pelos eventos de domínio expostos pelos contratos. No contexto do protocolo de *commit-reveal*, as principais operações de escrita são `commitVote` e `reveal`, responsáveis, respectivamente, por registrar os compromissos de voto e, em seguida, validar e contabilizar os votos revelados que são apresentados na Seção 3.4.5. Os detalhes da credencial derivada `credentialHash` e da derivação *off-chain* são apresentados na Seção 3.4.5.1.

Os eventos associados à participação são deliberadamente sintéticos e evitam carregar simultaneamente identificadores que possam aproximar identidade e conteúdo do voto; em particular, quando há menção a `tokenId` ou a um endereço, o evento não inclui o `credentialHash`. No que se refere à leitura, a interface pública é composta por funções de visualização, como `tally`[3] e `leadingOption` que diz quem está liderando a votação, além da consulta aos metadados da votação. Esse conjunto de operações define a superfície de transparência necessária para permitir reprodutibilidade e auditoria independente, sem expor informações suficientes para reidentificar o vínculo entre credencial e preferência de voto.

3.4.4 Ambiente de Execução e Consenso

A solução opera tanto em redes públicas compatíveis com EVM quanto em ambiente local de desenvolvimento. Quando implantada em redes públicas (testnet ou *mainnet*), como redes Ethereum e BNB Smart Chain, a aplicação passa a ser executada sobre mecanismos de consenso baseados em *stake* real. Assim sendo, há validadores que comprometem valor econômico na forma de *ether*, *BNB* ou tokens equivalentes propõem e atestam blocos, definindo a ordem e a validade das transações.

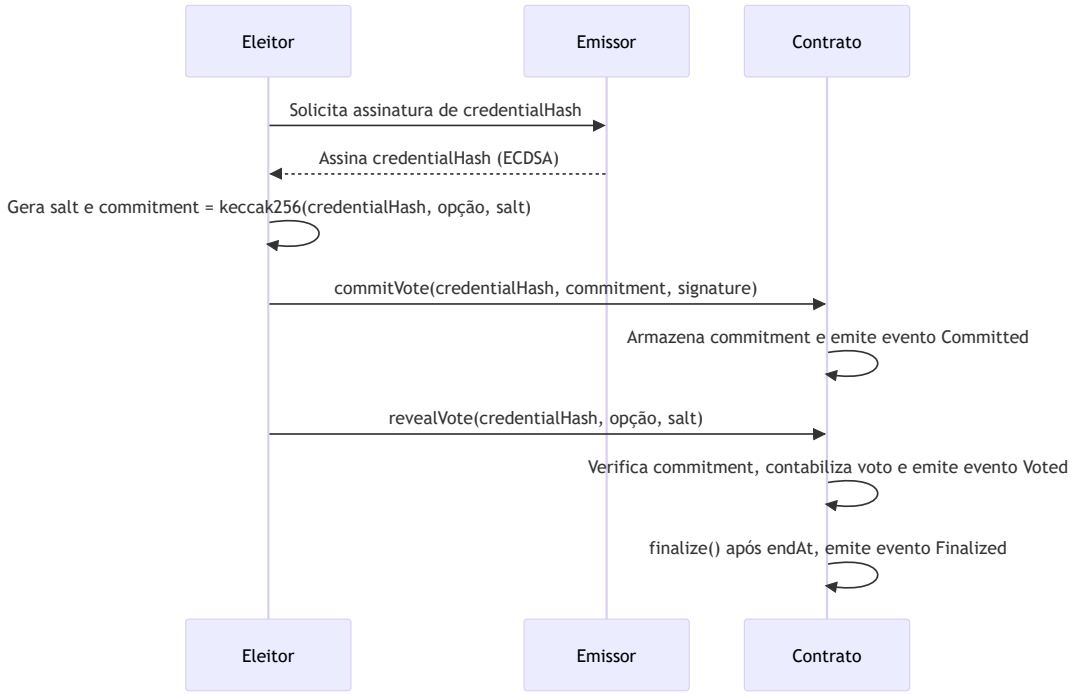
[3] No contexto deste trabalho, `tally` designa a função de apuração que retorna as contagens agregadas de votos por opção, e não os votos individuais.

Em ambiente local de desenvolvimento, utiliza-se a Hardhat Network, que é um nó único e não implementa consenso distribuído. A produção de blocos é simulada por política de mineração local: por padrão, cada transação gera um bloco imediatamente (automineração), opcionalmente, pode-se configurar intervalo fixo/aleatório de blocos ou acionar a criação de blocos e o avanço do tempo de forma programática. Não há validadores, *fork choice* ou finalização de blocos, apenas execução gerida pela EVM.

Do ponto de vista do protocolo de votação, as implicações são distintas. Na testnet/mainnet, o comportamento temporal (registro, ordem relativa entre *commit* e *reveal*) é definido pelo PoS, o que torna a demonstração mais fiel ao cenário real. Em Hardhat, as janelas são igualmente respeitadas, mas o tempo e a criação de blocos podem ser controlados pelo desenvolvedor, o que facilita testes das bordas (fechamento de janela, rejeição por atraso, recomputação do compromisso). Em ambos os casos, o desenho do *commit-reveal* e a transparência por eventos permanecem inalterados, muda apenas a camada de execução subjacente.

3.4.5 Implementação do *commit-reveal*

Figura 12 – Sequência do protocolo: credencial, `commitVote`, `revealVote` e apuração



Fonte: Elaboração dos autores

A Figura 12 sintetiza o fluxo transacional entre votante, emissor e contrato: emissão de credenciais, registro de compromissos (`commitVote`), revelação (`revealVote`) e finalização (`finalize`). Nessa configuração, o protocolo *commit-reveal* é materializado com uma separação explícita de responsabilidades: o cálculo do compromisso permanece *off-chain*, enquanto o contrato *on-chain* limita-se a validar credenciais, fases temporais e consistência

de estados, além de emitir eventos enxutos que sustentam a trilha de auditoria e permitem recontagem independente.

3.4.5.1 Fluxo de geração, *commit* e *reveal*

Na geração *off-chain*, cada votante cria um segredo de credencial, conhecido apenas por ele, e o submete à função de hash Keccak, obtendo um identificador compacto que passa a representar a sua autorização. Esse hash funciona como um "rótulo" estável da credencial: é o mesmo valor utilizado na etapa em que a autoridade assina a autorização e, mais adiante, na construção do compromisso do voto, permitindo que o contrato trabalhe sempre com o resumo em vez do segredo original. Esse identificador é enviado ao emissor, que assina digitalmente a mensagem correspondente e o devolve com uma credencial assinada. Em seguida, o votante escolhe a opção de voto, sorteia um valor aleatório e combina esses elementos com o seu endereço de carteira para produzir um compromisso criptográfico. Na fase de *commit*, o que segue para a blockchain é apenas esse compromisso, acompanhado do identificador de credencial e da assinatura do emissor, sem expor nem o segredo original nem a opção escolhida.

No fluxo *on-chain*, a etapa de *commit* no contrato `SimpleVoting` verifica se a chamada ocorre dentro da janela de tempo configurada, se ainda não existe voto associado à credencial informada e se a assinatura recebida de fato corresponde ao emissor autorizado. Somente quando essas condições são atendidas o compromisso é armazenado e um evento de *commit* é emitido para fins de auditoria. Quando o modo tokenizado está habilitado, essa mesma etapa passa a ser intermediada pelo contrato `TokenizedVoting`, que acrescenta uma verificação de posse do token de elegibilidade e registra que o identificador desse token já foi utilizado antes de encaminhar o fluxo para o contrato de votação básico.

A etapa de *reveal* também é guiada por restrições de tempo e de estado. O contrato só aceita revelações depois do encerramento da fase de *commit* e antes do término da votação, e apenas se existir um compromisso associado à credencial aceita. Nesse momento, o votante informa a opção de voto e o valor aleatório usados na geração do compromisso, e o contrato reconstrói localmente o mesmo valor criptográfico a partir desses dados. Se o resultado coincide com o compromisso armazenado e a opção pertence ao conjunto de alternativas válidas, o contador correspondente é incrementado e um evento de voto revelado é emitido; caso contrário, a revelação é descartada e não afeta a contagem.

Figura 13 – Máquina de estados da votação: fases e janelas temporais

Fonte: Elaboração dos autores

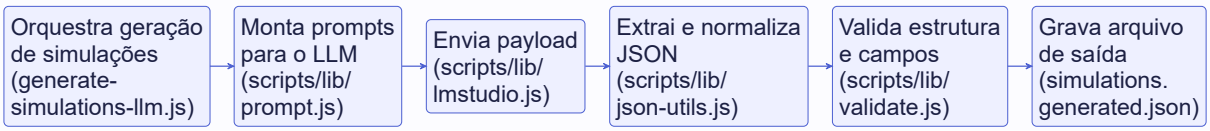
A Figura 13 resume essas regras em uma máquina de estados com quatro fases principais (*NotStarted*, *Commit*, *Reveal* e *Ended*), onde as transições são determinadas pelos marcos temporais configurados no contrato. Além dos relógios internos, a implementação mantém apenas os elementos necessários para reconstruir a votação e verificar seus invariantes: o emissor autorizado, o vínculo entre credenciais e compromissos, a contagem por opção e, na versão tokenizada, a referência ao contrato de elegibilidade e o registro de tokens já utilizados na fase de *commit*. Em conjunto, essas regras garantem que

cada credencial e, quando aplicável, cada token de elegibilidade originem no máximo um *commit*, que as fases temporais sejam respeitadas e que apenas revelações coerentes com o compromisso previamente registrado impactem os resultados apurados.

3.5 LLM e Gemma

A ideia da utilização de LLM e Gemma para o projeto se baseia na apresentação das simulações realizadas, com o objetivo de tratar com dados sintéticos para a aplicação da votação. Desta forma, é possível ter uma visualização mais semelhante a dados reais, facilitando a demonstração da prova de conceito de votações condominiais.

Figura 14 – Geração automatizada de cenários com LLM e validação



Fonte: Elaboração dos autores

A Figura 14 mostra do prompt à persistência em arquivos JSON. O processo ocorre com a integração local de uma LLM, partindo do script de geração, onde se definem as mensagens que serão enviadas ao modelo. O prompt realizado segue este padrão:

Para gerar os cenários de teste, utilizou-se um *prompt* composto por duas partes um *system prompt*, que restringe a saída a configurações JSON válidas para um contrato de votação *commit-reveal*, e um *user prompt* com regras detalhadas sobre o esquema `{"defaults": {...}, "simulations": [...]}`, limites de votos, variação de temas e cobertura de casos de teste. O texto completo desse *prompt* encontra-se no Apêndice C.

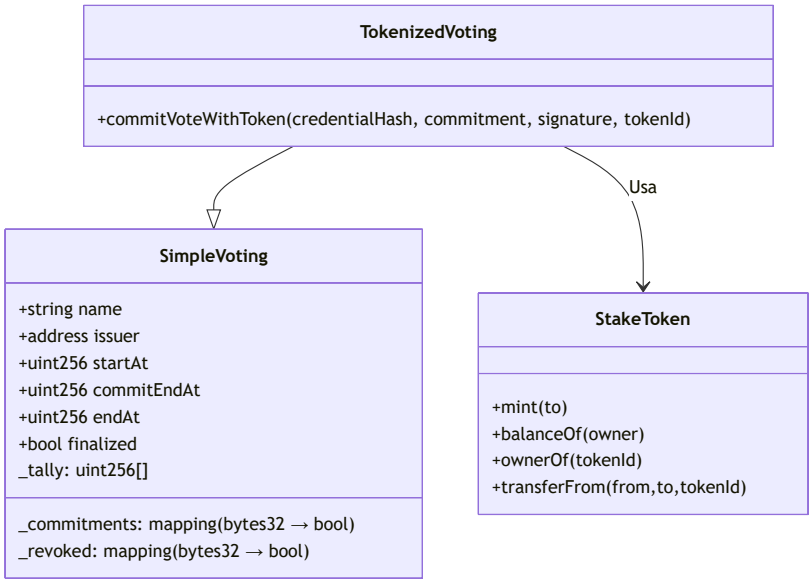
A resposta é então pós-processada para extrair apenas o trecho em JSON, que é submetido a uma etapa de validação estrutural, verificando se segue o esquema esperado (objeto `defaults` e vetor `simulations` contendo identificador, nome, opções e votos). Quando o conteúdo é considerado válido, o conjunto de simulações é persistido em um arquivo JSON, passando a compor a base de casos de teste. Essa rotina automatiza a geração de simulações diversificadas, reforça a consistência dos dados por meio de validação programática.

3.6 Contratos e Interface

A Figura 15 apresenta o modelo on-chain adotado, composto por um núcleo de votação e por um módulo opcional de autorização baseado em ativo. A separação busca reduzir acoplamento e tornar explícita a fronteira entre regras do protocolo e políticas de elegibilidade, o que facilita auditoria e evolução.

No núcleo de votação (`SimpleVoting`), permanecem os parâmetros do pleito (`name`, `issuer`, `startAt`, `commitEndAt`, `endAt`, `finalized`) e o estado mínimo para apuração. A contagem por opção é mantida em `tally`, enquanto estruturas de marcação como

Figura 15 – Modelo de contratos e principais operações



Fonte: Elaboração dos autores

`commitments` e `revoked` asseguram as propriedades do esquema *commit-reveal*, unicidade por credencial, rejeição de revelações inválidas e respeito às janelas temporais. A partir dos eventos de domínio, o resultado é recontável de forma independente, preservando transparência.

Quando a pauta exige elegibilidade por posse de um título, é utilizado o contrato `TokenizedVoting`, que acrescenta um ponto de entrada para *commit* com apresentação de `tokenId` e assinatura correspondente. A verificação de direito de voto ocorre no instante do *commit*, consultando a propriedade do token, e a marcação de uso impede reaproveitamento do mesmo identificador dentro da mesma votação. Essa extensão não altera a semântica de apuração; apenas adiciona a política de autorização ao núcleo já existente.

O ativo de elegibilidade é modelado por um token conforme o padrão ERC–721, oferecendo a superfície mínima esperada com emissão controlada (`mint`), consulta de saldo e propriedade (`balanceOf`, `ownerOf`) e transferência. O uso de um padrão amplamente suportado permite que clientes e indexadores auditem a elegibilidade via eventos do próprio ERC–721, enquanto o núcleo de votação permanece agnóstico ao histórico completo do ativo (importa apenas a propriedade no momento do *commit*).

Em conjunto, o desenho auxilia na observabilidade e integração estável. Clientes off-chain consomem a ABI dos contratos de forma determinística, diferenciando operações que alteram estado de operações de consultas. A emissão de eventos sustenta monitoramento e recontagem, e a política de autorização pode ser trocada ou removida sem reescrever as regras de contagem, preservando compatibilidade e minimizando riscos de regressão.

3.7 Segurança aplicada

Esta prova de conceito combina mecanismos criptográficos e operacionais para alcançar os pilares de integridade, anonimato e autenticidade. O desenho de cada mecanismo

fundamenta-se em teoria bem estabelecida e complementa a implementação descrita em 3.4.5.

3.7.1 Assinaturas digitais (ECDSA)

Para garantir que apenas credenciais emitidas pela autoridade sejam aceitas, cada *credentialHash* é assinado fora da cadeia utilizando o algoritmo ECDSA. O contrato rejeita compromissos cujo *hash* não esteja acompanhado de uma assinatura válida. A ECDSA é um algoritmo de assinatura digital de chave pública; uma chave privada produz a assinatura e a chave pública correspondente permite verificar sua autenticidade. Uma referência de segurança resume que as assinaturas digitais asseguram que o remetente é autêntico, que a mensagem não foi alterada e que o emissor não pode negar ter a enviado. Como apenas a autoridade detém a chave privada, é impraticável forjar credenciais, e qualquer modificação no *credentialHash* invalida a assinatura.

3.7.2 Sigilo do voto e aleatoriedade

Além do esquema commit–reveal, cada votante gera um salt aleatório e o combina com o índice da opção, seu endereço e a credencial antes de aplicar a função keccak. Esta composição torna impraticável derivar o voto a partir do compromisso e impede ataques de pré-imagem, comums para esse tipo de sistema. O segredo permanece fora da cadeia até a fase de revelação, preservando o anonimato e garantindo que o compromisso ao mesmo tempo não possa ter a opção alterada após registrar o *hash* e ninguém poderá deduzir o conteúdo antes da revelação.

3.7.3 Elegibilidade via token não fungível

Para condicionar a participação à legitimidade condominial, o voto tokenizado passa a associar o direito de voto à posse de um token de elegibilidade no padrão ERC-721, conforme discutido na Seção 2.4.4.1. Cada unidade desse token representa a possibilidade de registrar um único voto em uma pauta, de forma análoga a um título de participação associado a uma unidade condominial.

Nesse fluxo, o contrato faz a mediação a fase de *commit*, verifica se o remetente é o titular atual do identificador de stake, adota um padrão *owner-only* para o exercício do voto e marca o token como utilizado após o primeiro *commit*. Enquanto não houver voto associado, o token permanece transferível, permitindo refletir alterações legítimas de titularidade. Depois disso, novas tentativas de *commit* com o mesmo identificador são rejeitadas, mantendo a relação entre um título de elegibilidade e um único voto efetivo em cada processo.

3.7.4 Revogação e eventos de auditoria

É possível revogar credenciais, o administrador pode revogar um *credentialHash* antes da fase de commit e restaurá-lo se necessário. Esse recurso permite desabilitar credenciais suspeitas sem afetar as demais. Além disso, todos os eventos importantes do domínio, e `CredentialRevokedEvent` são emitidos com carimbos de data/hora e parâmetros mínimos. Esses eventos, somam para fornecer um histórico público suficiente

para auditoria independente. Terceiros podem reconstruir a linha do tempo da votação e validar cada etapa sem acesso a dados sigilosos.

3.8 Implementação com Hardhat e Testnet

Para desenvolver e testar os contratos inteligentes da votação, o projeto utiliza tanto a *Hardhat Network* local quanto uma testnet pública. Essa abordagem permite percorrer o ciclo de desenvolvimento de forma segura e realista, aproveitando as vantagens de cada ambiente.

3.8.1 Hardhat Network: rede local simulada

A *Hardhat Network* é uma blockchain simulada em memória, criada pelo próprio Hardhat quando os scripts são executados sem especificar uma rede externa. Conforme a documentação oficial, o Hardhat distingue dois tipos de rede: redes simuladas em memória e redes baseadas em JSON-RPC. A rede simulada é ideal para desenvolvimento por três motivos principais:

Ambiente isolado, onde todos os contratos e transações permanecem na memória e são descartados ao final da execução. É possível reiniciar a blockchain facilmente, o que facilita a repetição de testes. A pré-configuração de contas, pois o Hardhat cria 20 contas desbloqueadas com saldos elevados de ETH fictício, dispensando a configuração manual de chaves e permitindo testar interações complexas sem custos de gás. Velocidade e controle, em ambiente local os blocos são minerados instantaneamente, e parâmetros como custo base, limite de gás e carimbos de tempo podem ser ajustados via configuração. Isso viabiliza testes unitários e simulações com controle fino do tempo.

O *deploy* dos contratos começa pela obtenção de um provedor da Hardhat Network e pela listagem das contas de teste fornecidas pelo próprio ambiente. O primeiro endereço é utilizado como emissor das credenciais (`issuer`), enquanto os demais são reservados para uso como votantes. Em seguida, é definido o nome da pauta, as opções de voto e as janelas temporais da votação (início, fim da fase de *commit* e término da fase de *reveal*), calculando-se os *timestamps* em segundos a partir da data atual. Na etapa seguinte, procede-se à implantação do token de *stake*: são emitidos os NFTs que conferem o direito de votar, obtém-se a fábrica do contrato, realiza-se a implantação com nome, símbolo e URI base e, após essa etapa, cunham-se tokens para os votantes. Por fim, implanta-se o contrato de votação tokenizada: a fábrica do contrato é obtida e utilizada para implantação com os metadados definidos, o endereço do emissor e o endereço do token, mostrando no console os endereços dos contratos para futura consulta.

Na configuração, a rede hardhat é criada automaticamente. Caso seja necessário executar um nó local persistente, é possível utilizar a rede `localhost` do Hardhat, iniciada com o comando, e especificar a rede ao executar o script.

3.8.2 Testnet pública

Após os testes iniciais em rede local, o contrato também foi implantado em uma testnet pública para avaliar o comportamento em um ambiente mais próximo da realidade. Testnets são blockchains públicas com ativos de teste (*test-ether* ou equivalentes) que não possuem valor monetário real e replicam, em grande medida, o comportamento da *mainnet*.

Desenvolvedores utilizam essas redes para detectar falhas e otimizar os contratos antes de lidar com valores reais, sendo possível fazer quase tudo o que se precisa com um nó de testnet ou com uma blockchain local, sem a necessidade de executar um nó completo da *mainnet*.

Algumas diferenças práticas entre a Hardhat Network e uma testnet são os provedores e chaves: enquanto a Hardhat Network gera contas de teste automaticamente, uma testnet exige um nó remoto (RPC) e uma chave privada real. Nessa configuração, é necessário especificar o *endpoint* JSON-RPC (`url`), a lista de contas a utilizar (chaves privadas convertidas em endereços) e o `chainId` da testnet, além de parâmetros da versão do compilador Solidity.

Na testnet, a conta que realiza transações precisa de saldo em moeda de teste para pagar as taxas de gás (por exemplo, ativos de teste em redes compatíveis com Ethereum, como TBNB em testnets da BNB Chain), obtido em ambientes de teste (*faucets*) ou por mineração de teste. Em contraste, na rede local o saldo é, na prática, ilimitado. Além disso, a testnet persiste o estado entre execuções e conta com blocos minerados por participantes reais, o que permite exercitar aspectos de concorrência, latência e eventuais reorganizações que não ocorrem em um ambiente totalmente isolado, ao custo de sujeitar as transações a atrasos de confirmação ou reordenações. Por fim, a implantação de contratos em testnet é realizada por meio do mesmo *script* utilizado na rede local, alterando-se apenas o nome da rede de destino, o que facilita a repetibilidade dos testes e a comparação entre os diferentes ambientes de execução.

Durante o processo de implantação, a conta responsável pelo *deploy* é obtida a partir das funcionalidades de gerenciamento de carteiras do ambiente de desenvolvimento, permitindo registrar seu endereço na saída do procedimento e estimar o custo de gás associado. A interação com o contrato na testnet é feita por meio de provedores conectados a um *endpoint* remoto, seja utilizando o próprio ambiente de execução do *framework* de testes, seja por meio de bibliotecas de cliente para Ethereum compatíveis com EVM. Por questões de segurança, é recomendável que chaves privadas e URLs de acesso sejam armazenadas em variáveis de ambiente ou mecanismos equivalentes, evitando sua exposição direta no código.

Foi escolhido por empregar a BNB Chain (testnet) disponibilizada pela Binance como infraestrutura pública de teste. Trata-se de uma rede compatível com a EVM, gerenciada pela Binance, que permite executar contratos inteligentes em uma blockchain real e aberta sem exposição a valores monetários, utilizando o ativo de teste TBNB. Para a execução foi exigido três carteiras distintas (uma para o emissor de credenciais e duas para votantes), todas com saldo de TBNB de teste obtido por meio de *faucets* públicos.

A simulação foi construída de forma a poder ser executada tanto localmente, via Hardhat Network, quanto na testnet pública, apenas alternando a configuração de rede. Os resultados observados permaneceram equivalentes entre os ambientes e, na testnet, puderam ser acompanhados pela interface gráfica disponibilizada pela Binance, que permite inspecionar transações e blocos relativos às fases de criação do contrato (pauta) e ao protocolo de *commit–reveal*.

3.9 Transparência e auditoria

A transparência é tratada como uma propriedade do desenho do contrato e do fluxo *commit-reveal*, e não apenas como um requisito isolado. O objetivo é permitir que qualquer observador reconstrua a linha do tempo da votação usando exclusivamente dados on-chain, refaça a contagem de forma determinística e verifique a coerência de tempo das operações com as janelas definidas, preservando o anonimato por meio de compromissos criptográficos que separam a identidade do conteúdo. O sistema mostra confirmações de operações válidas por eventos e funções de leitura e deixa explícitas as rejeições por erros específicos, o que facilita auditorias de pontos que não seguem o caminho pretendido.

3.9.1 Mecanismos on-chain, reconstituição e auditoria

Os pilares de transparência e a auditabilidade são obtidas pela combinação de mecanismos on-chain e um procedimento único de verificação. Os mecanismos são os eventos que registram transições relevantes de estadol e funções de leitura sem efeitos colaterais, como `tally()` e `leadingOption()`

A auditoria unifica reconstituição da execução, recontagem independente e consolidação de artefatos. Parte-se do bloco de implantação para coletar sequencialmente os eventos; cada registro é associado ao andamento da votação no bloco, verificação temporal e remetente. A conformidade temporal é verificada confrontando os carimbos com o período da fase de commit e reveal. Caso haja inconsistências, elas serão sinalizadas, como duplicidades por `credentialHash` ou reutilização de `tokenId`. A recontagem agrega as opções dos votos válidos e compara com as funções de leitura no mesmo bloco (ou posterior sem novos eventos), garantindo determinismo. Nos casos negativos, as mensagens de erro evidenciam que as restrições foram aplicadas.

3.10 Testes e Validação

Esta seção apresenta os objetivos, o ambiente de execução e o plano de verificação adotados para validar o protocolo *commit-reveal*, a autorização por NFT e as regras de consistência: janelas temporais, unicidade de credenciais e contagem discreta. Os critérios derivam dos requisitos funcionais e não funcionais como mostrado em 3.2, com ênfase em anonimato, transparência pública, auditabilidade, segurança operacional e reprodutibilidade.

Os testes automatizados são conduzidos prioritariamente na *Hardhat Network* (blockchain simulada em memória), pela velocidade, controle de tempo e contas pré-configuradas, e são replicados em rede de testes pública para avaliar latência, ordenação e persistência de estado. A estrutura do *Hardhat* padroniza compilação, execução e testes, favorecendo reprodutibilidade. A execução em rede de testes reutiliza os mesmos scripts, apenas parametrizando a rede.

3.10.1 Casos de Teste

O1 Verificar o fluxo *commit-reveal* de ponta a ponta, garantindo que apenas cédulas válidas atualizam a apuração e que conteúdos não são revelados antes da fase de *reveal*.

O2 Assegurar rejeição de operações fora das janelas temporais de envio e de revelação, bem como a unicidade por `credentialHash`.

O3 Validar a contagem determinística nas funções de leitura `tally()` e `leadingOption()`, possibilitando recontagem independente.

O4 Impedir reuso de `tokenId` e preservando anonimato baseado no proprietário do token, quando função habilitada.

O5 Verificar funcionamento sobre erros intencionais, como janelas inválidas, duplicidade, opção fora do domínio e inconsistência do compromisso criptográfico.

3.10.2 Escopo e organização dos testes

Os testes foram estruturados em três níveis complementares: unidade, integração e propriedades de consistência. Essa divisão busca isolar as garantias internas de cada contrato, validar o comportamento conjunto entre componentes e assegurar que as constantes mais importantes da blockchain permanecem válidas em todas as execuções, independentemente da ordem de chamadas ou de condições temporais.

3.10.2.1 Testes de unidade (on-chain)

Os testes de unidade testam diretamente as interfaces do contrato principal, simulando interações típicas de um votante. Cada função é avaliada individualmente quanto às suas pré-condições, efeitos colaterais e pós-condições observáveis.

A função `commitVote(...)` é testada quanto a persistência do compromisso criptográfico, composto pela concatenação de opção, `salt` e endereço do remetente, garantindo que o valor registrado em cadeia não permite a dedução da escolha. Já a função `reveal(...)` é avaliada quanto a recomputação do mesmo hash a partir dos parâmetros revelados `(option, salt)`, assegurando que apenas revelações válidas atualizem o contador `tally`.

Os eventos de domínio são validados como evidências de que as transações foram aceitas e resultaram em alterações de estado coerentes. Esses eventos também funcionam como uma trilha de auditoria, permitindo a reconstrução da sequência de ações sem necessidade de acesso a dados off-chain.

3.10.2.2 Testes de integração com NFT

Os testes de integração validam o comportamento do sistema do mecanismo de autorização por token não fungível. Nesse cenário, o voto é autorizado pela posse de um token e não por uma credencial hash. A função `commitVoteWithToken(...)` é responsável por executar essa verificação, aplicando a política de voto apenas pelo proprietário do voto.

Durante os testes, é avaliado se apenas o titular legítimo do token consegue efetuar o envio e se, após o voto, o identificador `tokenId` é marcado como utilizado, impedindo reutilizações. Adicionalmente, é confirmado que o evento de uso do token é emitido sem incluir o `credentialHash`, de modo a preservar a separação entre identidade e voto e, consequentemente, o anonimato. Esses casos de teste também asseguram que o contrato

resiste a tentativas de reuso ou de falsificação de propriedade, características essenciais para a robustez do modelo.

3.10.2.3 Propriedades globais

A última categoria de testes é voltada à validação das propriedades globais do sistema, que devem permanecer verdadeiras mesmo sob diferentes combinações de transações ou estados intermediários. As principais constantes verificadas são:

1. Janelas válidas de *commit* e *reveal*: nenhuma operação de envio ou revelação deve ocorrer fora dos intervalos temporais definidos no contrato. Essa regra é verificada por meio de manipulação controlada do carimbo temporal do bloco na rede de teste.

2. Unicidade de credencial: cada `credentialHash` pode ser utilizado apenas uma vez, impedindo votos duplicados e assegurando a correspondência um-para-um entre votante e participação.

3. Domínio de opções: somente identificadores de opções previamente cadastrados são aceitos, garantindo a integridade dos resultados e prevenindo contaminação da contagem.

4. Recontagem determinística: a soma de votos computada pelas funções de leitura `tally()` e `leadingOption()` deve ser idêntica à reconstruída por análise de eventos, confirmando a transparência e verificabilidade do processo.

5. Mitigação de antecipação de ordens (*front-running*): o compromisso incorpora o endereço do remetente ao cálculo do hash, de forma que uma terceira parte não possa antecipar ou reproduzir um voto a partir de observação de transações pendentes. Os testes confirmam que qualquer tentativa de reaproveitar o mesmo hash com outro remetente falha por inconsistência na verificação.

A combinação desses três níveis de teste permite validar não apenas o funcionamento isolado das funções, mas também o comportamento emergente do sistema em condições realistas, cobrindo desde as garantias criptográficas básicas até a conformidade com princípios de anonimato, transparência e não-repúdio previstos no modelo.

4 Resultados

4.1 Cobertura funcional dos cenários

Com 16 execuções controladas, distribuídas entre dois modos de votação: um baseado em participação simples e outro em participação por meio de títulos de voto tokenizados. A maior parte dos testes valida o modo simples, enquanto uma parcela específica comprova o comportamento quando o direito de votar depende da posse de títulos digitais.

As simulações verificam se o sistema conduz uma votação completa, abordando cenários de empates e até oito alternativas na mesma pauta, se os critérios de quem pode votar permanece correto mesmo quando há mudança de posse do título durante a janela de votação, se é possível alternar quem emite as credenciais de acesso, variar a população de votantes e pré-atribuir títulos de voto quando necessário e se um mesmo participante pode ter mais de um título e como isso afeta o cômputo. Em conjunto, esses cenários reproduzem situações típicas de assembleias como aprovação de reformas, escolha de fornecedores e alocação orçamentária e demonstram que o sistema se mantém correto sob diferentes composições de participantes e regras.

4.2 Conformidade do fluxo *commit–reveal* e regras de negócio

Os casos de testes automatizados (unidade e integração) verifica aspectos centrais do protocolo, como mostrados na Seção 3.10:

1. Sigilo até o *reveal*: os votos não são inferíveis antes da janela de revelação; compromissos inválidos ou atrasados são rejeitados.
2. Unicidade por credencial: cada `credentialHash` só é aceito uma vez por pauta, impedindo duplo voto.
3. Elegibilidade acoplada ao NFT (quando habilitado): a elegibilidade acompanha o *owner* do token ERC/721; transferências *antes do commit* movem o direito de votar, e transferências *após o commit* não afetam votos já comprometidos.
4. Determinismo de apuração: contagens (`countsByOption`) e líder (`leadingOption`) são determinísticas e reprodutíveis, suportando recontagem independente.
5. Janelas temporais: operações fora de `commitStart/commitEnd` e `revealStart/revealEnd` são corretamente rejeitadas.

Em todos os cenários definidos no arquivo de simulações, a apuração final coincidiu com a soma dos votos válidos revelados e consistente com as regras de elegibilidade especificadas no cenário.

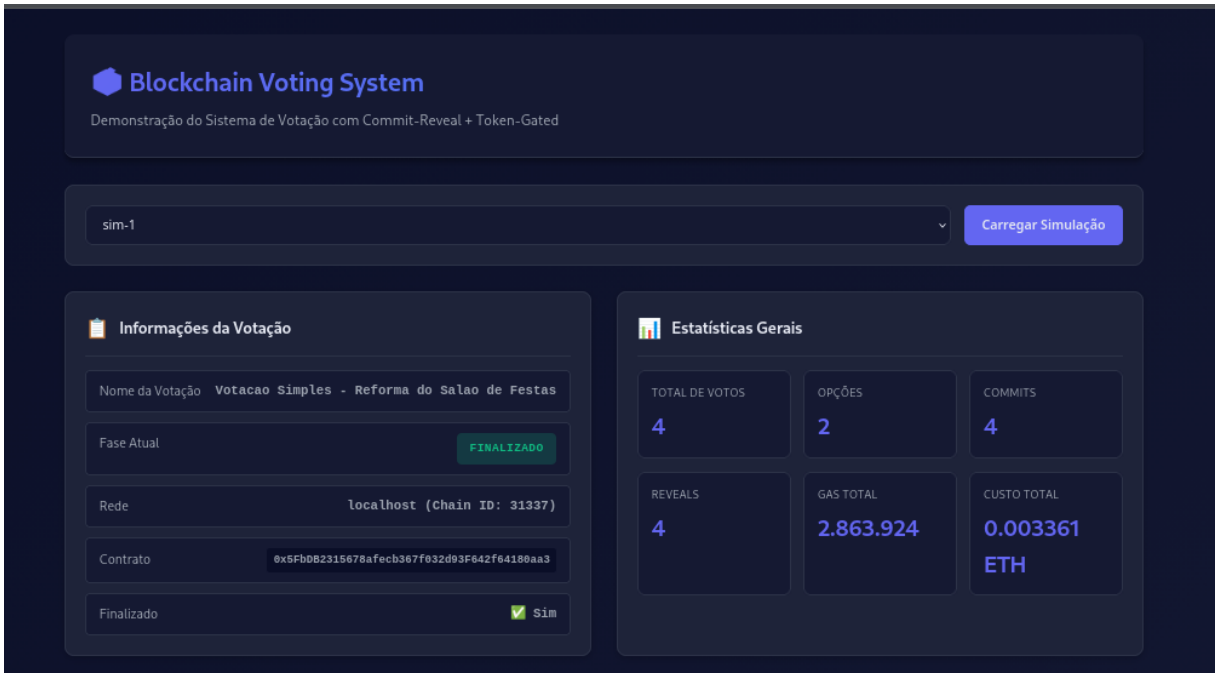
4.3 Transparência e trilha de auditoria

Do ponto de vista de auditabilidade, os resultados obtidos permitem afirmar que a recontagem é isenta, uma vez que a leitura *on-chain* das funções de apuração reproduz o mesmo resultado do roteiro de simulação, sem depender de qualquer estado externo à cadeia. Verifica-se, ainda, o isolamento de identidade, pois o uso de credenciais assinadas fora da cadeia, combinadas com a verificação *on-chain*, preserva simultaneamente o vínculo entre autorização e anonimato operacional do voto. Ademais, cada simulação gera um *payload* de eventos e leituras que funciona como artefato de auditoria, possibilitando a reconstrução detalhada da linha do tempo da votação, abrangendo criação, fase de *commit*, fase de *reveal*, contagem e divulgação do resultado.

4.3.1 Visualização de dados

Abaixo, demonstramos imagens que representam o frontend deste projeto, com as camadas de visualização e controle.

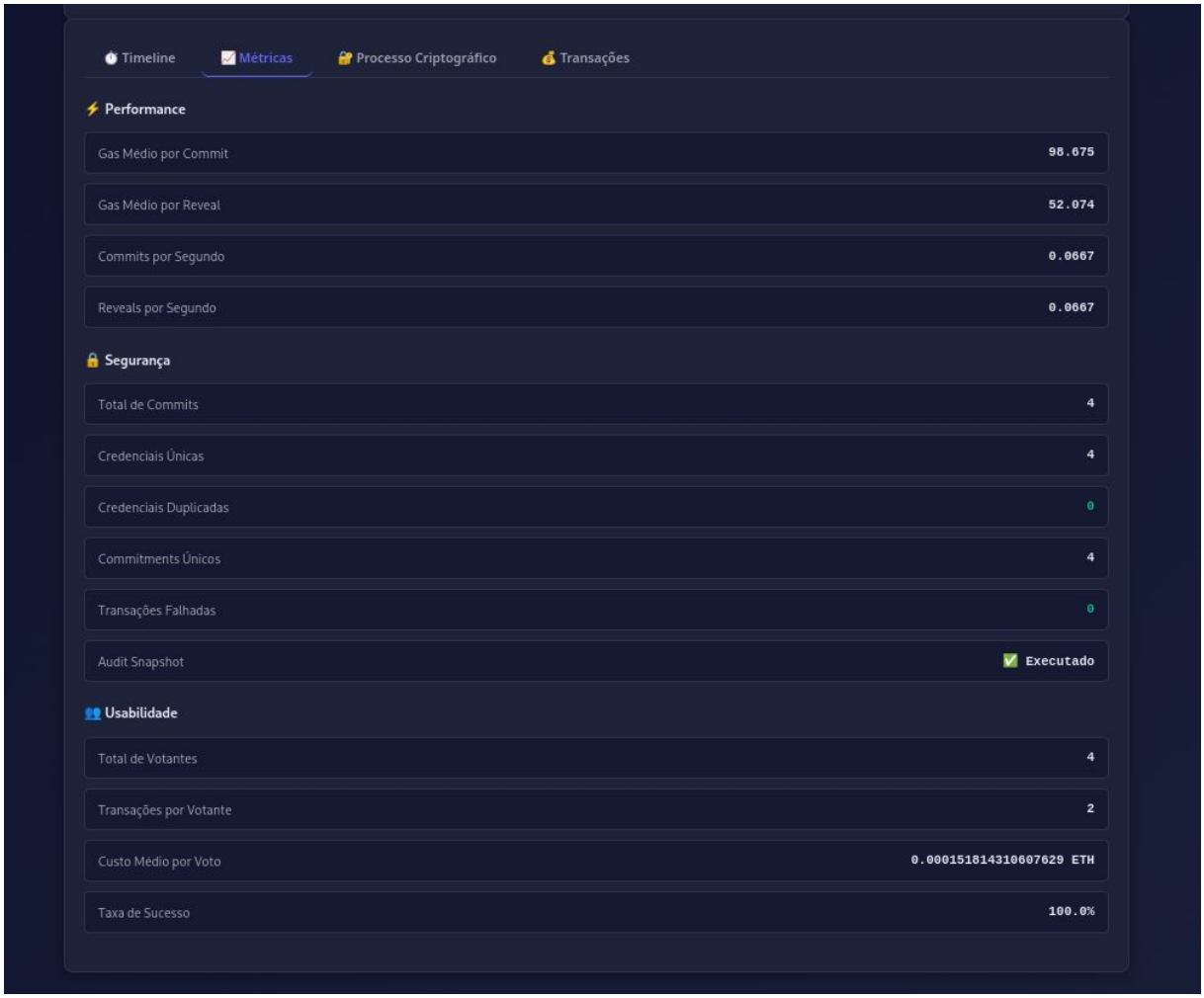
Figura 16 – Visão geral do sistema de votação em blockchain



Fonte: Elaboração dos autores

A Figura 16 apresenta a tela principal do sistema, na qual é possível observar o estado final da votação, o número total de votos, commits e reveals, bem como métricas de custo associadas à execução do protocolo na rede blockchain.

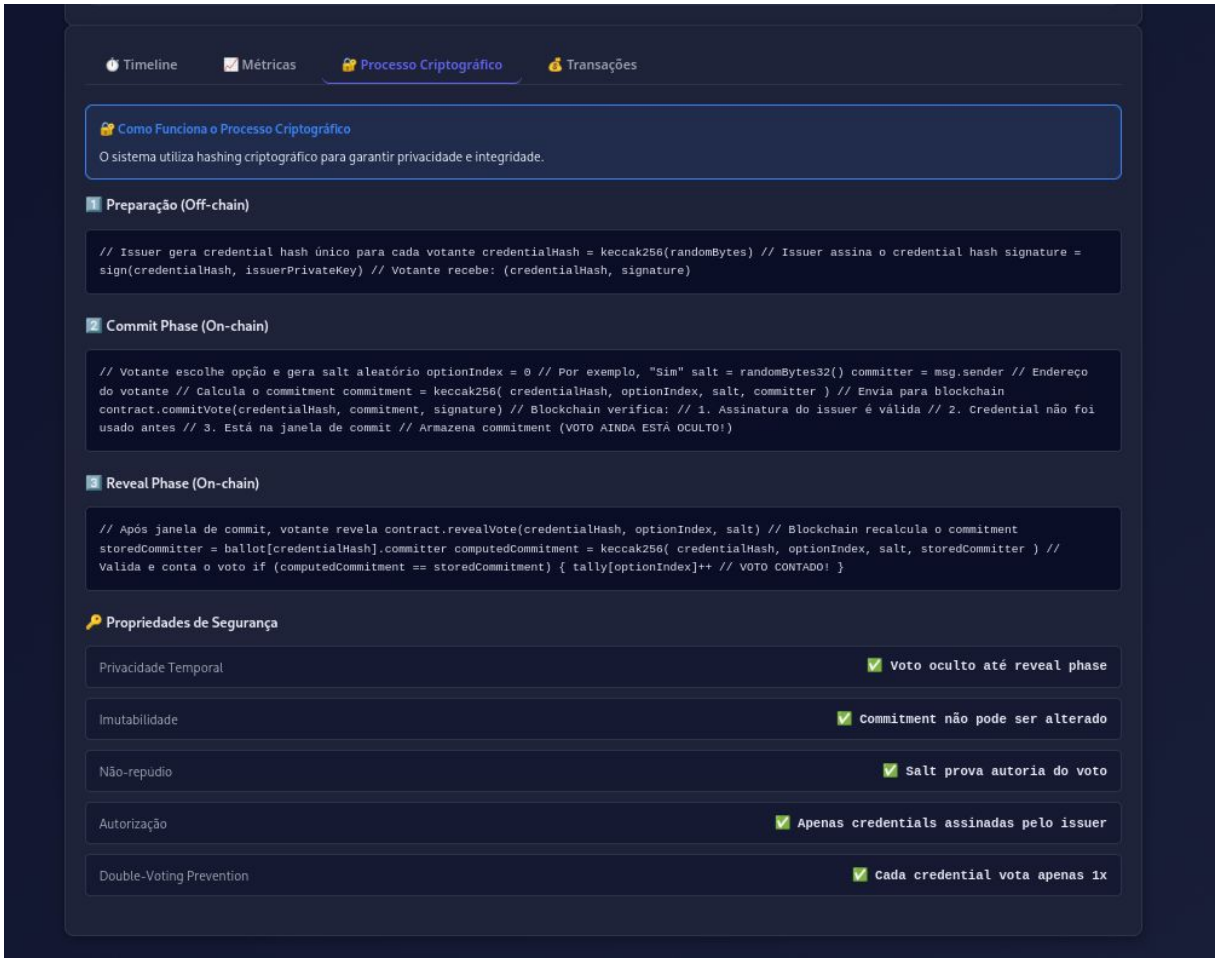
Figura 17 – Métricas operacionais da simulação de votação



Fonte: Elaboração dos autores

As métricas apresentadas na Figura 17 permitem avaliar o comportamento do sistema sob diferentes cenários simulados, fornecendo evidências quantitativas sobre eficiência, confiabilidade e previsibilidade de custos.

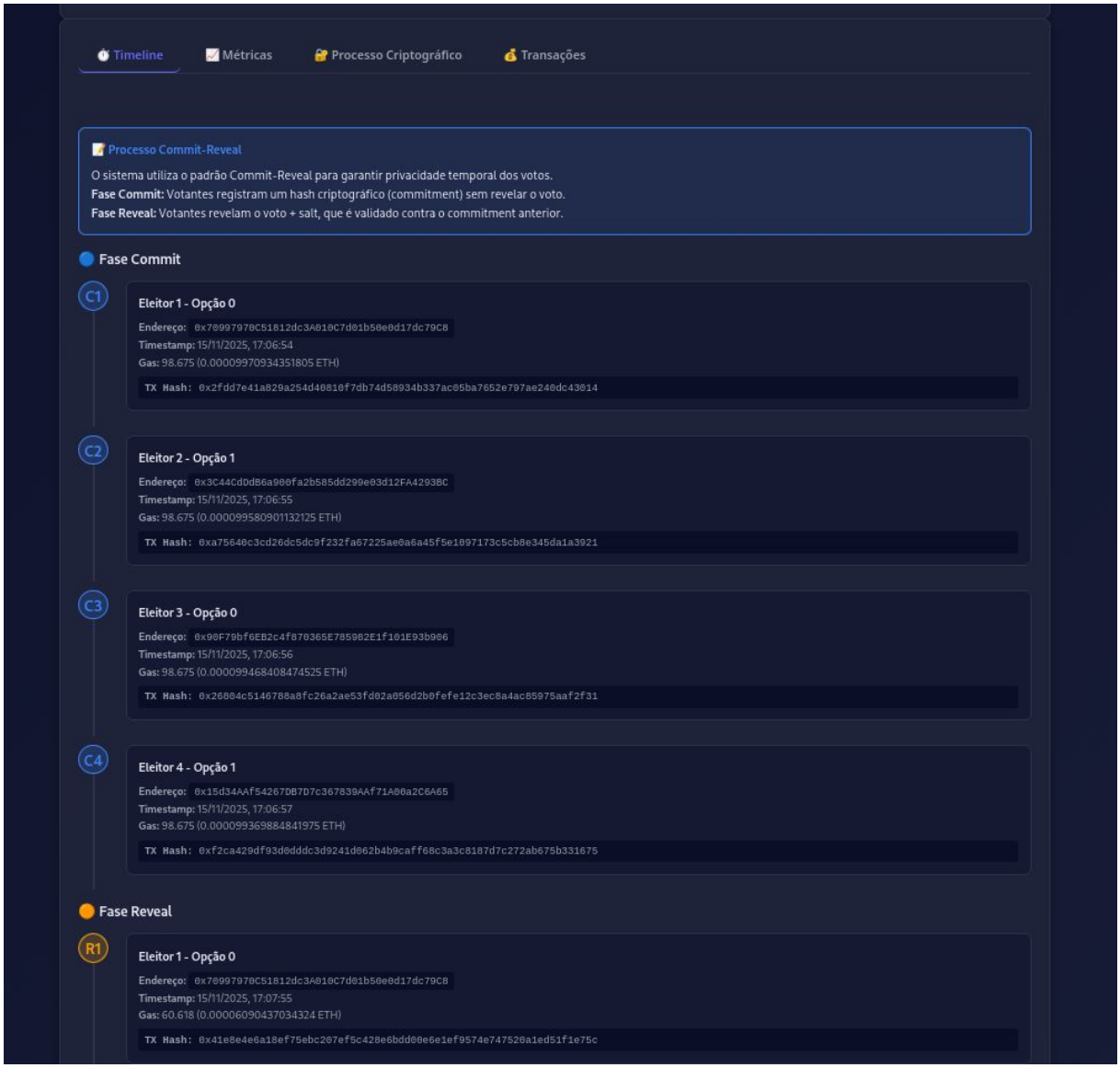
Figura 18 – Processo criptográfico do protocolo commit–reveal



Fonte: Elaboração dos autores

A Figura 18 explicita o fluxo criptográfico adotado, evidenciando como o voto permanece oculto durante a fase de compromisso e como a integridade é verificada na fase de revelação.

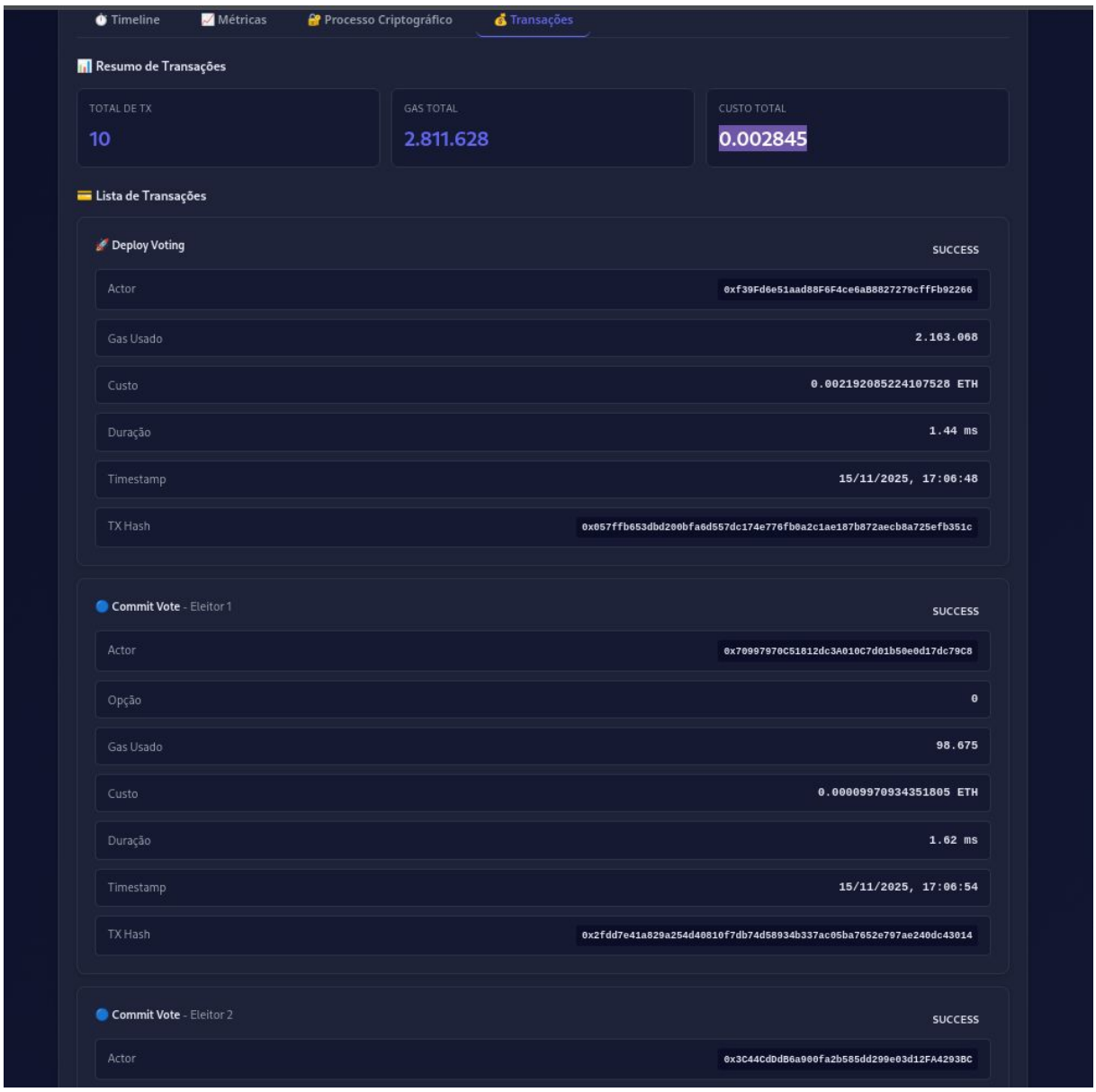
Figura 19 – Linha do tempo das fases de commit e reveal



Fonte: Elaboração dos autores

Por fim, a Figura 19 demonstra a capacidade de reconstrução completa da linha do tempo da votação a partir dos dados registrados on-chain, reforçando a auditabilidade e a transparência do sistema proposto.

Figura 20 – Registro das transações on-chain da votação



Fonte: Elaboração dos autores

A visualização detalhada das transações, apresentada na Figura 20, permite a validação independente das operações realizadas, funcionando como um artefato de auditoria alinhado aos objetivos de transparência e verificabilidade do projeto.

4.4 Métricas empíricas e custo operacional

Como apoio a decisões de implantação, recomenda-se medir sistematicamente tempo de execução e custo de gás nas principais operações do protocolo. A Tabela 4 resume as métricas agregadas obtidas nas simulações realizadas neste trabalho, apresentando valores mínimos, máximos e médios observados para cada indicador.

Tabela 4 – Métricas agregadas

Métrica	**Unidade**	**Máximo**	**Mínimo**	**Média**
Gás por votação	gas	4127251.00	2357705.80	2930484.22
Custo por voto	ETH/voto	0.000255	0.000211	0.000235
Votos válidos por ETH	VV/ETH	4739.36	3919.72	4277.21
Votos por segundo	votos/s	0.133300	0.033300	0.056931

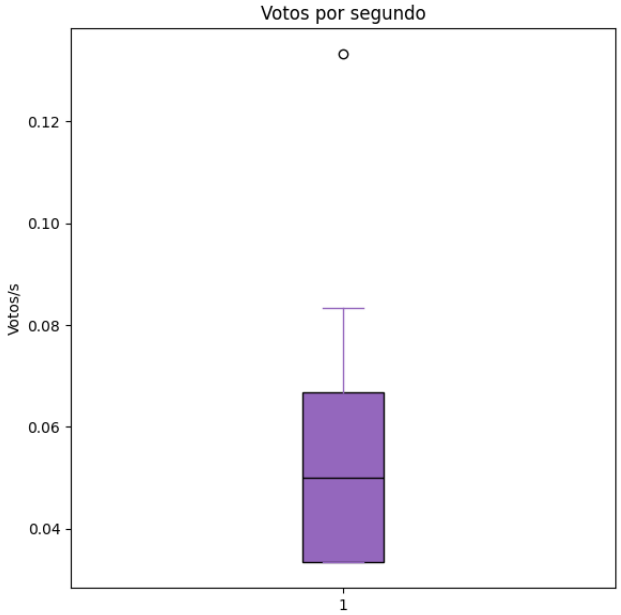
Fonte: Elaboração dos autores

Em ambiente local (Hardhat Network), observa-se latência sub-segundo por transação. Em *testnets* públicas, a latência está sujeita à variabilidade de rede e à capacidade do provedor RPC. Já em redes de produção privadas (PoS), o custo por operação depende da configuração de gás e da política de taxas definida pelo operador, de modo que os valores da Tabela 4 servem como ordem de grandeza para calibração inicial.

As simulações executadas permitiram, ainda, caracterizar a distribuição empírica de desempenho e custo. A Figura 21 apresenta a distribuição da taxa de processamento em votos por segundo ao longo dos experimentos. É visível um processamento estável, variação relativamente contida, o que indica que, mesmo em cenários com número reduzido de nós validadores, o sistema consegue manter uma fluxo compatível com assembleias de pequeno e médio porte, onde os picos de concorrência são limitados.

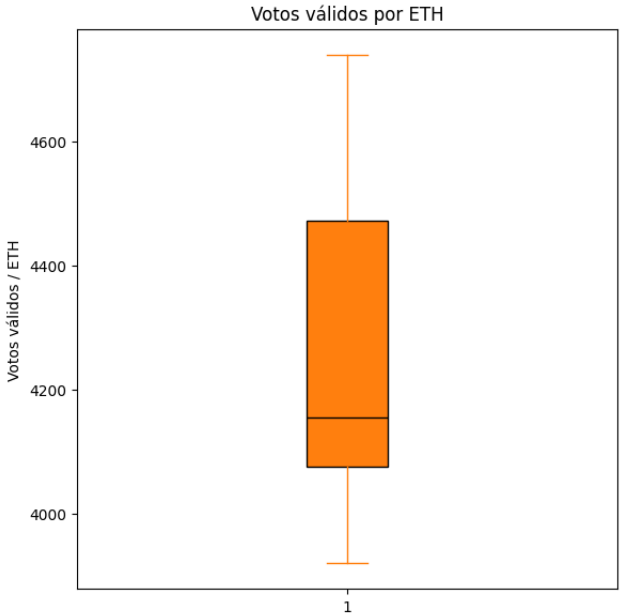
Na visão de custo, a Figura 22 sintetiza quantos votos válidos são obtidos por unidade de Ether. Os valores concentram-se acima dos 4000 votos por ETH, sugerindo boa eficiência. E a medida que o número de votantes cresce, o custo por voto tende a se manter baixo, desde que as operações de *commit* e *reveal* sejam bem agregadas.

Figura 21 – Distribuição da taxa de votos por segundo nas simulações



Fonte: Elaboração dos autores

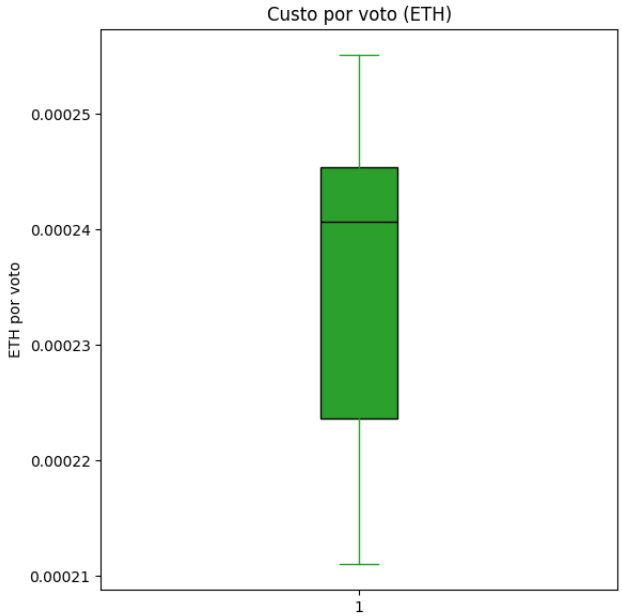
Figura 22 – Votos válidos por unidade de ether consumida em gás



Fonte: Elaboração dos autores

Essa visão fica melhor pela Figura 23, que explicita diretamente o custo por voto em ETH. Onde indicam um custo estável, na ordem de 10^{-4} ETH por voto, com baixa dispersão entre os cenários simulados, o que reforça a previsibilidade do gasto e facilita a estimativa de custo total.

Figura 23 – Custo unitário em ether por voto válido

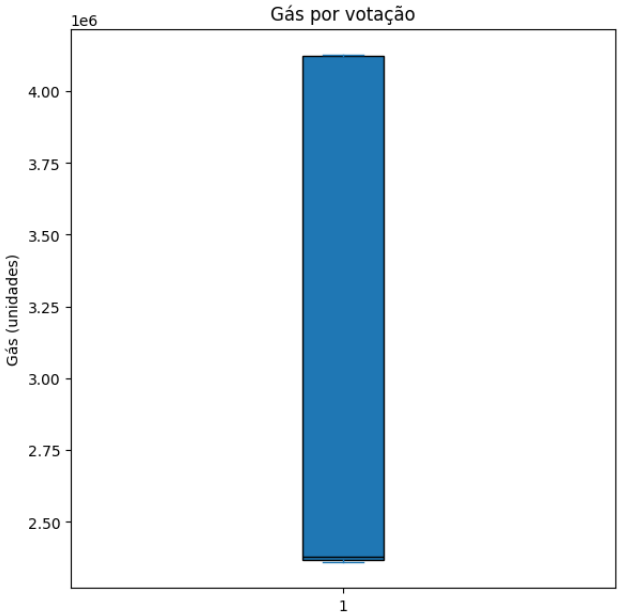


Fonte: Elaboração dos autores

Por fim, a Figura 24 mostra o consumo total de gás por votação, incluindo implan-

tação dos contratos, fase de *commit*, de *reveal* e auditoria. Os valores permanecem na ordem de 10^6 unidades de gás, com mudanças a partir de cenários com maior quantidade de votos. Ainda assim, a variação observada permanece dentro de um intervalo desejável, indicando que o custo fixo de preparação não inviabiliza execuções repetidas do protocolo.

Figura 24 – Consumo total de gás por instância de votação



Fonte: Elaboração dos autores

4.5 Interpretação frente aos objetivos do projeto

Os resultados obtidos indicam que os objetivos funcionais do sistema foram atendidos: o fluxo *commit–reveal* opera de forma íntegra e reproduzível, as janelas temporais são respeitadas, e a elegibilidade pode ser acoplada a um título transferível (ERC/721), viabilizando regras de governança mais ricas. Do ponto de vista não funcional, a arquitetura preserva auditabilidade e recontagem independente, pilares essenciais para confiança institucional. Métricas operacionais concretas (tempo e gás) devem ser registradas conforme a Tabela 4 no ambiente-alvo de implantação, compondo a base para projeção de custo e escala.

4.5.1 Evidências, artefatos e reprodutibilidade

A validação do sistema produz evidências on-chain que confirmam o correto funcionamento das fases de envio, revelação e apuração. Entre os principais artefatos coletados estão os eventos emitidos pelo contrato e o evento de uso de token; as leituras das funções de contagem e de janelas temporais; as mensagens de erro retornadas em cenários negativos. Esses registros permitem reconstruir o histórico de execução e verificar o cumprimento das invariantes definidas, sem necessidade de dados externos a blockchain.

Os artefatos de teste incluem ainda os estados de armazenamento obtidos entre blocos consecutivos, que possibilitam a recontagem independente dos votos e reforçam a

auditabilidade do processo. Em conjunto, esses dados formam uma trilha verificável que demonstra a integridade e transparência do sistema.

A reprodutibilidade é assegurada pela fixação das versões do compilador Solidity, do ambiente Hardhat e dos scripts de teste e implantação. Essa padronização garante consistência entre execuções, tanto em ambiente local quanto em redes de teste públicas, permitindo que os resultados sejam replicados de forma determinística e comparável.

5 Conclusão

Considerando o objetivo apresentado na Seção 1.2, o presente trabalho atingiu o objetivo de desenvolver uma aplicação capaz de suportar uma votação baseada em blockchain, utilizando simulações integradas para o contexto da prova de conceito em votações condominiais.

Em relação aos objetivos específicos propostos, compreende-se que eles foram atendidos da seguinte forma:

O primeiro objetivo, de especificar e documentar a arquitetura, foi contemplado na Seção 3.4, em que a solução é descrita por meio de diagramas e textos que detalham a estrutura do projeto e o seu modo de funcionamento. Esse registro arquitetural é relevante porque oferece uma visão consolidada dos componentes e de suas interações, facilitando a análise crítica da solução, a manutenção futura do código e a eventual extensão do sistema para novos cenários de votação, sem perder a rastreabilidade das decisões de projeto.

O segundo objetivo, de implementar uma prova de conceito em blockchain privada com interface web e *backend* descentralizado, materializou-se na construção da POC sobre Hardhat (rede local), preparada para implantação em testnet, com scripts de implantação e execução. O contrato de votação implementa o protocolo *commit-reveal*, com verificação ECDSA de credenciais e controle explícito das fases de *commit*, *reveal* e *finalize*, enquanto a interface web consome o resultado on-chain e apresenta pauta, opções e contagem de votos. Esse resultado é importante porque demonstra, de forma operacional, a viabilidade técnica do modelo proposto, permitindo observar o comportamento da solução em um ambiente executável e reduzindo a distância entre o conceito teórico e uma aplicação efetivamente utilizável.

O terceiro objetivo, de elaborar um relatório de cenários de uso com simulações integradas, foi atendido pela integração de um gerador de cenários com LLM (Gemma, via LM Studio), responsável pela construção de *prompts* que originam diferentes configurações de votação; esses cenários são validados, executados e têm seus eventos e resultados registrados em arquivos JSON, permitindo análise sistemática e ampliando a cobertura de temas, tamanhos de votantes e número de opções, com menor esforço manual. Esse objetivo é relevante porque a diversidade de cenários exercita a solução em condições variadas, aumentando a confiança de que o comportamento observado não está restrito a um único exemplo didático e fornecendo uma base mais rica para discutir limitações, pontos fortes e possibilidades de generalização para outros contextos de assembleias.

Por fim, o quarto objetivo, de conduzir experimentos de votação na POC coletando métricas de desempenho, segurança e usabilidade, foi contemplado pela inclusão de medições de tempo e validações de segurança na geração de cada JSON associado às simulações, de modo que o próprio resultado produzido sirva como fonte direta para monitorar e avaliar o comportamento da solução nesses três eixos. A coleta dessas métricas se mostra fundamental porque oferece evidências quantitativas e qualitativas sobre o custo de operação, as garantias de integridade e o impacto da solução na experiência de uso, subsidiando decisões futuras sobre adoção, ajustes de arquitetura e evolução do protótipo para um ambiente de produção.

Em síntese, os quatro objetivos específicos para a prova de conceito foram atendidos com evidências no código, nos dados gerados e nas seções do texto, articulando um ciclo completo que vai da concepção arquitetural à avaliação prática. Conforme descrito a seguir, a Seção 5.1 apresenta possibilidades de expansão e refinamento do projeto, indicando caminhos para aprofundar as contribuições deste trabalho em contextos reais de deliberação coletiva.

5.1 Trabalhos Futuros

Para este trabalho, identificam-se algumas possibilidades para adições futuras, como forma de melhorias.

As melhorias incluem:

- O uso de assinaturas cegas (*Blind Signature*). Permitiria que uma autoridade assine uma credencial sem conhecer seu conteúdo, garantindo outra camada de anonimato ao votante, mesmo durante o processo de emissão.
- Implementação de provas de conhecimento zero (*Zero-Knowledge Proofs*) para validação de credenciais e votos sem revelar informações sensíveis.
- Integração de contratos de segunda camada para otimizar taxas de gás e permitir escalabilidade em ambientes com maior número de votantes.
- Extensão do suporte multi-token, possibilitando votações ponderadas por stake ou por classes distintas de credenciais.
- Ampliação do escopo de testes automatizados e integração contínua em testnets públicas para validação de desempenho em larga escala.

Referências

AAAS. *American Association for the Advancement of Science - Internet Online Voting*. 2020. <https://www.aaas.org/epi-center/internet-online-voting>. Acesso em: 29 abr. 2025. Citado na página 18.

ALMEIDA, R. A. d. *Inovação na administração pública federal*. 2024. <https://bibliotecadigital.economia.gov.br/bitstream/123456789/531661/1/Inovacao_na_Administracao_Publica_Federal.pdf>. Citado na página 12.

ALQARNI, M. et al. Use of blockchain-based smart contracts in logistics and supply chains. *Electronics*, v. 12, p. 1340, 03 2023. Citado na página 36.

ALVES, P. H. C. et al. Desmistificando blockchain: Conceitos e aplicações. In: MACIEL, C.; VITERBO, J. (Ed.). *Computação e Sociedade: A Tecnologia*. Cuiabá: EdUFMT Digital, 2020. p. 166–197. Acesso em: 19 maio 2025. Disponível em: <https://www-di.inf.puc-rio.br/~kalinowski/publications/AlvesLNRLK20.pdf>. Citado 6 vezes nas páginas 30, 31, 32, 33, 36 e 37.

ANDROULAKI, E. et al. *Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains*. 2018. <https://arxiv.org/pdf/1801.10228>. Acesso em: 28 maio 2025. Citado na página 32.

BAMBARA, J. J.; ALLEN, P. R. *Blockchain: a practical guide to developing business, law and technology solutions*. New York: McGraw-Hill Education, 2018. 1 p. Disponível em: <https://www.google.com.br/books/edition/Blockchain_A_Practical_Guide_to_Developi/z5hIDwAAQBAJ?hl=pt-BR&gbpv=0&bsq=Blockchain:%20a%20practical%20guide%20to%20developing%20business,%20lawand%20technology%20solutions>. Citado 4 vezes nas páginas 28, 29, 30 e 31.

BOMMASANI, R. et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021. Disponível em: <https://arxiv.org/abs/2108.07258>. Citado na página 14.

BOS, J. W. et al. *Elliptic Curve Cryptography in Practice*. 2013. Acesso em: 20 maio 2025. Disponível em: <https://eprint.iacr.org/2013/734.pdf>. Citado na página 22.

BRASIL. *Lei n.º 4.591, de 16 de dezembro de 1964 — Dispõe sobre o condomínio em edificações e as incorporações imobiliárias*. 1964. <https://www.planalto.gov.br/ccivil_03/leis/l4591.htm>. Acesso em: 29 abr. 2025. Citado na página 18.

BRASIL. *Lei n.º 10.406, de 10 de janeiro de 2002 — Institui o Código Civil*. 2002. <https://www.planalto.gov.br/ccivil_03/leis/2002/l10406.htm>. Acesso em: 29 abr. 2025. Citado na página 18.

BRASIL. *Câmara dos Deputados - Modalidades de votação*. s.d. <https://educacaoadistancia.camara.leg.br/clique_regimento/card/189>. Acesso em: 29 abr. 2025. Citado na página 16.

BROWN, T. B. et al. Language models are few-shot learners. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [s.n.], 2020. Disponível em: <https://arxiv.org/abs/2005.14165>. Citado na página 20.

BUCHANAN, W. *The Wonderful World of Elliptic Curve Cryptography*. 2018. Acesso em: 27 maio 2025. Disponível em: <https://medium.com/coinmonks/the-wonderful-world-of-elliptic-curve-cryptography-b7784acdef50>. Citado na página 24.

BUTERIN, V. *Why Proof of Stake*. 2020. Online essay. Acesso em: <inserir data de acesso>. Disponível em: <https://vitalik.eth.limo/general/2020/11/06/pos2020.html>. Citado na página 34.

Capitol Technology University. *Voting Technology: How New Tech is Being Used in the Election Process*. 2023. <https://www.captechu.edu/blog/voting-technology-how-new-tech-being-used-election-process>. Acesso em: 29 abr. 2025. Citado 2 vezes nas páginas 16 e 17.

CASINO, F.; DASAKLIS, T. K.; PATSAKIS, C. A systematic literature review of blockchain-based applications: Current status, classification and open issues. *Telematics and Informatics*, 2019. Disponível em: <https://doi.org/10.1016/j.tele.2018.11.006>. Citado 2 vezes nas páginas 32 e 33.

CASTRO, M.; LISKOV, B. Practical byzantine fault tolerance. In: *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI)*. New Orleans, USA: USENIX Association, 1999. p. 173–186. Acesso em: 22 maio 2025. Disponível em: <http://pmg.csail.mit.edu/papers/osdi99.pdf>. Citado na página 33.

CIPA. *Assembleias virtuais: uma realidade nos condomínios*. 2023. <https://www.cipa.com.br/assembleias-virtuais-uma-realidade-nos-condominios/>. Citado na página 14.

Cointelegraph. *A Beginner's Guide to Understanding the Layers of Blockchain Technology*. 2025. Acessado em 19 out. 2025. Disponível em: <https://cointelegraph.com/learn/articles/a-beginners-guide-to-understanding-the-layers-of-blockchain-technology>. Citado 2 vezes nas páginas 32 e 33.

Cyfrin. *Keccak256*. 2025. Acessado em 23 out. 2025. Disponível em: <https://www.cyfrin.io/glossary/keccak256>. Citado 2 vezes nas páginas 28 e 29.

Cysne Administradora. *Como funciona a votação em assembleia?* 2023. <https://cysne.com.br/como-funciona-votacao-em-assembleia/>. Acesso em: 29 abr. 2025. Citado na página 18.

DIFFIE, W.; HELLMAN, M. E. New directions in cryptography. *IEEE Transactions on Information Theory*, v. 22, n. 6, p. 644–654, 1976. Disponível em: <https://ee.stanford.edu/~hellman/publications/24.pdf>. Citado na página 22.

ELECTIONGUARD. *Verifiability*. s.d. Acesso em: 27 maio 2025. Disponível em: <https://www.electionguard.vote/concepts/Verifiability/>. Citado na página 16.

ENTRIKEN, W. et al. *ERC-721: Non-Fungible Token Standard*. 2018. Ethereum Improvement Proposal 721. Acesso em: 20 nov. 2025. Disponível em: <https://eips.ethereum.org/EIPS/eip-721>. Citado na página 39.

Ethereum Foundation. *O que é Ethereum?* 2025. Acessado em 18 out. 2025. Disponível em: <https://ethereum.org/pt-br/what-is-ethereum/>. Citado na página 38.

Ethereum Foundation. *Token standards*. 2025. Acesso em: 20 nov. 2025. Disponível em: <https://ethereum.org/pt-br/developers/docs/standards/tokens/>. Citado na página 39.

FAHIM, S.; RAHMAN, S. K.; MAHMOOD, S. Blockchain: A comparative study of consensus algorithms pow, pos, poa, pov. *International Journal of Mathematical Sciences and Computing*, MECS Press, v. 9, n. 3, p. 46–57, August 2023. Autores afiliados à American International University–Bangladesh. Autor correspondente: Sharfuddin Mahmood. Disponível em: <https://www.mecs-press.org/ijmsc/ijmsc-v9-n3/IJMSC-V9-N3-4.pdf>. Citado na página 44.

FIGUEIRA, A.; VAZ, G. Survey on synthetic data generation, evaluation methods and GANs. *Mathematics*, v. 10, n. 15, p. 2733, 2022. Disponível em: <https://www.mdpi.com/2227-7390/10/15/2733>. Citado 2 vezes nas páginas 14 e 19.

Gemma Team. Gemma 2: Improving open language models at a practical size. *Google DeepMind Technical Report*, 2024. Disponível em: <https://arxiv.org/abs/2408.00118>. Citado 2 vezes nas páginas 20 e 21.

GOODMAN, L. C. S. N. Reducing the cost of voting: An evaluation of internet voting's effect on turnout. *British Journal of Political Science*, 2020. Acesso em: 29 abr. 2025. Disponível em: <https://www.cambridge.org/core/journals/british-journal-of-political-science/article/reducing-the-cost-of-voting-an-evaluation-of-internet-votings-effect-on-turnout/6FF8DA77C59806F0175656D66DE66907>. Citado 2 vezes nas páginas 17 e 18.

HABER, S.; STORNETTA, W. S. How to time-stamp a digital document. *Journal of Cryptology*, v. 3, n. 2, p. 99–111, 1991. Citado na página 35.

HANKERSON, D.; MENEZES, A.; VANSTONE, S. Elliptic curve cryptography. In: TILBORG, H. C. A. V.; JAJODIA, S. (Ed.). *Encyclopedia of Cryptography and Security*. Springer, 2011. p. 397–402. Acesso em: 20 maio 2025. Disponível em: <https://link.springer.com/doi/10.1007/978-1-4419-5906-5_245>. Citado 2 vezes nas páginas 22 e 23.

HARDJONO, T.; LIPTON, A.; PENTLAND, A. Towards a design philosophy for interoperable blockchain systems. 2018. Disponível em: <https://arxiv.org/pdf/1805.05934>. Citado na página 34.

HASAN, M. K. et al. Lightweight cryptographic algorithms for guessing attack protection in complex internet of things applications. *Complexity*, v. 2021, p. 1–13, 2021. Disponível em: <https://onlinelibrary.wiley.com/doi/10.1155/2021/5540296>. Citado na página 22.

HJáLMARSSON, F. et al. Blockchain-based e-voting system. In: *2018 IEEE 11th International Conference on Cloud Computing (CLOUD)*. [S.l.: s.n.], 2018. p. 983–986. Acesso em: 28 maio 2025. Citado na página 40.

JAFAR, U.; AZIZ, M. J. A.; SHUKUR, Z. Blockchain for electronic voting system—review and open research challenges. *Sensors*, MDPI, v. 21, n. 17, p. 5874, 2021. Disponível em: <https://www.mdpi.com/1424-8220/21/17/5874>. Citado na página 37.

KESSLER, G. C. *An overview of cryptography*. 2025. <https://www.garykessler.net/library/crypto.html>. Citado 4 vezes nas páginas 21, 22, 26 e 27.

KING, S.; NADAL, S. Ppcoin: Peer-to-peer crypto-currency with proof-of-stake. 2012. Whitepaper original que introduz o modelo de Proof-of-Stake como alternativa ao Proof-of-Work. Disponível em: <https://decred.org/research/king2012.pdf>. Citado na página 34.

LUU, L. et al. Making smart contracts smarter. In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS '16)*. [s.n.], 2016. p. 254–269. Disponível em: <https://eprint.iacr.org/2016/633>. Citado na página 26.

MOLLAH, M. B.; ZHAO, J.; NIYATO, D. *Blockchain Implementation Quality Challenges: A Literature Review*. 2017. <https://www.researchgate.net/publication/320127088_Blockchain_Implementation_Quality_Challenges_A_Literature_Review>. Acesso em: 22 maio 2025. Citado na página 35.

MOURA, L. M. F. d.; BRAUNER, D. F.; JANISSEK-MUNIZ, R. Blockchain e a perspectiva tecnológica para a administração pública: Uma revisão sistemática. *RAC - Revista de Administração Contemporânea*, v. 24, n. 3, p. 259–274, 2020. Disponível em: <https://www.scielo.br/j/rac/a/fDJkpGFF4gH8xmgnLCYRB8z/>. Citado 3 vezes nas páginas 12, 13 e 14.

MUGICA, A. Internet voting in estonia: from apathy to efficiency. *European View*, Springer, v. 14, n. 1, p. 39–45, 2015. Disponível em: <https://journals.sagepub.com/doi/10.1007/s12290-015-0355-5>. Citado na página 17.

NAKAMOTO, S. *Bitcoin: A peer-to-peer electronic cash system*. 2008. <https://bitcoin.org/bitcoin.pdf>. Citado 5 vezes nas páginas 13, 29, 30, 31 e 33.

NARAYANAN, A. et al. *Bitcoin and Cryptocurrency Technologies: A Comprehensive Introduction*. Princeton University Press, 2016. Disponível em: <https://www.google.com.br/books/edition/Bitcoin_and_Cryptocurrency_Technologies/LchFDAAAQBAJ?hl=pt-BR&gbpv=0>. Citado na página 35.

NETO, J. A. d. L.; VASCONCELOS, R. O. Proposta de modelo para votação eletrônica utilizando blockchain e contratos inteligentes. *Texto Livre*, Universidade Federal de Sergipe, v. 18, 2025. Acesso em: 28 maio 2025. Disponível em: <https://doi.org/10.1590/1983-3652.2025.50815>. Citado 2 vezes nas páginas 12 e 39.

NIST. *Guide to General Server Security*. Gaithersburg, MD, 2008. Disponível em: <https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-123.pdf>. Citado na página 30.

NIST. *NIST Selects Winner of Secure Hash Algorithm (SHA-3) Competition*. 2012. Acesso em: 19 maio 2025. Disponível em: <https://www.nist.gov/news-events/news/2012/10/nist-selects-winner-secure-hash-algorithm-sha-3-competition>. Citado na página 28.

NIST. *FIPS PUB 180-4: Secure Hash Standard (SHS)*. Gaithersburg, MD, 2015. Acesso em: 19 maio 2025. Disponível em: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>. Citado na página 27.

NITAHARA, A. *Estudo mostra que pandemia intensificou uso das tecnologias digitais*. 2021. <https://agenciabrasil.ebc.com.br/geral/noticia/2021-11/estudo-mostra-que-pandemia-intensificou-uso-das-tecnologias-digitais>. Citado na página 12.

Nomic Foundation. *Hardhat Documentation*. 2024. <https://hardhat.org/docs>. Ferramenta de desenvolvimento que padroniza o ciclo de vida de contratos inteligentes (compilação, testes, deploy e verificação). Citado na página 38.

Nomic Foundation. *Getting Started | Hardhat*. 2025. Acessado em 18 out. 2025. Disponível em: <https://hardhat.org/docs/getting-started>. Citado na página 39.

OETTLER, M. *Commit-Reveal*. 2021. Acesso em: 27 maio 2025. Disponível em: <https://blockchain-academy.hs-mittweida.de/courses/solidity-coding-beginners-to-intermediate/lessons/solidity-11-coding-patterns/topic/commit-reveal/>. Citado na página 26.

PAAR, C.; PELZL, J. *Understanding cryptography: a textbook for students and practitioners*. 1. ed. Berlin: Springer, 2009. Disponível em: <https://www.google.com.co/books/edition/Understanding_Cryptography/f24wFELSzkoC>. Citado 8 vezes nas páginas 12, 13, 21, 23, 24, 25, 27 e 28.

PEREIRA, R. L.; ANGONESE, A. Utilização de blockchain em votações eletrônicas para reuniões de condomínios. *Revista UNIFESO – Caderno UNIFESOTEC*, v. 1, n. 1, 2023. Disponível em: <https://revista.unifeso.edu.br/index.php/unifesotec/article/download/4643/1779>. Citado 3 vezes nas páginas 12, 13 e 14.

SANTOS, G. G. *Smart contracts [manuscrito]: conceitos, limitações e potencialidades*. Dissertação (Dissertação (Mestrado em Direito)) — Universidade Federal de Minas Gerais, Faculdade de Direito, Belo Horizonte, 2022. Disponível em: <https://repositorio.ufmg.br/handle/1843/57345>. Citado 2 vezes nas páginas 25 e 26.

SHAMOUT, O. *System Design Implementation of Hyperledger Fabric E-Voting System: A Case Study of the Lebanese American University*. 2023. Acesso em: 28 maio 2025. Disponível em: <https://doi.org/10.13140/RG.2.2.31957.95205>. Citado na página 40.

SOLIDITY. *Solidity Documentation*. 2024. <https://docs.soliditylang.org>. Documentação oficial da linguagem e padronização de ABI e interface. Citado 2 vezes nas páginas 38 e 39.

SOUSA, G. d. M. EficiÊncia e transparÊncia na administraÇÃo pÚblica: Desafios e perspectivas para uma gestÃo responsÁvel e inovadora. *REVISTA FOCO*, v. 17, n. 11, p. e7044, 2024. Disponível em: <https://ojs.focopublicacoes.com.br/foco/article/view/7044>. Citado na página 12.

STADLER, T.; OPRISANU, B.; TRONCOSO, C. Synthetic data – a privacy mirage? In: *31st USENIX Security Symposium (USENIX Security 22)*. [s.n.], 2022. Disponível em: <https://www.usenix.org/system/files/sec22-stadler.pdf>. Citado na página 19.

STEVENS, M. et al. *The first collision for full SHA-1*. S. l.: [s.n.], 2017. Acesso em: 18 maio 2025. Disponível em: <https://shattered.io/static/shattered.pdf>. Citado na página 27.

TABATABAEI, S. M.; VITENBERG, R.; VEERARAGHAVAN, M. Understanding blockchain: Definitions, architecture, design, and system comparison. *arXiv preprint arXiv:2207.02264*, 2022. Disponível em: <https://arxiv.org/abs/2207.02264>. Citado 3 vezes nas páginas 29, 31 e 35.

TEDYYANA, A. et al. Transforming the voting process integrating blockchain into e-voting for enhanced transparency and securiy. *TELKOMNIKA (Telecommunication Computing Electronics and Control)*, v. 22, 2024. Citado na página 41.

TSE. *Tribunal Superior Eleitoral - Sistemas eleitorais brasileiros*. 2021. <https://www.tse.jus.br/institucional/escola-judiciaria-eleitoral/publicacoes/revistas-da-eje/artigos/revista-eletronica-eje-n.-4-ano-4/sistemas-eleitorais-brasileiros>. Tribunal Superior Eleitoral. Acesso em: 29 abr. 2025. Citado na página 16.

uCondo. *Assembleia ordinária e extraordinária: entenda as diferenças*. 2023. Acesso em: jan. 2026. Disponível em: <https://www.ucondo.com.br/blog/assembleia-ordinaria-e-extraordinaria>. Citado na página 18.

UNIãO, T. de Contas da. Blockchain no setor público: fundamentos, aplicações e desafios regulatórios. *Revista do TCU*, v. 52, n. 147, p. 48–62, 2021. Disponível em: <https://revista.tcu.gov.br/ojs/index.php/RTCU/article/download/1921/1916/3933>. Citado na página 14.

UNIãO, T. de Contas da. *Sumário Executivo – Levantamento de Tecnologia Blockchain*. 2021. <https://portal.tcu.gov.br/data/files/59/02/40/6E/C4854710A7AE4547E18818A8/Blockchain_sumario_executivo.pdf>. Acesso em: 25 maio 2025. Citado 2 vezes nas páginas 13 e 14.

VASWANI, A. et al. Attention is all you need. *Advances in Neural Information Processing Systems*, v. 30, p. 5998–6008, 2017. Disponível em: <https://papers.neurips.cc/paper/7181-attention-is-all-you-need.pdf>. Citado na página 20.

WANG, X.; YIN, Y. L.; YU, H. Finding collisions in the full SHA-1. In: *CRYPTO 2005: 25th Annual International Cryptology Conference, Santa Barbara, California, USA, August 14–18, 2005. Proceedings*. Berlin: Springer, 2005. (Lecture Notes in Computer Science, v. 3621), p. 17–36. Acesso em: 18 maio 2025. Disponível em: <https://iacr.org/archive/crypto2005/36210017/36210017.pdf>. Citado na página 27.

WOOD, G. *Ethereum: A Secure Decentralised Generalised Transaction Ledger (Yellow Paper)*. [S.l.], 2021. Especificação formal do runtime EVM, ABI e execução de contratos. Disponível em: <https://ethereum.github.io/yellowpaper/paper.pdf>. Citado na página 38.

XU, X. et al. A taxonomy of blockchain-based systems for architecture design. In: . [S.l.: s.n.], 2017. Citado 2 vezes nas páginas 37 e 38.

ZHAI, S. et al. Research on the application of cryptography on the blockchain. *Journal of Physics: Conference Series*, v. 1168, n. 3, p. 032077, 2019. Disponível em: <https://iopscience.iop.org/article/10.1088/1742-6596/1168/3/032077>. Citado na página 34.

ZOU, M. J. *Inovação no setor público: a implementação do e-Governo no Brasil*. Dissertação (Dissertação (Mestrado em Administração Pública)) — Universidade do Minho, Braga, 2022. Disponível em: <https://repositorium.sdum.uminho.pt/bitstream/1822/81579/1/Ming%20Juan%20Zou.pdf>. Citado na página 12.

Apêndices

APÊNDICE A – Repositório principal

O projeto principal se encontra disponível no repositório <https://github.com/muosilva/blockchain-voting-api>

APÊNDICE B – Link do repositório com o código para cálculo de métricas

O código utilizado para o cálculo das métricas está disponível em: <https://github.com/henriquemcmedeiros/metrics_blockchain_voting>.

APÊNDICE C – Prompt para geração dos cenários em JSON

System prompt (SYSTEM_PROMPT)

```
Voce cria configuracoes JSON de simulacoes para um contrato commit-reveal de
votacao. Responda exclusivamente com JSON valido.
```

User prompt base (BASE_USER_PROMPT)

```
Regras gerais:
- Responda somente com JSON (sem markdown).
- Utilize o esquema "defaults": ..., "simulations": [...] e mantenha a
estrutura completa.
- O campo "defaults"e obrigatorio: SEMPRE inclua-o com as chaves listadas
abaixo.
- Se um valor nao for solicitado, mantenha o valor sugerido.
- Cada simulacao precisa de id unico, name, options (array de strings) e votes
(array de objetos com optionIndex ou option + label).
- Cada "name"deve agir como um titulo curto explicando claramente o
objetivo/nuance do teste de integracao.
- Limite cada simulacao a no maximo 10 votos e garanta pelo menos 2 votos.
- Opcionalmente acrescente issuerIndex, timing, voterOffset ou output.file
apenas quando fizer sentido.
- Varie os cenarios: altere temas (garagem, reformas, orcamento, obras
emergenciais, sustentabilidade, etc.), numero de votantes (2 a 10 votantes),
quantidade de opcoes (entre 2 e 8) e ajuste os defaults quando fizer sentido.
Evite repetir nomes ou combinacoes de votos.
- Distribua as simulacoes para cobrir o maior numero possivel das situacoes
abaixo (se o total de simulacoes for menor que a lista, priorize na ordem
apresentada):
  1. SimpleVoting com maioria clara.
  2. SimpleVoting com empate ao final.
  3. SimpleVoting com tres ou mais opcoes e quoruns distintos.
  4. SimpleVoting com janela de tempo customizada (timing nao padrao).
  5. TokenizedVoting com useForVoting e mintsPerVoter permitindo commits
multiplos do mesmo votante.
  6. TokenizedVoting definindo token.mintTo explicito para distribuir NFTs
antes do commit.
  7. TokenizedVoting com token.transfers simulando negociacao de NFTs antes do
commit.
  8. Cenarios que alternem issuerIndex e voterOffset para exercitar diferentes
contas.
- Quando produzir TokenizedVoting, sempre inclua "contractName":
"TokenizedVoting"e descreva um bloco "token"coerente (pode combinar mintTo,
```

```
useForVoting, mintsPerVoter, transfers).
- Para token.mintTo e token.transfers, referencie enderecos reais (ou os mesmos
rotulos usados nos votos, como "Votante 1") e nunca use o placeholder
"0x..."nem reticencias.

Defaults sugeridos (copie e ajuste apenas quando necessario):
{
  "issuerIndex": 0,
  "voterOffset": 1,
  "resetStateBetweenSimulations": true,
  "timing": {
    "startDelay": 5,
    "commitDuration": 60,
    "revealDuration": 60
  }
}
```